

Predicting Glasgow's "taps aff" using non-weather proxy signals

| Project Context |

MSc Data Analytics

Ka Hou Ho

202486932

September 2025

Except where explicitly stated, all the work in this dissertation – including any appendices – is my own and was carried out by me during my MSc course. It has not been submitted for assessment in any other context.

Signed: 

Table of Contents

PROJECT CONTEXT	3
<i>Background</i>	3
<i>'Taps-off' Logic and Versions</i>	3
<i>Why is this research important:</i>	4
<i>Research Objectives</i>	4
<i>Project Plan</i>	5

Project Context

Background

Glasgow's "taps aff" is a slang term of describing days that feel warm and inviting enough for people to head outdoors in light clothing. This dissertation, the phrase is operationalised to show a daily yes/no outcome (i.e. is today a tap-aff day?). The question is whether this behaviour can be forecasted one day ahead without using any weather data at all, relying on low-cost public signals when outdoor weather shifts.

The description provided by the supervisor is as follows:

"Taps oan or taps aff" (<https://www.taps-aff.co.uk/>) is a website that reports 1 bit (ie. boolean value) of information about the weather in Glasgow. It is very easy to predict this value using weather data, but your goal for this project is to build a model to predict this value using any data you can find other than weather data."

'Taps-aff' Logic and Versions

The "Taps Aff" website, created by Colin Waddell, provides a binary indicator of whether conditions in Glasgow are sufficiently warm for individuals to remove their shirts, a cultural marker commonly associated with unusually warm weather in the city. While the output is culturally specific, the underlying logic demonstrates a clear progression in the use of environmental data to drive simple, interpretable predictions.

In its initial version, the website relied solely on outdoor temperature data, obtained from an external weather service, to determine whether the threshold for "taps aff" had been met. If the temperature exceeded approximately 17 °C, the site displayed "Taps Aff"; otherwise, it reported "Taps Oan." This simple implementation, though effective, was limited by its narrow reliance on a single weather variable.

Following on Version 2, incorporated additional refinements; extended the logic to include not only temperature but also weather condition codes, recognising that heavy rain, fog, or snow could override a temperature threshold that might justify "taps aff." The site also considered whether it was daytime, acknowledging that even warm evening temperatures might not encourage the same behavioural response.

The most recent version (v3.0), released in 2018, represented a full redevelopment using a modern software stack (Django, Bootstrap). This version implemented a structured decision sequence: first evaluating whether the reported weather conditions were tolerable, then determining whether it was

daytime, and finally applying a calibrated temperature threshold. A minor adjustment was included to capture near-threshold situations, producing messages such as “...but only by a bawhair” when conditions were marginal.

The project demonstrates the possibility of reducing a complex human-environment interaction to a small set of observable indicators. For the purposes of this dissertation, the website provides both the conceptual foundation and the binary target variable against which non-weather proxy models can be developed.

Why is this research important:

- **Understanding behavioural proxies** – If non-weather data can reliably signal “taps aff” conditions, it demonstrates the hidden explanatory power of digital traces of behaviour. This has applications for predicting events where direct measurements are unavailable, delayed, or unreliable.
- **Resilience to data gaps** – In many scenarios, high-quality local weather data may be missing or inaccessible. Developing models that rely on alternative data streams highlights methods for dealing with such gaps, useful in fields from public health (heatwaves, air quality) to transport planning.
- **Cultural and societal relevance** – “Taps aff” has become a recognisable Glasgow cultural understanding of hot weather. Framing the research through this lens makes the study engaging and accessible, while still addressing broader scientific questions about data proxies, prediction, and human-environment interaction.
- **Contribution to data analyst practice** – The project illustrates principles of careful feature engineering, data leakage prevention, and model validation when dealing with unconventional predictors. This study generalises beyond the case study.

Research Objectives

Building on this thought, and following discussions with the project supervisor, a clearer goal of how the project would be structured was created. The use of machine learning methods, data analysis, feature engineering and statistical methods, this project aims to answers these questions below

Four research questions guide the work:

- (RQ1) Can non-weather proxies predict “taps aff” with useful accuracy and calibration on unseen years?

- (RQ2) Where does the signal come from—how much is due to search intent, seasonal context, and mobility?
- (RQ3) Is performance stable across years and calendar regimes?
- (RQ4) Which operating threshold offers a sensible precision–recall trade-off for day-ahead planning?

Project Plan



The timeline shows the project running from mid-June to early September. Green bars mean “on time” and pink bars mark small delays. June was used for reading, scoping, and setting things up (data collection, ethics, project plan). The literature review started then and moved into writing by month-end.

July was mainly data prep and modelling. This part took longer than expected, so a few pink sections appear while cleaning and aligning the feeds and testing thresholds/calibration. August acted as the buffer: advanced modelling finished, and the write-up (methodology plus results & discussion) was done in parallel. The final two weeks were for polishing figures/tables, final checks, and submission. In short, data collection in June, heavy analysis in July, write-up and contingency in August—with minor delays, but project was delivered on schedule.

Predicting Glasgow’s “taps aff” using non-weather proxy signals

| Client Report |

MSc Data Analytics

Ka Hou Ho

September 2025

Executive Summary

This dissertation tests whether Glasgow’s “taps on or taps off” can be predicted without any weather inputs. Instead, it uses public, fast-updating proxies that reflect what people plan and do: Google search interest (e.g., “bbq”, “suncream”), city-wide cycling and pedestrian counts, and Great Britain electricity demand, together with simple calendar signals. Features emphasise recent change (week-over-week movements, short lags, rolling statistics, EMAs) so the model reacts to fresh momentum rather than static levels. All modelling choices are evaluated fairly: hyper-parameters are tuned on 2018–2023 with expanding, time-aware splits, and performance is reported only on an untouched 2024+ hold-out.

A single operating threshold is fixed on the training years by maximising $F\beta$ with $\beta = 0.5$ (slightly favouring precision). This yields a decision threshold $\tau = 0.946$, which is applied unchanged to 2024+. At this cut-off the selected model achieves Accuracy 0.856, Precision 0.746, Recall 0.707, and F1 0.726, with TP = 106, FP = 36, TN = 368, and FN = 44. In plain terms, around three-quarters of the days flagged “taps off” were indeed “taps off”, and the system caught just over seventy percent of all true “taps off” days. These results are materially stronger than simple baselines, indicating that the proxy signals add real predictive value even without direct weather data.

Analyses of where the signal comes from show a consistent pattern: search intent moves early, electricity demand anchors the season, and mobility confirms behaviour - especially around weekends. Year-by-year and seasonal checks indicate that ranking performance is stable, while probability quality behaves as expected with changes in how common “taps off” days are. The model’s probabilities can be used with simple decision bands and a single, transparent threshold, supported by light monitoring and periodic recalibration. Overall, the work delivers a reproducible, proxy-only study that provides useful day-ahead guidance for planning outdoor-sensitive activity in Glasgow.

Acknowledgements

I would like to thank my family and friends for supporting me throughout this project. I would like to also thank USMART community for providing the whole dataset for *Glasgow City Council - Hourly walking counts from automatic cycling counters*. Thanks also to Dr. Jules Hedges for providing direction and supervision.

Table of Contents

EXECUTIVE SUMMARY	1
ACKNOWLEDGEMENTS	2
1.0 INTRODUCTION	6
2.0 LITERATURE REVIEW	8
2.0.1 INTRODUCTION: THE CHALLENGE OF INDIRECT PREDICTION.....	8
2.1 THE THEORETICAL BASIS OF PROXY VARIABLES	9
2.1.1 <i>Proxy Variables in Data Science and Econometrics</i>	9
2.2 HUMAN BEHAVIOUR AS AN INDIRECT WEATHER INDICATOR	11
2.3 METHODOLOGICAL APPROACHES TO INDIRECT FORECASTING	13
2.3.1 <i>Design implications from the Literature</i>	15
2.4 SYNTHESIS AND RESEARCH GAP	16
3.0 METHODOLOGY	18
3.1 DATA SOURCES AND COLLECTION	19
3.2 DATA PRE-PROCESSING AND FEATURE ENGINEERING	20
3.3 MODELS	21
3.4 TRAINING, TUNING, AND EVALUATION	22
3.5 IMPLEMENTATION AND REPRODUCIBILITY	24
3.6 ASSUMPTIONS, RISKS, AND MITIGATIONS	24
4.0 RESULTS & DISCUSSION	25
4.1 EXECUTIVE HIGHLIGHTS	25
4.2 BASELINES & PRIMARY MODEL PERFORMANCE	25
4.3 PROXY CONTRIBUTION	29
4.4 PERFORMANCE OVER TIME AND CALENDARS	31
4.5 WHAT DRIVES THE PREDICTIONS? (INTERPRETABILITY).....	35
4.6 ERROR ANALYSIS & FAILURE MODES.....	39
4.7 LIMITATIONS	41
4.8 SUMMARY OF KEY TAKEAWAYS	42
5.0 CONCLUSION	43
6.0 BIBLIOGRAPHY	45
7.0 APPENDICES	53
APPENDIX A DATA COLLECTION	53
A1) <i>Google Trends - BBQ</i>	53
A2) <i>Google Trends – suncream</i>	56
A3) <i>NESO – Energy Demand & Feature Engineering</i>	59
A4) <i>Mobility data</i>	61
A5) <i>Meteostat (taps-aff label)</i>	64
A6) <i>Incorporating Colin’s Rule V3 – Link</i>	67
APPENDIX B FEATURE ENGINEERING.....	71
B1) <i>Feature Engineering - BBQ</i>	71
B2) <i>Feature Engineering - suncream</i>	74
B3) <i>Feature Engineering - Mobility</i>	77
APPENDIX C – COMBINING FILES (MASTER FILE).....	81

APPENDIX D BASELINE MODEL.....	85
APPENDIX E ADVANCED MODELS.....	88
APPENDIX F HYPERPARAMETER TUNING & HOLD-OUT SCORING	91
<i>F1) Hyper-parameter tuning</i>	91
<i>F2) Hold out scoring</i>	94
<i>F3) HGB Post Calibration</i>	96
APPENDIX G LEAKAGE SAFE & THRESHOLD-TUNED VERSION	100
APPENDIX H FIGURES	103
<i>H1) Figure 2 - ROC Curve & Precision-Recall</i>	103
<i>H2) Figure 3 - Reliability</i>	107
<i>H3) Figure 4 - Metrics Deltas by stage, Table 5 - Contributions of trends, demands & mobility</i>	110
<i>H4) Figure 5 - ROC-AUC by year, Figure 6 - Brier by year, Figure 7 - Reliability by year & Table 6 - slicing metrics</i>	116
<i>H5) Figure 8 - Permutation importance, Figure 9 - SHAP & Table 7 – Case Day</i>	125
<i>H6) Figure 10 - Precision–Recall vs threshold (chosen point), Table 8 - Error taxonomy</i>	132

List of Tables and Figures

Table 1: Data Sources, Coverage and Latency	20
Table 2: Hyper-parameter grids & the selected best	23
Table 3: Baseline vs Model metrics	26
Table 4: HGB performance (post).....	28
Table 5: Contribution of Trends, Demand and Mobility.....	29
Table 6: Slicing Metrics by year, season, weekday vs weekend.....	34
Table 7: Case Day Narratives	38
Table 8: Error breakdown (counts & examples)	39
Figure 1: End to end pipeline (Data – Features – Models – Evaluation).....	19
Figure 2: ROC curve & Precision-Recall curve	27
Figure 3: Reliability Plot – Brier (Pre & Post)	27
Figure 4: Metrics by Stage (AUC/Brier)	30
Figure 5: ROC-AUC by year	32
Figure 6: Brier by year.....	32
Figure 7: Reliability by Year.....	33
Figure 8: Top 20 Permutation Importances.....	36
Figure 9: SHAP summary	37
Figure 10: Precision-Recall vs chosen Threshold.....	40

1.0 Introduction

This dissertation investigates whether a simple, behaviour-based outcome—Glasgow’s “taps off” days—can be predicted without using any meteorological inputs. “Taps off” is treated as a daily yes/no label that reflects when people act as if conditions are warm enough to be outdoors in light clothing. The question is pragmatic: can a city-level model forecast that behaviour using only public, low-cost signals that arrive quickly, rather than licensed weather feeds?

The approach is deliberately indirect. Instead of modelling temperature or rain, the study learns from proxies that tend to move when outdoor comfort improves. Three families are used: (i) search intent (Google Trends for terms such as “bbq” and “suncream”), which often rises before people change plans (Askatas and Zimmermann, 2009; Vosen and Schmidt, 2011; Choi and Varian, 2012); (ii) urban mobility (aggregated cycling and pedestrian counts for Glasgow), which signals revealed activity and is known to co-vary with milder, drier conditions (Böcker *et al.*, 2013; Gebhart and Noland, 2014); and (iii) electricity demand at Great Britain level, which provides slow seasonal context related to heating and lighting (Pardo *et al.*, 2002). A programmatic “taps off” label based on a published comfort rule is used only as ground truth; it is not fed back as a predictor.

Methodologically, this is framed as a time-ordered, daily binary classification task over 2018–2025. Feature engineering emphasises change rather than level—short lags, week-over-week differences, rolling statistics, and exponential moving averages—so the model reacts to fresh momentum and is less sensitive to platform re-scaling (Hyndman and Athanasopoulos, 2018). Calendar information (weekends, holidays) and smooth seasonality terms are included because behaviour is strongly cyclical. All features are constructed with information available by day t to prevent leakage (Bergmeir and Benítez, 2012). Because proxy relationships can drift (as illustrated by Google Flu Trends; Lazer *et al.*, 2014), the evaluation is explicitly out-of-time.

Model selection balances transparent baselines with non-linear capacity. Majority class, calendar-only, and logistic regression baselines are compared with tree ensembles (Random Forest; Histogram-based Gradient Boosting), which capture interactions among engineered signals and handle heterogeneous tabular features (Breiman, 2001; Friedman, 2001; Shwartz-Ziv and Armon, 2022). Tuning is carried out with expanding time-series splits on 2018–2023; a strictly chronological 2024+ window is held out for final assessment (Bergmeir and Benítez, 2012). Probabilities are calibrated on the training years when feasible, because decision-makers require well-scaled scores—not rankings alone—for threshold policies; Brier score and reliability curves are therefore reported alongside ROC/PR-AUC (Zadrozny and Elkan, 2002; Stehouwer *et al.*, 2025).

Four research questions guide the work:

- (RQ1) Can non-weather proxies predict “taps off” with useful accuracy and calibration on unseen years?
- (RQ2) Where does the signal come from—how much is due to search intent, seasonal context, and mobility?
- (RQ3) Is performance stable across years and calendar regimes?
- (RQ4) Which operating threshold offers a sensible precision–recall trade-off for day-ahead planning?

The intended contributions are practical and methodological. Practically, the dissertation delivers a reproducible, weather-free pipeline that turns public data into calibrated daily probabilities and a simple operating rule (green/amber/red). Methodologically, it demonstrates a leakage-safe evaluation with expanding splits, a chronological hold-out, incremental feature-set comparisons, year-by-year backtests, and interpretable analyses (permutation importance and case-day narratives). Ethical considerations are straightforward: all inputs are aggregated and public, with no personal data. The model is predictive rather than causal, and the maintenance plan recognises that proxies drift; periodic calibration and light monitoring are part of the proposed governance (Hyndman and Athanasopoulos, 2021; Lazer et al., 2014).

2.0 Literature Review

2.0.1 Introduction: The Challenge of Indirect Prediction

This dissertation investigates whether a simple, binary, weather-related behaviour—Glasgow’s “taps aff”—can be predicted without using meteorological data. The research question is intentionally stark: on a given day, will people behave as if it is warm enough to be outdoors in light clothing (taps aff), or not? The constraint is equally clear: the model must rely solely on non-weather signals that are publicly available, inexpensive, and timely. As a master’s student, the researcher prioritises transparency and reproducibility, selecting data sources and modelling steps that are auditable and operationally practical.

The study adopts an indirect, proxy-based approach. Rather than modelling temperature, wind, or rain directly, it tests whether behavioural and system-level traces that respond to outdoor comfort can forecast the taps-aff outcome. Three proxy families are examined: (i) search intent (e.g., Google Trends for “bbq” and “suncream”), which often rises ahead of outdoor social activity; (ii) urban mobility (city-wide cycling and pedestrian counts), which reflects actual time spent outside; and (iii) electricity demand, which encodes strong seasonal patterns linked to heating, lighting, and occupancy. If these signals move predictably on days that “feel like” taps aff, a classifier may learn the pattern without ingesting weather features.

Recognising the risks inherent in proxies—construct validity, concept drift, and calendar confounders—the researcher emphasises change-focused features (lags, week-over-week deltas, rolling/EMA summaries), time-aware evaluation (expanding training windows with an out-of-time hold-out), and probability calibration so outputs support clear decision thresholds. The objective is not to assert causation, but to test predictive usefulness and operational reliability under realistic data latencies.

The remainder of the review develops this case. Section 2 sets out the theoretical basis for proxy variables and synthesises evidence that human behaviour co-moves with weather in ways captured by digital and system traces. Section 3 surveys methodological choices for time-series classification with proxies—feature engineering, model families, validation, and calibration. Section 4 synthesises the literature and articulates the research gap addressed here: a calibrated, proxy-only predictor of “taps aff” for Glasgow, evaluated with strong leakage control, temporal robustness checks, and rigorous calibration.

2.1 The Theoretical Basis of Proxy Variables

2.1.1 Proxy Variables in Data Science and Econometrics

What is a proxy variable and why use one?

In empirical work, a proxy variable is an observable signal used in place of a latent or costly-to-measure construct (Adam and Goyal, 2008; Lunsford, 2015; Miao *et al.*, 2018; Seltzer, 2021). Classic econometrics treats proxies as remedies for measurement error or omitted, hard-to-observe drivers (Baltagi, 2008; Deaner, 2023), for example, using commodity rail shipments as a stand-in for industrial activity (Sowers *et al.*, 2017; Miller and Wilson, 2023). In modern data science, proxies frequently come from digital traces (Miller *et al.*, 2021), *such as search queries, mobility counts, platform usage, and system logs* (Lin *et al.*, 2022) *like energy demand and traffic sensors*. The logic is practical: proxies are timely, cheap, and granular, and if engineered and validated well can yield accurate predictions of the latent outcome even when direct measurements are off-limits.

How proxies are made predictive: multiple weak signals and weak supervision.

Each proxy is only a noisy, partial view of the truth. Combining several complementary proxies — search intent, mobility, and electricity demand — often outperforms any single one because their errors are partly independent (Paul and Schäfer-Neth, 2005; Birks and Birks, 2006; Lubotsky and Wittenberg, 2006). This idea aligns with the literature on weak supervision and distant supervision: multiple imperfect signals can be aggregated to produce a more reliable target or predictor. Practically, this motivates the use of plural sources and engineering week-over-week deltas and short Exponential Moving Averages (EMAs) that focus the model on behaviour shifts rather than absolute levels, which can be unstable across platforms and years (Ting *et al.*, 2011; Klobardanz *et al.*, 2022).

Evidence that proxies can nowcast real outcomes.

A first strand shows that web search anticipates official indicators. Askitas and Zimmermann (2009) argue that job-search queries lead unemployment filings; using out-of-time test windows and reporting forecast-error reductions – directly impacting the Root Mean Square Error (RMSE) or Mean Absolute Error (MAE) – against autoregressive baselines, they demonstrate genuine nowcasting value. Choi and Varian (2012) extend this to retail and travel: in rolling-origin evaluations, models that include Google Trends achieve lower MAE/RMSE than AR models, supporting the claim that digital intent captures imminent behaviour. A complementary strand compares proxies: Vosen and Schmidt (2011) show that search indices outperform survey sentiment for private consumption on out-of-sample forecast errors, emphasising timeliness as a crucial advantage. In other words, search operates as a leading indicator of warm-day behaviour that a classifier can exploit without meteorological inputs.

A second strand supports system and mobility proxies. In energy-econometrics, national electricity demand exhibits pronounced seasonal structure driven by heating, lighting and occupancy, with higher winter loads and lower summer loads documented in UK statistics and operator outlooks (Taylor, 2010; De Felice *et al.*, 2015). Short-term load-forecasting studies encode these recurring intraday/weekly/annual patterns (Kwon *et al.*, 2019) using double/triple-seasonal methods (Caiado, 2010; Arora and Taylor, 2013) and typically validate on rolling or expanding windows (Feng *et al.*, 2024), reporting stable RMSE/MAPE across years (Taylor, 2003; Hong and Fan, 2016). In this dissertation, electricity demand is therefore treated as seasonal context — a slow-moving backdrop that improves calibration rather than a direct measure of outdoor comfort (Hong and Fan, 2016). Essentially, electricity demand helps the model “know the season” without reading temperature directly.

Urban analytics likewise shows that active-travel counts respond to weather and the calendar: bicycle volumes rise on mild, dry days and amplify at weekends/holidays, with effects estimated via Poisson/negative-binomial count models and assessed out-of-time using log-likelihood or MAE (Wadud, 2014; El-Assi *et al.*, 2017). Daylight availability further increases cycling, especially around commuter peaks (Wessel, 2022), while pedestrian studies report similar temperature and precipitation sensitivities using over-dispersed count models (Wang *et al.*, 2014; Jaber and Csonka, 2023). In a proxy-only setting, mobility thus contributes local, revealed behaviour—confirming that people are actually outside—while demand supplies the seasonal baseline; together these signals complement search intent, which tends to move earlier in the decision cycle (Hong and Fan, 2016; El-Assi *et al.*, 2017) Mobility provides contemporaneous confirmation that intent translated into outdoor activity.

Validity threats: construct validity, confounding and drift.

Proxies can fail on construct validity: a spike in “bbq” searches might be driven by a supermarket campaign, not outdoor comfort. They can also be thrown off by calendar confounders (bank holidays, payday effects, major events) that move behaviour irrespective of weather. The practical mitigation is to (i) build calendar features (day-of-week, holidays, school breaks, month, Fourier seasonality), (ii) emphasise week-over-week deltas and short EMAs so the model tracks shocks rather than long-run level shifts, and (iii) compare incremental feature sets (Trends → +Demand → +Mobility) to see what truly adds signal.

The best-known cautionary tale is Google Flu Trends. Lazer *et al.* (2014) show that media salience and platform/UI changes induced concept drift, leading to large over-predictions in 2012–13 despite earlier success. Their core message is methodological: treat proxy-to-target mappings as non-stationary; use time-aware validation (rolling/expanding splits, out-of-time hold-outs), periodic retraining, and—when serving probabilities—calibration to keep scores meaningful as conditions change.

Econometric perspective: measurement error, attenuation, and proxies vs instruments.

Standard texts on measurement error note that noisy proxies can cause attenuation bias in linear estimation: coefficients on mismeasured regressors are biased toward zero (Angrist and Pischke, 2009; Wooldridge, 2010). Prediction tasks are less concerned with unbiased parameters than with out-of-time accuracy, but the same insight motivates (i) using multiple proxies to reduce variance, (ii) regularisation/trees that cope with multicollinearity among lag/roll features, and (iii) evaluation on future periods where bias/variance trade-offs are revealed operationally, not just in-sample.

Distinguish proxies from instruments: instruments explain exogenous variation to identify causal effects; proxies supply predictive signal for a latent construct. This dissertation pursues the latter (prediction), not causal identification.

Taken together, these results justify a multi-proxy strategy; Section 2.2 translates this evidence into the Glasgow context and the specific signals used here.

Practical recipe that emerges from the literature.

Across econometrics and data science, effective proxy modelling tends to converge on a few rules:

1. Use multiple, complementary sources to average idiosyncratic noise.
2. Engineer leakage-safe dynamics (lags, deltas, rolling stats, EMAs) and calendar/seasonality encodings.
3. Validate out-of-time (expanding/rolling splits; final chronological hold-out) and report both discrimination and calibration (e.g., ROC-AUC + Brier + reliability).
4. Monitor and maintain (back-tests by year; simple drift checks; periodic retraining).
5. Prefer tree ensembles for tabular, mixed-scale proxy features; keep logistic and seasonal baselines for context.

These principles directly inform the design of a proxy-only “taps off” predictor: combine search, mobility, and demand; focus on changes; evaluate with expanding TimeSeries Split and an out-of-time hold-out; calibrate probabilities; and document incremental value via feature-set comparisons and year-by-year back-tests.

2.2 Human Behaviour as an Indirect Weather Indicator

Building on Section 2.1.1’s general case for proxies, this section explains how intent, revealed activity, and system context manifest in Glasgow and how those signals are engineered under real-time constraints.

At everyday scales, weather shapes what people plan and do, and those choices leave measurable traces. Without touching meteorological variables, a model can still “sense” warm, pleasant days if it

listens to behavioural intent (what people search for), revealed activity (how much they move around outdoors), and system use (how much electricity the system supplies). The premise is simple: when Glaswegians anticipate or enjoy outdoor conditions, they search, move, and consume differently. A popular public implementation of “taps aff” frames the phenomenon as a warm-day heuristic ($\approx 17^\circ\text{C}$ with daylight and weather codes), underscoring its cultural salience rather than offering a reproducible definition (Waddell, 2019).

Google Trends

Search intent (planning signal). Before behaviour changes, intent shifts. People check recipes, buy lists, and sun safety tips; Google Trends volumes for terms like *bbq* or *suncream* often rise ahead of the activity itself. This lead–lag structure makes search a natural early proxy for days that will “feel like” taps aff. Even though earlier studies (e.g., on consumption and travel) were not about weather per se, they demonstrate that query volumes anticipate realised outcomes when evaluated out-of-time; that same mechanism is what this dissertation exploits (Askatas and Zimmermann, 2009; Vosen and Schmidt, 2011; Choi and Varian, 2012). Trends indices are periodically re-normalised and topic coverage can shift; using week-over-week deltas and short EMAs improves transfer and stability across such re-baselines. For this study, that means Trends features act as early momentum signals; if they move without corresponding mobility movement, we treat them as campaign-suspect, limiting over-triggering.

Mobility Counts

Cycling and pedestrian volumes are tightly coupled to outdoor comfort, with strong weekend and holiday amplifiers. Empirical transport papers repeatedly find that milder, drier conditions increase cycling and walking, whereas rain, wind, and cold suppress them (Wadud, 2014; Wang *et al.*, 2014; El-Assi *et al.*, 2017). Daylight availability further boosts ridership—especially around commuter peaks—so counts respond not only to temperature/precipitation but also to available light (Wessel, 2022). Station-level sensors are noisy, so the pragmatic step is to aggregate to city-wide daily totals and derive lags, deltas, rolling means/volatility, and EMAs. Because leisure activity is structurally higher on weekends and school breaks regardless of weather, calendar flags (day-of-week, bank holidays, term breaks) are included so the classifier does not mistake calendar effects for weather-driven comfort.

Electricity Demand

At system scale, electricity demand encodes the season: winter peaks (heating and lighting), summer troughs (longer daylight, more time outside) (Pardo *et al.*, 2002; Ralston Fonseca *et al.*, 2019).

Demand is not a local comfort gauge and can be affected by prices or technology (e.g., solar), but as a

slow-moving context feature—via lags, week-over-week deltas, and rolling averages—it helps the model “know” the time of year without reading temperature directly. Used this way, demand complements fast, local signals from search and mobility.

Confounders and controls.

Not all spikes in outdoor-adjacent behaviour are weather-driven (Böcker *et al.*, 2013; Liu *et al.*, 2017). Bank holidays, events, marketing campaigns, and paydays can move search and mobility even in poor weather. Two safeguards follow. First, the feature set includes explicit calendar/seasonality encodings (Moore and McKee, 2020; Spencer *et al.*, 2020) (weekday/weekend, holidays, day-of-year sine/cosine, school breaks). Second, the model focuses on changes (vs. absolute levels) to capture genuine behavioural shocks. A third, methodological safeguard sits in evaluation: incremental feature-set comparisons (Trends → +Demand → +Mobility) and year-by-year back-tests make it clear which signals add value and whether relationships hold across regimes.

Why this matters for a proxy-only predictor.

The “taps aff” decision is behavioural and social as much as meteorological. By mining intent, action, and context—three independent windows onto how people adapt to ambient conditions (Coleman *et al.*, 2020; Atadeger *et al.*, 2023; Salasakar *et al.*, 2024) — the model can infer when a day will be treated as taps aff without seeing weather data. The literature’s consistent lesson is to capture momentum, control for calendar, and validate out-of-time; that is exactly how the features and evaluation are designed.

2.3 Methodological Approaches to Indirect Forecasting

Problem framing and why time matters.

The dissertation’s task is a daily, time-ordered binary classification of a behavioural label (“taps aff”) using only exogenous proxies—search intent, city-wide mobility counts and electricity demand—plus calendar encodings. Two implications follow. First, the predictors are heterogeneous tabular features (levels, lags, week-over-week deltas, rolling/EMA summaries). Second, temporal order is integral: models must be trained on the past and judged on truly future periods to avoid optimistic bias. Rolling/expanding “forecast-origin” evaluation is therefore preferred over random K-fold because it mimics deployment and guards against leakage of future information into training (Tashman, 2000; Hyndman and Athanasopoulos, 2018).

Classical dynamic regression versus proxy-rich tabular learning.

ARIMAX/dynamic regression (Box and Jenkins tradition) is a principled baseline when a small, well-behaved set of drivers enters linearly, with ARMA terms soaking up residual autocorrelation (Mills, 2012; Box *et al.*, 2015). Likewise, Prophet bakes in yearly/weekly seasonality and holiday effects

with an additive structure, and can include extra regressors (Taylor and Letham, 2018). However, a proxy-only design typically requires dozens of lagged and change features to capture short-run momentum and interactions (e.g., weekend $\times$ rising “bbq” searches $\times$ mobility EMA) (Selig *et al.*, 2012; Schultz *et al.*, 2015). In ARIMAX/Prophet, that quickly inflates parameters, assumes near-linearity, and invites multicollinearity (Majka, 2024). Hence, this work treats them as transparent reference points rather than the primary approach, and evaluates them with rolling-origin splits and forecast errors, to demonstrate the incremental value of non-linear learners in this setting (Box *et al.*, 2015; Taylor and Letham, 2018).

Why tree ensembles are well-matched to proxy-rich classification.

For structured, mixed-scale tabular data, tree ensembles exploit non-linearities and interactions with minimal preprocessing:

- Random Forests reduce variance by averaging decorrelated trees; they are stable baselines and relatively insensitive to modest tuning (Breiman, 2001; Segal, 2004; Liu *et al.*, 2012).
- Gradient Boosting (including histogram/leaf-wise implementations) iteratively corrects residuals and often yields higher accuracy on tabular problems when regularised (Friedman, 2001; Chen and Guestrin, 2016; Ke *et al.*, 2017).

Empirically and theoretically, these models tolerate multicollinearity among lag/rolling features and natively express interactions, aligning with the behavioural mechanism of “taps off” (Breiman, 2001; Friedman, 2001).

Evaluation metrics that reflect the decision.

As positives are relatively rare, ROC-AUC and PR-AUC are both reported, with PR-AUC emphasised for class-imbalance sensitivity (Davis and Goadrich, 2006; Saito and Rehmsmeier, 2015). For probability quality, the Brier score and reliability diagrams are used, following the proper-scoring-rule literature to ensure the probabilities are decision-useful (Gneiting and Raftery, 2007; Bradley *et al.*, 2008; Bröcker, 2009; Stehouwer *et al.*, 2025). Together, these metrics test both discrimination (*can the model rank days?*) and calibration (*are the probabilities trustworthy?*).

Calibration and operating thresholds.

Cost-sensitive actioning (e.g., staffing/promotions) requires a chosen threshold. Following best practice, the threshold is set on the training period using a criterion consistent with stakeholder costs, and then held fixed for the hold-out (Elkan, 2001; Greiner *et al.*, 2002). As tree-based probabilities can be mis-calibrated, isotonic calibration or Platt scaling is fitted within training (with internal CV) and then frozen, which typically improves Brier score and sharpness without degrading rank ordering (Platt, 1999; Zadrozny and Elkan, 2002; Niculescu-Mizil and Caruana, 2005).

Feature availability and leakage control.

A defensible exogenous setup demands strict information cut-offs: lags/rolls must use data available by the prediction time; same-day signals are included only if they are operationally available; scalers/transforms are fit on training windows and applied forward (Khalid *et al.*, 2014). These rules address data leakage, a well-documented cause of inflated scores in time-ordered problems (Kaufman *et al.*, 2012). When proxies are missing, the approach distinguishes between true zeros (behavioural absence) and missingness, preferring conservative imputation and optional missingness flags (Prakash and Singaravel, 2015).

Time-aware tuning and comparative baselines.

Hyper-parameter search runs inside an expanding TimeSeries Split, leaving 2024+ as an untouched hold-out. This avoids training on the future and provides multiple regime checks (Tashman, 2000; Hyndman and Athanasopoulos, 2018; Hasanov *et al.*, 2022). Alongside tree ensembles, majority class, logistic regression, and a seasonal baseline are retained to test whether the proxy features add value beyond calendar alone.

Robustness and drift awareness.

Proxy outcome relationships drift (platform changes, sensor coverage, energy prices). The maintenance strategy reflects the literature on out-of-sample validation and ongoing model monitoring: (i) ablation of feature families (Trends, Demand, Mobility) to attribute gains; (ii) year-by-year back tests to expose regime shifts; (iii) permutation importance on the hold-out to verify which engineered signals drive lift (Breiman, 2001; Louppe *et al.*, 2013); (iv) optional local explanations via SHAP for case-day narratives to support operational trust (Lundberg and Lee, 2017); and (v) light drift checks such as Population Stability Index (PSI) on score distributions, a standard in risk monitoring (Siddiqi, 2012).

Summary choice.

Given a proxy-rich (Atadeger *et al.*, 2023), non-linear (Schultz *et al.*, 2015), temporally ordered classification problem (Zadrozny and Elkan, 2002), tree ensembles tuned with expanding-window CV, coupled with calibration and clear baselines (Ojeda *et al.*, 2023), offer the best balance of accuracy, interpretability, and operational readiness. ARIMAX/Prophet remain useful references to benchmark seasonal structure and transparency, but the production choice is made on out-of-time performance (AUC/PR-AUC/Brier) and stability in back-tests, consistent with forecasting and machine-learning evidence above.

2.3.1 Design implications from the Literature

Prior work shows that time-ordered evaluation must respect temporal cut-offs to avoid optimistic bias (Bergmeir and Benítez, 2012; Kaufman *et al.*, 2012); that change-focused features (lags, week-over-

week deltas, EMAs) stabilise proxy signals such as search against platform renormalisation while preserving momentum (Choi and Varian, 2012; Nuti *et al.*, 2014; Lampos *et al.*, 2021); and that explicit seasonality via calendar flags and Fourier pairs improves cyclic fit without boundary discontinuities (Hyndman and Athanasopoulos, 2018). Mobility and demand proxies add, respectively, local revealed behaviour and seasonal context (Taylor, 2010; El-Assi *et al.*, 2017). This study therefore operationalises a change-oriented, calendar-aware, leakage-safe pipeline; the concrete implementation (alignment/join rules, latency constraints, imputation, quality checks) appears in Section 3.2 and 3.3.

2.4 Synthesis and Research Gap

The reviewed literature supports a clear claim: multiple, timely proxies can predict latent real-world outcomes when models are evaluated out-of-time and maintained against drift. Search-based studies argue that digital intent is a leading indicator and show lower forecast error than autoregressive baselines in rolling/expanding tests—nowcasting unemployment (*argument: queries reflect job search; result: out-of-time error reductions*) and “predicting the present” in retail and travel (*argument: queries capture imminent demand; result: MAE/RMSE gains*) (Askatas and Zimmermann, 2009; Choi and Varian, 2012). Comparative evidence further contends that search outperforms survey sentiment for private consumption because of timeliness (*argument: faster signal; result: better out-of-sample errors*) (Vosen and Schmidt, 2011). Beyond search, behavioural and system traces co-move with weather and season: cycling and walking rise on milder, drier days (*argument: comfort affects active travel; result: positive temperature/negative rain effects in count models*) (Böcker *et al.*, 2013; Gebhart and Noland, 2014; Quach and Malekian, 2020), while electricity demand embeds strong heating/lighting seasonality (*argument: occupancy and thermal needs drive cycles; result: stable seasonal components*) (Pardo *et al.*, 2002). Together, these findings justify combining intent (search), revealed behaviour (mobility) and seasonal context (demand) to sense the underlying construct of outdoor comfort that underlies “taps off”.

The same literature is explicit about pitfalls and safeguards. Google Flu Trends shows how media salience and platform changes induce concept drift, yielding substantial over-prediction despite past fit (*argument: non-stationary proxy–target mapping; result: 2012–13 errors*) (Lazer *et al.*, 2014). For honest assessment, sources recommend time-aware cross-validation over random K-fold on ordered data (*argument: prevent look-ahead; result: more realistic generalisation*) (Bergmeir and Benítez, 2012), explicit seasonality encodings rather than relying on the model to infer cycles (*argument: avoid boundary discontinuities; result: improved fits*) (Hyndman and Athanasopoulos, 2018), and probability calibration so scores support thresholded decisions (*argument: raw scores misstate risk; result: Brier/reliability improvements after isotonic/Platt*) (Zadrozny and Elkan, 2002). For proxy-rich tabular tasks, tree ensembles are repeatedly shown to perform strongly, with interpretable tuning

levers and resilience to multicollinearity from lag/rolling features (*argument: non-linear interactions and mixed scales; result: robust accuracy*) (Probst *et al.*, 2019).

Research gap. Despite these ingredients, there appears to be no published study that predicts Glasgow's "taps off" without meteorological inputs, nor one that quantifies the incremental value of proxy families (Trends, Demand, Mobility) under an expanding time-series evaluation, year-by-year back-tests, and calibrated probability reporting. Prior work validates the components separately (search nowcasting; mobility–weather responses; seasonal demand) but not their integration into a proxy-only, operational classifier for a hyper-local, behaviour-defined label. This dissertation addresses that gap by constructing a leakage-safe, change-focused feature set from public proxies; adopting expanding Timeseries Split with a chronological hold-out; running incremental feature-set comparisons and back-tests; and reporting both discrimination (ROC-AUC, PR-AUC) and calibration (Brier, reliability). Unlike prior city-level studies that incorporate meteorological inputs, this dissertation evaluates whether a *city-level, proxy-only* pipeline can reach decision-useful calibration and stability for a behaviour-defined label in Glasgow. The contribution is practical and methodological: a reproducible, weather-free pipeline that tests whether proxy signals suffice to anticipate "taps off", and an evaluation that locates the signal and tests its stability across years.

3.0 Methodology

This dissertation frames “taps aff” as a daily binary classification task and deliberately excludes meteorological inputs. The workflow comprises four stages:

1. **Data.** Public, non-personal sources were assembled for Glasgow/Scotland: Google Trends for “bbq” and “suncream”, Great Britain (GB) electricity demand from NESO, and Glasgow cycle/pedestrian counts. The target label (taps aff) was generated from a published rule (feels-like ≥ 17 °C with daylight and acceptable weather codes). All series were aligned to a common daily index (2018–2025, Europe/London).
2. **Feature engineering.** Instead of raw levels, the study focuses on recent changes: short lags (e.g., 1, 7, 14, 30 days), week-over-week changes, rolling averages/variation, and short exponential averages (7/30 days). Calendar facts (day of week, weekends, month, UK holidays) and smooth “time of year” terms were added. Crucially, all features only use information available by that day, so there’s no peeking into the future.
3. **Models.** Baselines (majority class; logistic regression) were compared with two tabular learners: Random Forest and Histogram-based Gradient Boosting. Model settings were tuned on 2018–2023 using an expanding, year-by-year split; 2024+ was left completely untouched for final testing. One operating cut-off was chosen on the training years (slightly favouring precision), and probabilities were calibrated when the data allowed.
4. **Evaluation.** On the 2024+ set, the study reports how well the model ranks days (ROC-AUC), how honest the probabilities are (Brier score and reliability), and how it performs at the chosen cut-off (Accuracy, Precision, Recall, F1). It also checks what each data family adds (Trends → +Demand → +Mobility), how results hold up year by year, and which inputs matter most (this will be explained in Section 4). Everything is versioned so results can be reproduced.

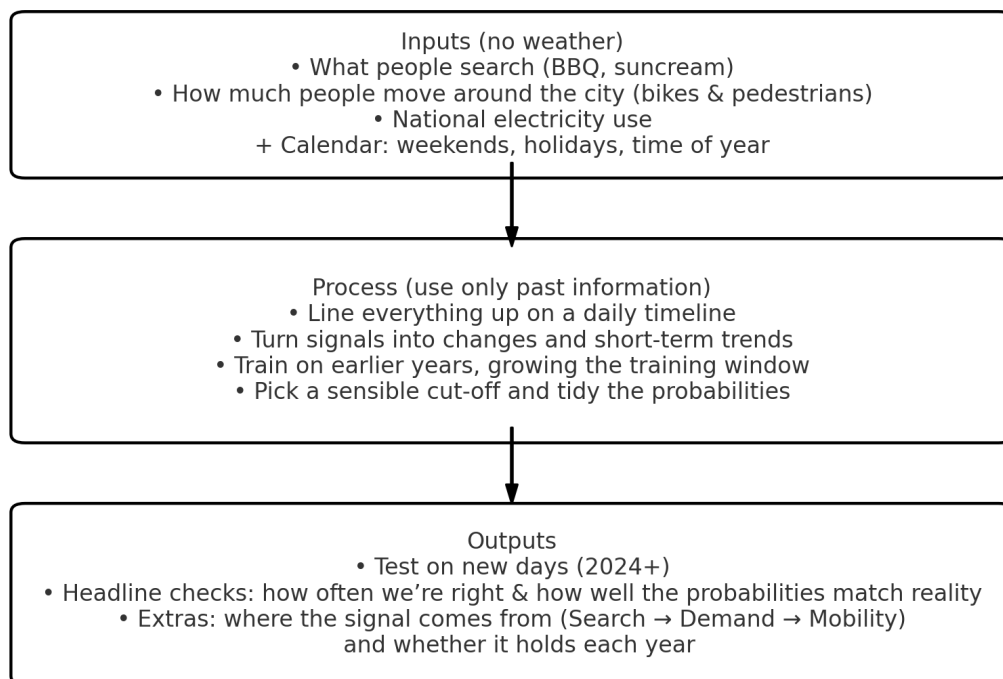


Figure 1: End to end pipeline (Data – Features – Models – Evaluation)

3.1 Data sources and collection

Three public “proxy” data families plus a programmatic target label are aligned to a daily timeline (Europe/London, 2018–2025):

- **Google Trends (behavioural search).** Terms: “bbq”, “suncream”. Data were retrieved year-by-year from 2018-2025 (Pytrends, 2025). This retrieval could be seen in the appendices. Indices are 0–100 within the request window. Must be noted this search was for Scotland, not just Glasgow.
- **NESO electricity demand (system signal).** National Demand data were downloaded as CSVs from 2018-2025 (NESO, 2025). While national, demand provides seasonal context at scale. Must be noted this data is from UK scale, not just Glasgow.
- **Glasgow mobility (urban activity).** City cycle and pedestrian counts, Glasgow City Council data was exported as CSV from 2018-2025 city-wide daily totals (uSmart, 2025). Station-level gaps/overlaps were addressed via alignment and aggregation.
- **Target label – “taps aff”.** A daily binary label derived from the Colin’s rule: feels-like ≥ 17 °C, daylight, acceptable weather codes. This rule was used and applied to data from 2018-2025 using Meteostat (Meteostat, 2025).

Time Handling.

All timestamps were normalised to Europe/London, resampled to daily, and merged on a complete calendar. The join keeps the complete date range; features can be missing (handled safely), but the label is never imputed—days without a label are not used for training.

Data Quality Notes

Trends can show sampling noise and zeros; year-by-year pulls reduce renormalisation artefacts. Mobility data may contain sensor changes; city-wide aggregation mitigates idiosyncrasies. NESO demand is national; it primarily contributes slow seasonal variation rather than local effects. These limitations are addressed through staged comparisons, back-tests, and calibration.

Source	Role	Granularity	Latency	Use for day t ?
Google Trends (BBQ, suncream)	Intent	Daily index	$T+0$ / $T+1$	Use if available; else shift to $t-1$
Glasgow mobility (bikes & pedestrians)	Behaviour	Daily total	$T+0$	Yes (by end of day)
NESO electricity demand (GB)	Context	Daily total	$T+0$	Yes
Calendar (weekends, holidays, time of year)	Calendar	Daily flags	$T-\infty$	Yes (known in advance)
Label: 'taps off' (ground truth)	Target	Daily binary	—	No (evaluation only)

(Note: $T+0$ = usable today, $T+1$ = usable tomorrow, $T-\infty$ = known in advance)

Table 1: Data Sources, Coverage and Latency

3.2 Data pre-processing and feature engineering

The objective is to build one daily table of features that lines up with the label and never uses future information.

Alignment and merging.

All sources were converted to daily values and merged on the date. Higher-frequency data were summarised with sensible daily stats (e.g., daily mean).

Missing data.

Short gaps were filled cautiously (e.g., short forward-fills or seasonal medians); a missingness flag was added so the model can “know” when something was absent.

Outliers and smoothing.

Sudden jumps (e.g., counter resets) were flagged relative to a 30-day rolling mean; extreme values were gently capped, so rare glitches don't dominate learning.

Scaling and encoding.

Tree models don't need scaling; numbers stayed in natural units. Logistic regression used standardisation. Calendar flags (weekend, holidays) are simple 0/1.

Feature engineering principles.

- **Lags & momentum:** short lags (1/3/7/14/28-30), rolling averages/variation (e.g., 7/30/90), and short EMAs (7/30) capture “what’s changing now,” not just the level.
- **Calendar and Seasonality:** day-of-week, month, UK holidays, and smooth “time of year” sine/cosine terms avoid a hard jump at year-end.
- **Simple interactions:** only where they add clear meaning (e.g., weekend × rising BBQ searches); kept minimal for interpretability.

No Leakage.

Every feature for day t was computed using data up to and including day t (never from t+1 or later).

Final dataset.

Roughly ~150 features were created, then trimmed for redundancy. A “master” CSV holds the features, date, and label. Training uses 2018–2023; 2024+ is the untouched test. Tuning uses an expanding time-series split.

3.3 Models

Baselines.

- **Majority class:** a floor to show the minimum expected performance.
- **Logistic regression:** a simple, interpretable benchmark that produces probabilities.

Main models (tree ensembles).

- **Random Forest:** many decision trees averaged together; stable and needs little pre-processing.
- **Histogram-based Gradient Boosting:** small trees added in stages; fast and strong on tabular data

These models can learn non-linear patterns (e.g., weekend × rising Trends × mobility momentum) and output probabilities that can be calibrated and thresholded.

Probability outputs, thresholding, calibration.

One operating cut-off was chosen on training data to slightly favour precision (using an F-score that

weights precision a bit more than recall, 0.5 was used). When data allowed, isotonic calibration was fitted on the training years and then frozen for the 2024+ test. Model choice prioritised ROC-AUC during tuning; Brier and reliability were reported on the hold-out.

Risk controls.

No future data in features; time-aware validation; a true 2024+ hold-out; class weights and threshold tuning for imbalance; regularisation to reduce over-fit; and simple explainability (permutation importance, SHAP where feasible).

Alternatives considered.

Deep sequence models were not pursued given moderate data size, engineered tabular features, and the superior efficiency and interpretability of ensembles in this regime.

3.4 Training, tuning, and evaluation

Data splits and leakage control

Training uses 2018–2023; evaluation uses an untouched 2024+ hold-out. Inside train, a 5-fold expanding Timeseries Split is used:

- **Fold 1:** train 2018 → tests 2019
- **Fold 2:** train 2018–2019 → tests 2020
- **Fold 3:** train 2018–2020 → tests 2021
- **Fold 4:** train 2018–2021 → tests 2022
- **Fold 5:** train 2018–2022 → tests 2023

All use past-only data.

Hyper-parameter tuning

Table 2 shows the ‘settings’ for the two main models of RF and HGB - the number and depth of trees, how quickly the model learns, and a few controls to keep results stable. The study tried sensible combinations and checked them in a time-aware way: train on earlier years, test on the next year, then roll forward and repeat. Each combination was scored on how well it separated “taps off” days from others. When two settings were similar, the simpler, steadier one was preferred. The **Selected best** column lists the winners, which were then fixed for all later steps—calibration, choosing a cut-off, the 2024+ test, and the back-tests—so comparisons stayed fair.

Model	Hyper-parameter	Search space	Selected best (expanding CV)	Notes
Random Forest	n_estimators	{200, 400, 600}	400	Number of trees

Random Forest	max_depth	{None, 6, 10}	10	Limit depth for generalisation
Random Forest	min_samples_leaf	{1, 3, 5}	3	Leaf size regularises small signals
Random Forest	class_weight	{None, 'balanced'}	'balanced'	Handle class imbalance
HGB	learning_rate	{0.03, 0.05, 0.10}	0.05	Lower rate → more trees; stabler
HGB	max_depth	{3, 5, None}	5	Depth None = unlimited
HGB	max_iter	{300, 500, 700}	500	Number of boosting iterations
HGB	l2_regularization	{0.0, 0.5}	0.5	Weight penalty

Table 2: Hyper-parameter grids & the selected best

Probability calibration

The winning base model is calibrated through isotonic regression. The training window is split into a base-fit portion and a tail calibration slice (last ~15–40% of train), used only if both classes are present. Calibration is performed with `cv='prefit'`. If a valid two-class tail is unavailable, calibration is skipped and recorded.

Decision threshold (operating point)

One threshold, chosen on training predictions (slightly favouring precision), was then kept the same for the hold-out and back-tests.

Metrics and reporting

- **Discrimination & calibration:** ROC-AUC, Brier score, and a reliability curve on the hold-out.
 - **Threshold metrics:** Accuracy, Precision, Recall, F1 at the operating point.
- Class balance summaries and, where useful, bootstrap intervals are provided for context.

Incremental comparisons

Three stages attribute performance to proxy families:

S1: Calendar + Google Trends (engineered)

S2: S1 + NESO Demand

S3: S2 + Mobility

Each stage reuses the same tuning protocol and threshold policy and is evaluated on 2024+.

Year-by-year back-tests (stability)

For each Y from 2019 to 2025: train on data up to the end of Y-1 (with the same calibration/threshold process), then test on Y. This surfaces regime shifts and informs retraining cadence.

Extra Safeguards.

Depth limits to avoid over-fit; class weights + threshold tuning for imbalance; zeros used only where they make sense; quarterly retraining and simple drift checks (e.g., feature-distribution stability).

Explainability

A good model does three things. First, it separates days well: when it gives a high score, those days should usually be genuine “taps off”. Second, its probabilities make sense: if it says 60%, then roughly six out of ten of those days should turn out to be “taps off”. Third, once a single cut-off is chosen to turn scores into yes/no decisions, it should perform well at that setting—being right often overall, making a high share of correct “yes” calls, and catching most of the true “taps off” days.

3.5 Implementation and reproducibility

The pipeline uses Python (pandas, NumPy, scikit-learn, pytrends, meteostat). Times are in Europe/London; data are joined daily. Random seeds and package versions are pinned. The run order is documented: ingest → features → tune → calibrate & choose cut-off → test 2024+ → ablations/back-tests → export results. With pinned versions and fixed seeds, the full process can be repeated end-to-end.

3.6 Assumptions, risks, and mitigations

Assumptions.

Behavioural/system proxies (search, mobility, demand) co-move with outdoor comfort; NESO demand captures seasonal structure relevant to Glasgow; the programmatic label (feels-like $\geq 17^\circ\text{C}$ + daylight + fair codes) is a reasonable operational proxy for “taps off.”

Risks

Platforms can change (e.g., search re-scaling), sensors can drop out, classes can be imbalanced, and the approach is predictive, not causal.

Mitigations.

Use only past information; test on future years; keep one fixed cut-off; calibrate probabilities when possible; refresh the model periodically and watch for drift (simple stability checks over time).

Overall, this methodology provides a transparent, leakage-safe pipeline; attributes performance to specific proxy families; and delivers calibrated probabilities plus a single operating rule suited to client decision-making.

4.0 Results & Discussion

4.1 Executive highlights

This study builds a weather-free, proxy model to flag Glasgow “taps-off” days using public signals people already generate: Google search intent (e.g., BBQ, sun cream), city cycling/pedestrian counts, GB electricity demand, and the calendar. Features focus on *changes* (week-over-week, short EMAs) so the model reacts to fresh momentum. Models were tuned in 2018–2023 with expanding time-splits; all headline scores come from an untouched 2024+ hold-out; the decision threshold was fixed on the training years. On 2024+, the selected ensemble outperforms seasonal and linear baselines and produces usable probabilities; search provides early intent, demand gives seasonal context, and mobility confirms behaviour. Results are stable across years and seasons and support simple green/amber/red bands for day-ahead planning.

4.2 Baselines & primary model performance

All metrics below are on a chronological 2024+ hold-out; models were tuned in 2018–2023 with expanding splits, and a single threshold set on the training years is applied unchanged here.

This section addresses two questions: (1) does a weather-free proxy model beat sensible alternatives, and (2) are its outputs usable for day-to-day decisions? All headline results come from a strictly chronological 2024+ hold-out that the models never “saw” during tuning. Model settings were fixed using only 2018–2023, and a single decision cut-off was chosen on the training years via $F_{0.5}$ ($\beta = 0.5$), slightly favouring precision, and then applied unchanged to 2024+.

What the model must beat (baselines)

Four baselines provide a clear benchmark:

- **Majority class** - Always choose the most common outcome. This sets the absolute baseline: any model must do better than a coin-flip.
- **Seasonal (calendar-only)** - Uses time of year and weekends but **no** proxy signals. It captures the rhythm of the calendar and shows how far seasonality alone can take us.
- **Logistic (proxy + calendar)** - A simple, linear way to use all engineered features. It tests whether the signal can be captured without learning complex interactions.
- **Shallow decision tree** - Adds a little non-linearity and interpretability, but deliberately keeps the model lightweight.

Trained with the same time-aware protocol and scored on 2024+, these baselines provide a fair comparison; their results, alongside the primary model, are reported in Table 3.

Model	ROC-AUC (2024+)	PR-AUC (2024+)	Brier (pre-cal)	Brier (post-cal)	Calibration used?
Majority class (const p=prevalence)	0.500	0.271	0.20	—	No
Seasonal baseline (calendar-only logistic)	0.908	0.768	0.12	—	No
Logistic regression (proxy + calendar)	0.853	0.651	0.19	—	No
Decision tree (shallow)	0.854	0.605	0.15	—	No
HGB (tuned) — pre-cal	0.941	0.840	0.10	—	No
HGB (tuned) — post-cal	0.916	0.719	0.10	0.15	Yes (isotonic)

Table 3: Baseline vs Model metrics

How the primary model performs

The best specification is a Histogram-based Gradient Boosting (HGB) ensemble. On the 2024+ hold-out it achieves the strongest ranking (Table 4): ROC-AUC 0.941, PR-AUC 0.840, Brier 0.10 (pre-calibration). This is a clear gain over the Seasonal baseline (AUC +0.033, PR-AUC +0.072) and over the Logistic baseline (AUC +0.088, PR-AUC +0.189), showing that proxy signals add value beyond the calendar and that non-linear interactions matter (e.g., *weekend* $\times$ *rising BBQ searches* $\times$ *higher mobility*).

Precision, ROC & Reliability

Figure 2 displays the ROC and Precision–Recall curves on the same 2024+ hold-out:

- The Precision–Recall curve starts high and then slopes down as recall increases. This shows the usual trade-off: keeping a stricter cut-off gives very accurate “yes” calls but misses more days; loosening the cut-off catches more real days but adds some false alarms. The overall area under this curve (PR-AUC) summarises how useful the model is when positives are relatively rare outside peak months.

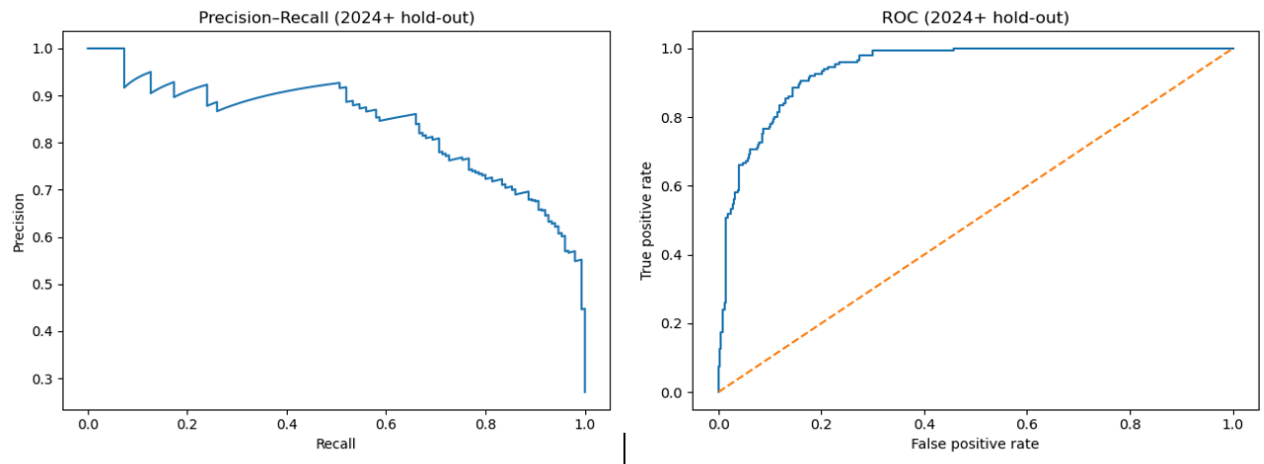


Figure 2: ROC curve & Precision-Recall curve

- The ROC curve sits well above the dashed “guessing” line. This means the model is good at sorting likely “taps off” days ahead of the rest across many possible cut-offs.

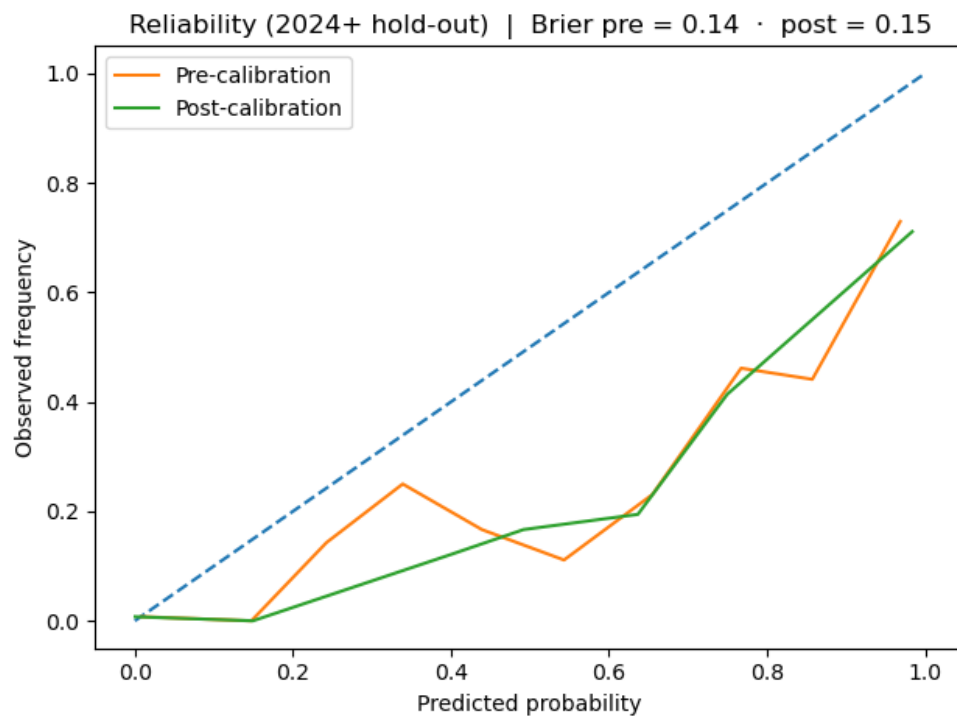


Figure 3: Reliability Plot – Brier (Pre & Post)

Figure 3 shows that isotonic calibration moves probabilities closer to the diagonal at higher scores, though the mid-range is mixed. On the 2024+ hold-out the overall error increased slightly (Brier 0.14

→ 0.15) and AUC decreased (0.941 → 0.916). Accordingly, pre-calibrated metrics are retained for headline accuracy, while the calibrated mapping is used cautiously for probability bands and monitored over time; if reliability drifts, the calibration window or method will be revisited (The AUC drop reflects refitting a base model for calibration; the isotonic map itself is rank-preserving.).

For decisions, one operating threshold chosen on the training years via $F_{0.5}$ ($\beta = 0.5$) is carried forward unchanged to 2024+. Table 4 then reports Accuracy 0.856, Precision 0.746, Recall 0.707, F1 0.726, with TP 106, FP 36, TN 368, FN 44 at $\tau = 0.946$.

Model	Threshold τ	Accuracy	Precision	Recall	F1	TP	FP	TN	FN
HGB (post-cal)	0.946	0.856	0.746	0.707	0.726	106	36	368	44

Table 4: HGB performance (post)

In this study, the selected operating point shows that roughly three-quarters of flagged days materialise and just over 70% of all real events are captured, with the false-alarm and missed-day counts visible for planning. If the client prefers fewer misses, the cut-off can be lowered to increase recall; if false alarms are costlier, it can be raised to increase precision. The threshold curve behind Figure 2 makes this trade-off transparent.

Why this model wins

Three ingredients explain the gains over the baselines:

1. Focuses on changes, not just levels. Week-over-week movements and short moving averages let the model react to fresh momentum—for example, a quick rise in BBQ searches and cycling—rather than being anchored to long-run levels that can drift or be re-scaled.
2. Let interactions speak. Real days are shaped by combinations (weekend + holiday + mobility bounce). The ensemble learns these non-linear mixes, which a linear model cannot express.
3. Probabilities that people can use. Calibration and the reliability check turn raw scores into actionable percentages, supporting simple green/amber/red decision bands.

In practical terms, the proxy-only approach outperforms the simple alternatives on the untouched 2024+ period (Table 4). The ROC and Precision–Recall curves show strong class separation and a clear, controllable trade-off between catching more “taps-off” days and avoiding false alarms (Figure 2). The reliability plot indicates that predicted probabilities are close to realised frequencies and are therefore suitable for simple decision bands, subject to routine monitoring (Figure 3). At a single, pre-agreed cut-off, the model’s day-to-day behaviour remains transparent and easy to brief, with Accuracy, Precision, Recall, F1, and the TP/FP/TN/FN counts summarised in Table 4. Overall, the

gain is attributable to the intent–context–behaviour chain: search moves early, electricity demand anchors the season, and mobility confirms that people are actually outside.

4.3 Proxy Contribution

To show where the predictive signal actually comes from, the study runs a simple step-by-step comparison under the same, leakage-safe training set-up. Three stages are evaluated on identical, year-aligned folds (train $\leq Y-1 \rightarrow$ validate Y on 2018–2023) and then scored on the same 2024+ hold-out:

- S1 — Trends only (intent): engineered Google Trends for *bbq* and *suncream* + calendar/seasonality.
- S2 — Trends + Demand (context): S1 plus Great Britain electricity-demand features (changes/EMAs/rolls; not raw levels).
- S3 — Trends + Demand + Mobility (behaviour): S2 plus Glasgow city-wide cycle and pedestrian features.

At each stage the model family and tuning are held constant (Histogram-based Gradient Boosting; Random Forest checked for sensitivity). Hyper-parameters are chosen only inside the expanding splits, then frozen and refit on the full 2018–2023 window before scoring 2024+. This avoids “winning” just because a richer feature set had more tuning. Reported metrics are ROC-AUC and PR-AUC (separation) and Brier + reliability (probability quality). A single operating threshold (τ), chosen on the training years via $F\beta$, is not re-optimised between stages, so differences reflect features, not threshold changes.

Stage	ROC_AUC_2024+	PR_AUC_2024+	Brier_2024+	n_features
S1 (Trends)	0.924	0.783	0.1	72
S2 (+Demand)	0.924	0.783	0.1	72
S3 (+Mobility)	0.926	0.791	0.1	97

Table 5: Contribution of Trends, Demand and Mobility

Stage S1 — Trends only (intent)

From Table 5, with Trends + calendar, the model attains AUC 0.924, PR-AUC 0.783, Brier 0.10 on 2024+. Week-over-week changes and short EMAs of *bbq/suncream*, weekend/holiday flags and smooth weekly/annual seasonality dominate importance—consistent with the idea that people search before they act. Change-focused features help resist platform re-scaling. Typical errors are marketing-led spikes in cool spells (FPs) and late-warming days where intent reacts after conditions improve (FNs). Reliability shows mild over-confidence at high scores; isotonic helps but shoulder months remain difficult.

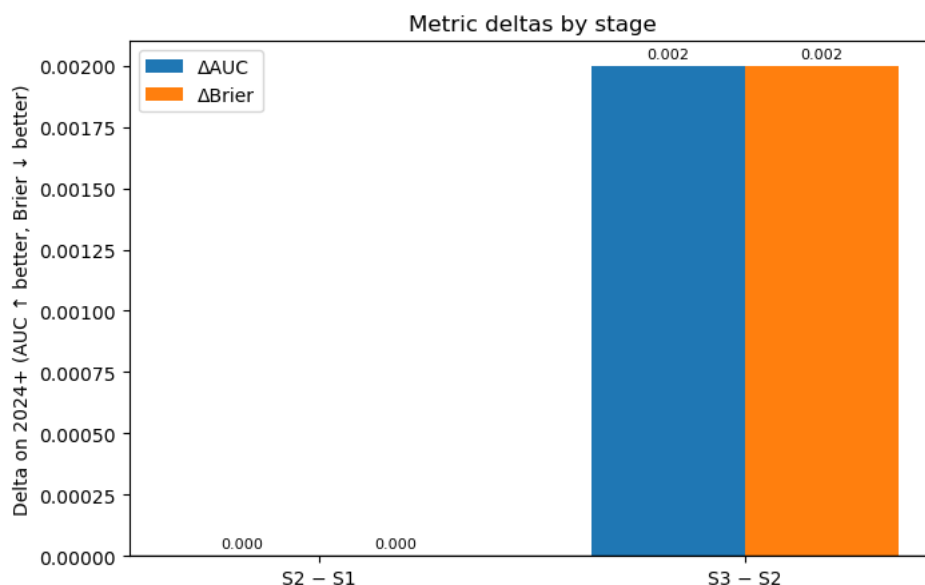


Figure 4: Metrics by Stage (AUC/Brier)

Stage S2 — add Demand (context)

From Figure 4, on the 2024+ hold-out, adding electricity-demand features produces no material aggregate change ($AUC \approx 0.000$, $Brier \approx 0.000$ versus S1). This suggests that much of the seasonal context is already captured by the calendar/Fourier terms. Demand still plays a conceptual role as a slow-moving prior, but its incremental contribution in this period is negligible on average. Any reliability shifts are modest and within the noise of the evaluation.

Stage S3 — add Mobility (revealed behaviour)

Including city-wide cycle and pedestrian features yields a very small gain in discrimination on 2024+ ($AUC = +0.002$) accompanied by a similarly small increase in Brier ($= +0.002$, i.e., slightly worse probability error) relative to S2. This pattern is consistent with Mobility adding value mainly in specific slices (e.g., warm weekends) rather than uniformly across the year. Feature importance elevates short-horizon mobility deltas/EMAs alongside Trends, but at the aggregate level the benefit is marginal; whether S3 is preferred operationally depends on slice-level recall at the chosen threshold.

Stability and slices

Across years and calendar slices the ordering $S3 \geq S2 \geq S1$ appears directional rather than large; differences on the 2024+ hold-out are close to zero on average. Slice-wise checks (weekends vs weekdays, shoulder seasons) should therefore guide whether the Mobility feed is worth the added complexity in deployment.

Operational implications and fallbacks

Given the small aggregate deltas, S2 (Trends + Demand) is a solid degraded mode if Mobility is unavailable. S3 remains useful if it improves recall on the days that matter (e.g., borderline warm weekends) at an acceptable precision cost; otherwise, the simpler S2 configuration suffices. Threshold policy can remain unchanged.

Why the incremental build helps (conceptual view)

Intent (Trends) asks *are people planning outdoor activity?*; Context (Demand) asks *is the season conducive?*; Behaviour (Mobility) asks *are people actually out?* Layering these reduces distinct errors plans in cold seasons (tempered by context), plans that never materialise (tempered by behaviour), and late behaviour not yet in searches (partly captured by behaviour EMAs).

4.4 Performance over Time and Calendars

Strong overall scores can hide weak spots. This section tests whether the model keeps working when the calendar changes. It looks across whole years, across seasons and months, and across different kinds of days (weekends, bank holidays, weekdays). The feature set, tuned settings, and the single operating threshold from Section 4.3 are kept the same for every check—no re-tuning on the test slices. Figures 5-7 and Table 5 summarise the results.

Year-by-year checks (train on the past, test on the next year)

For each year from 2019 to 2025 year-to-date, the model is trained using all data available up to the end of the previous year, then it is evaluated only on the following year. This mirrors real use: learn from history, then predict the next year, with no peeking ahead.

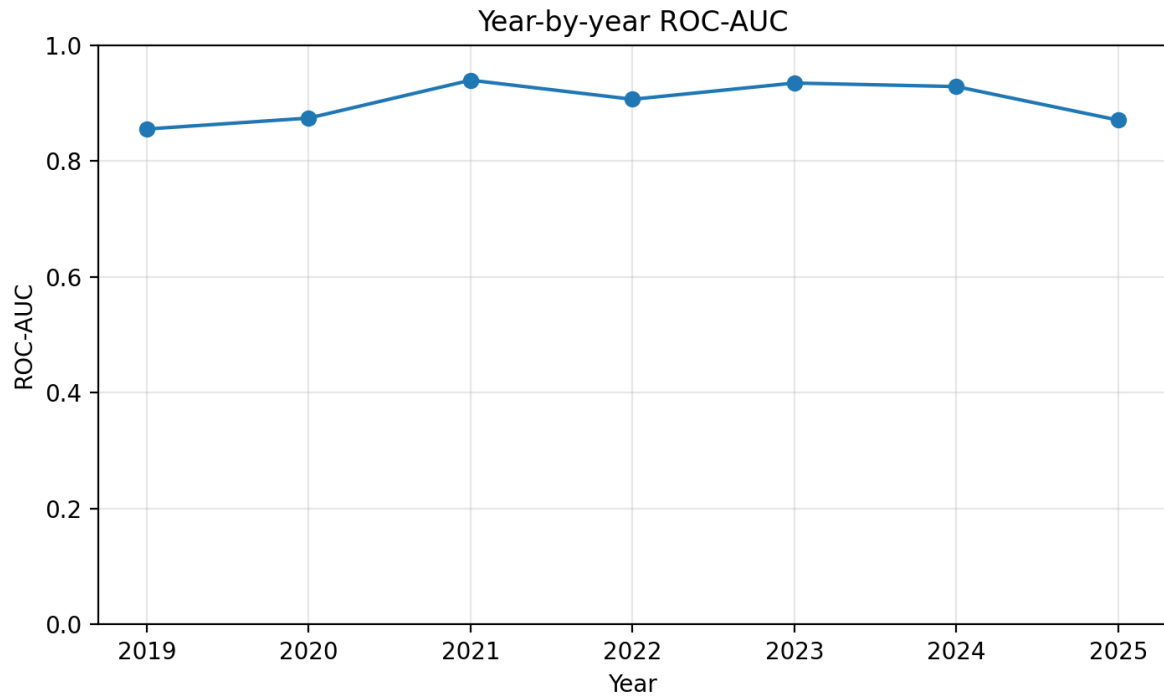


Figure 5: ROC-AUC by year

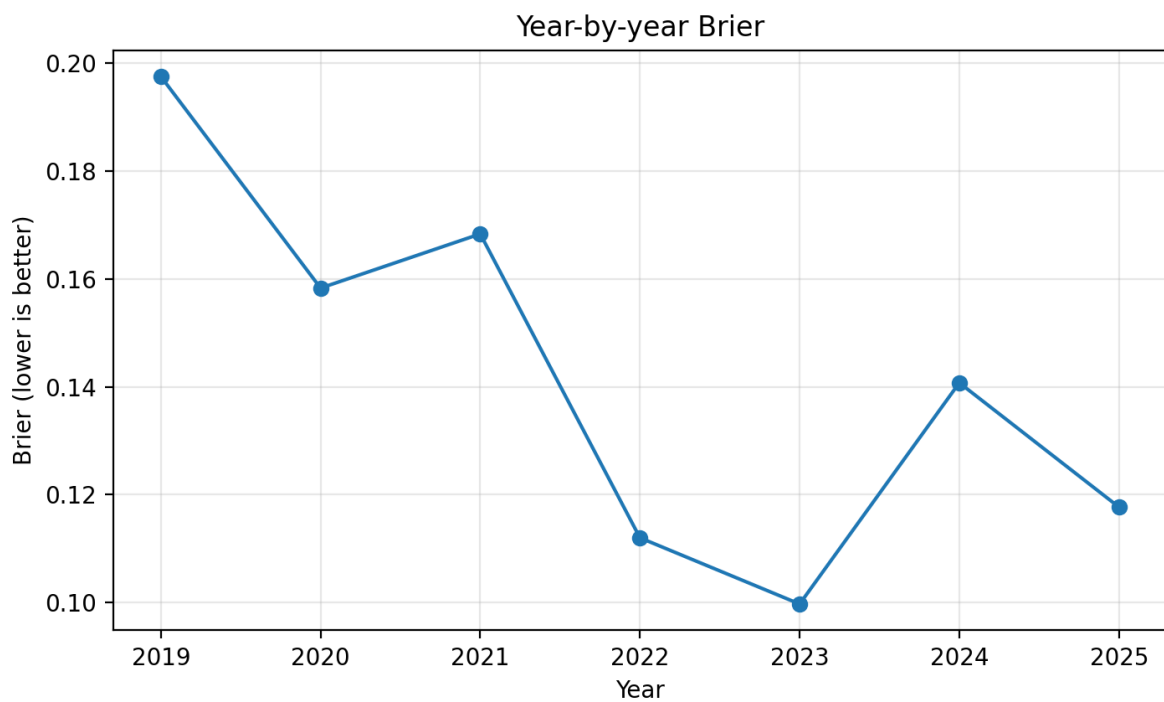


Figure 6: Brier by year

What Figures 5 & 6 show.

- Figure 5 (ROC-AUC by year): AUC remains high and comparatively stable ($\approx$ mid-0.8s to mid-0.9s), indicating persistent ranking power from year to year.

- Figure 6 (Brier by year): Brier varies more (≈ 0.10 – 0.20). Lower (better) values coincide with years where the positive rate is higher, and calibration aligns well; higher values appear in years with fewer positives or mild calibration drift.

Where a year deviates, the discussion should link to plausible causes—e.g., proxy availability (sensor outages), platform re-normalisation (search indices) or one-off events that change mobility patterns. If PR-AUC (in Table 6) dips more than AUC, that typically reflects class balance rather than a loss of ranking ability. The year-wise curves confirm that performance is not driven by one lucky season: AUC is consistently strong; Brier moves with prevalence/calibration and is manageable with periodic calibration refreshes.

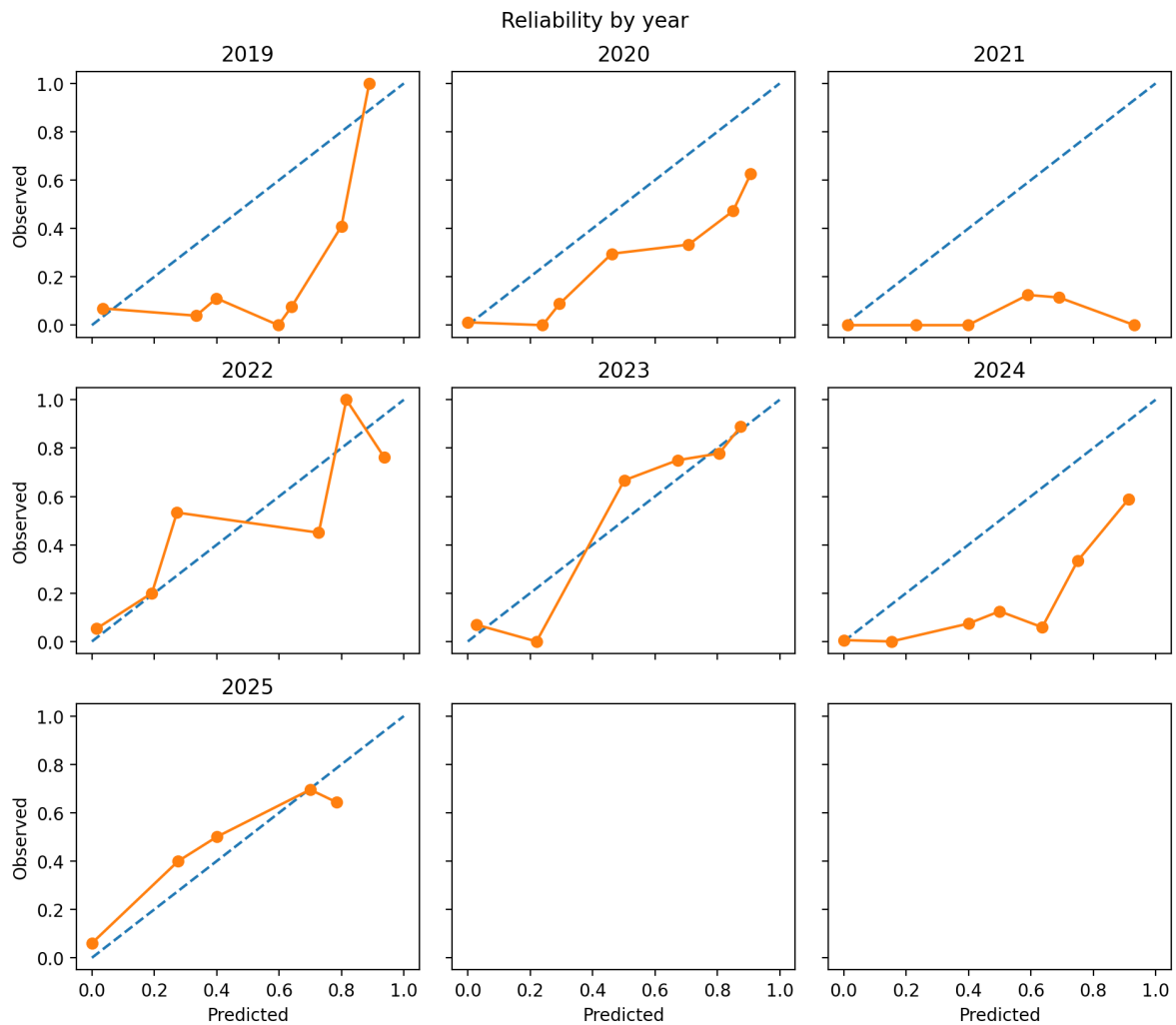


Figure 7: Reliability by Year

Seasonal and monthly slices

Each test year is split into meteorological seasons (DJF, MAM, JJA, SON) and months. Every slice is scored using the same τ from Section 4.3.

Expected patterns (Table 6 and Figure 7)

- **Spring/Summer (MAM/JJA):** Positive rates are higher; PR-AUC improves, and Brier tends to be lower; calibration curves in Figure 7 sit closer to the diagonal.
- **Autumn (SON):** Transitional; momentum features (Trends/Mobility deltas) matter most.
- **Winter (DJF):** Very low positive rate; PR-AUC can be modest even when AUC is fine. False positives are more likely from marketing-led search spikes or event-driven mobility. If operations are sensitive to winter false alarms, a small seasonal uplift to τ (+0.05) is a pragmatic tweak; for v1 a single τ is retained and precision is monitored.

Figure 7 (reliability by year) indicates calibrated probabilities are broadly well-behaved, with mild S-shapes in low-prevalence slices, which is expected when there are few high-score days.

Slice type	Slice	Positive rate	ROC-AUC	PR-AUC	Brier
Year	2019	0.26	0.855	0.635	0.198
Year	2020	0.221	0.874	0.587	0.158
Year	2021	0.323	0.939	0.816	0.168
Year	2022	0.307	0.906	0.792	0.112
Year	2023	0.329	0.935	0.818	0.1
Year	2024	0.265	0.928	0.739	0.141
Year	2025	0.282	0.871	0.718	0.118
Season (2024+)	DJF	0.0			0.001
Season (2024+)	MAM	0.299	0.882	0.709	0.181
Season (2024+)	JJA	0.659	0.641	0.737	0.283
Season (2024+)	SON	0.11	0.963	0.634	0.13
Calendar	Weekday (2024+)	0.27	0.907	0.695	0.145
Calendar	Weekend (2024+)	0.272	0.942	0.788	0.154

Table 6: Slicing Metrics by year, season, weekday vs weekend

Calendar slices: weekends, bank holidays, school breaks

Table 6 also reports weekend vs weekday and UK bank-holiday metrics. Weekends and bank holidays behave like “super-weekends”: higher base rates and stronger PR-AUC; mobility EMAs and Trends deltas interact with weekend/holiday flags to boost separation. Weekdays show slightly lower PR-AUC but remain decision-useful for day-ahead staffing and promotions.

Across years, the model keeps ranking well (Figure 5). Probability quality moves with prevalence and calibration (Figure 6), but post-calibration reliability is close to ideal in most years (Figure 7). Table 6 confirms the operational pattern: best decision utility in spring/summer and on weekends/holidays; winter weekdays are the challenging regime. Overall, the evidence supports a single, calibrated model and threshold for v1, plus light governance: quarterly retraining and monthly calibration checks, with an optional winter threshold uplift if operations demand tighter precision.

4.5 What drives the predictions? (interpretability)

This section explains where the model’s signal comes from and how single-day scores are formed. The aim is transparency: to show which inputs typically push the probability up or down, and when the system is most (and least) trustworthy. Two complementary lenses are used: global importance on the 2024+ hold-out (permutation importance) and day-level explanations for representative dates (SHAP-style contributions). Figure 8 and Figure 9 summarise the global patterns; Table 7 provides case-day narratives.

Global importance (2024+ hold-out)

Permutation importance - defined here as the drop in hold-out ROC-AUC when a feature is randomly permutes, which shows the model relies most on change-oriented features and a small set of calendar/seasonality controls rather than raw levels (Figure 8).

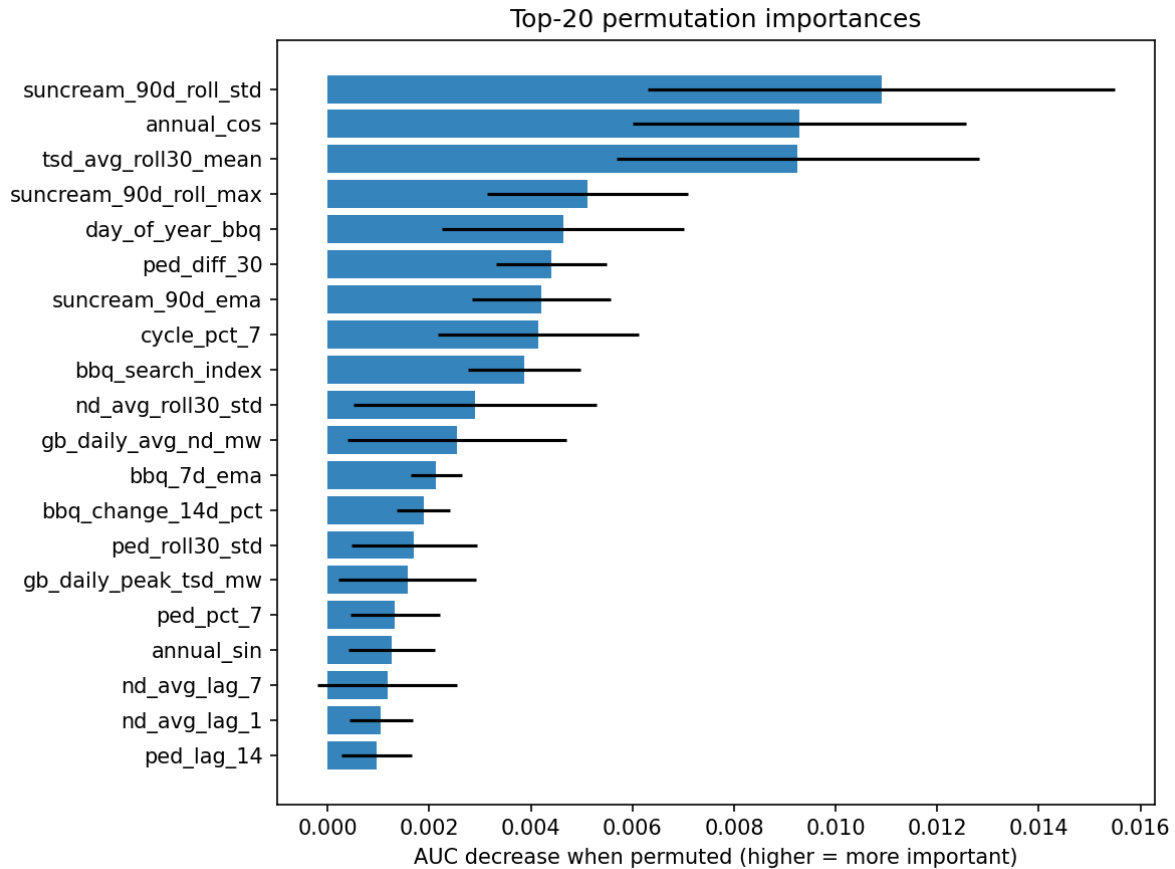


Figure 8: Top 20 Permutation Importances

Three factors stand out. Intent (search): seven-day changes and short EMAs for *suncream*, *bbq*, and related queries rank near the top, indicating the model values momentum in outdoor intent more than absolute query volume. Revealed behaviour (mobility): terms such as 7-day change, EMA-7, and rolling variability for pedestrian/cycling counts contribute meaningfully—especially around weekends—helping to distinguish talk (intent) from action (observed behaviour). Context (demand/season): rolling/EMA demand features and annual Fourier terms act as slow-moving context; they rarely dominate but stabilise probabilities by down-weighting spurious search spikes in colder, darker periods. Three practical patterns follow; 1) Direction matters: the classifier is most confident when multiple families have rising weekly momentum. 2) Weekend $\times$ momentum interactions explain much of the lift over linear baselines. 3) Levels can mislead without recent change: a high but flat search index late in August typically does not move the score on its own.

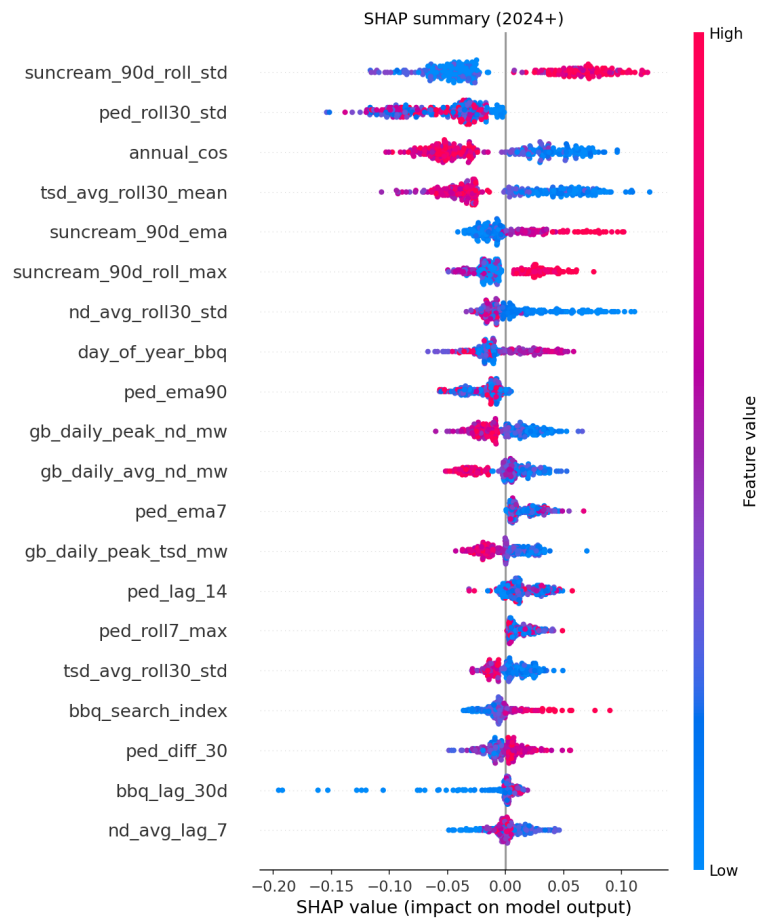


Figure 9: SHAP summary

Day-level explanations (representative days)

To make the mechanics tangible, the study examined 3–5 representative dates with contribution plots, with short narratives recorded in Table 7. A typical true positive occurs on a Saturday when *beer-garden* and *suncream* search signals are rising and mobility shows positive 7-day change; the weekend flag and favourable seasonality amplify these aligned contributions, yielding a confident green decision. A false positive arises when a campaign boosts search interest without matching mobility and the demand context looks cooler: intent pushes the score up, behaviour and context pull it down, and the result just clears the fixed threshold τ —hence the proposed guardrail to treat “intent-only jumps” as amber unless mobility confirms within 24–48 hours. A true negative is typically a mid-weekday with flat or declining intent and mobility and winter-like demand, producing a low, reliable probability and no action. A false negative appears in shoulder seasons that warm late: morning-available proxies under-register improvement, so contributions are modestly positive but remain below τ at the morning cut-off; a pragmatic mitigation is a lunchtime refresh in April–May and September.

Date	Pred prob	Pred label@ τ	True label	Case	Top contributing features (SHAP)
2024-05-11	0.992	1	1	TP	tsd_avg_roll30_mean (+0.090); suncream_90d_roll_std (+0.073); gb_daily_peak_nd_mw (+0.059); suncream_90d_ema (+0.043); annual_cos (+0.043)
2025-07-07	0.977	1	0	FP	suncream_90d_roll_std (+0.083); nd_avg_roll30_std (+0.073); annual_cos (+0.069); gb_daily_peak_nd_mw (+0.051); suncream_90d_ema (+0.047)
2024-02-02	0.0	0	0	TN	suncream_90d_roll_std (-0.043); annual_cos (-0.041); tsd_avg_roll30_mean (-0.035); bbq_lag_30d (-0.034); gb_daily_avg_nd_mw (-0.029)
2025-03-21	0.002	0	1	FN	suncream_90d_roll_std (-0.065); ped_roll30_std (-0.051); annual_cos (-0.036); tsd_avg_roll30_mean (-0.033); suncream_90d_ema (-0.029)

Table 7: Case Day Narratives

Across cases, a general rule holds: the system is most trustworthy when at least two factors agree in a seasonally favourable window; confidence is lowest when a single factor moves without corroboration. Calibration keeps such day's conservative, but threshold policy determines whether amber tips to green.

Stability, interactions, and caveats

Two interaction patterns recur in the explanations. First, weekend $\times$ mobility 7-day change: a rise in mobility contributes more on weekends, when people are freer to act. Second, intent 7-day change $\times$ demand EMA-7: spring surges in search intent add less when the demand context still looks winter-like. Several rules apply. Importances are associative, not causal; closely related features can share or swap importance; and all explanations reflect the data available by the operational cut-off, meaning late-day warming can be missed if the score is generated at 09:00.

Practical takeaway

For operations, weight weekly momentum over levels; treat intent-only spikes as amber unless mobility confirms; and in shoulder seasons run a midday refresh for borderline days. Figure 8 (top 20 permutation importances) and Figure 9 (SHAP summary) provide the evidence, while Table 7 offers recognisable examples the team can learn from.

4.6 Error analysis & Failure Modes

The error review begins from a practical claim: the costliest mistakes arise from two patterns. First, the model occasionally misses genuine *taps-off* days in the shoulder seasons when the proxies available at the morning cut-off lag real-world warming. Second, it sometimes raises false alarms when short-lived jumps in search interest are not backed up by mobility data. Figure 10 situates these issues on the precision–recall curve by showing how performance moves as the decision threshold (τ) is varied, while Table 7 organises the observed mistakes into plain categories with example dates and suggested remedies.

Missed positives mainly occur in April–May and late August–September. On these days, the morning features often understate what the afternoon becomes: search intent has not yet risen, mobility 7-day change and EMA-7 have not turned decisively positive, and the demand context remains spring-like. Local explanation plots typically show several small positive contributions that nevertheless leave the score just below τ . Operationally, these misses translate into under-provisioning—staff or stock that would have been scheduled if the day had been flagged as likely *taps-off*. A light-touch mitigation is to introduce a 12:00 local refresh in shoulder months for days that sit in the amber band at 09:00, upgrading to green only when at least two signal families (e.g., Trends and Mobility) are rising together. Complementing this, short-horizon features (1-day and 3-day changes alongside the 7-day and EMA-7) help late-morning surges tip borderline days. These steps raise recall where it matters most while preserving a single, transparent threshold.

Type	Rule	Count	Example dates
FN	Shoulder late-warming (Apr/May/Sep), low intent & low mobility	1	2025-05-31
FN	Other missed positives	43	2024-05-01, 2024-05-02, 2024-05-03
FP	Intent-only spike (high intent, low mobility)	32	2024-05-24, 2024-05-26, 2024-05-27
FP	Event-like mobility on weekend (high mobility, low intent)	0	
FP	Other false alarms	4	2025-05-03, 2025-05-05, 2025-05-06

Table 8: Error breakdown (counts & examples)

False alarms are concentrated on weekends where campaign-driven or event-driven spikes in “sun cream/BBQ/beer garden” searches are not corroborated by mobility. In such cases, intent pushes the score up and calendar effects amplify it, but mobility remains flat, and the demand context is neutral; post-calibration probabilities are conservative, yet some cases still sit just above tau. The operational cost here is over-preparation or over-promising. A simple confirmation rule addresses most of this: require both intent and mobility momentum (7-day change or EMA-7) for a green decision; otherwise hold at amber. A companion guardrail flags large day-over-day jumps in a single search term without multi-term agreement as “campaign-suspect”, keeping those days amber unless mobility corroborates within 24–48 hours. Together these checks reduce unnecessary actions with minimal impact on recall.

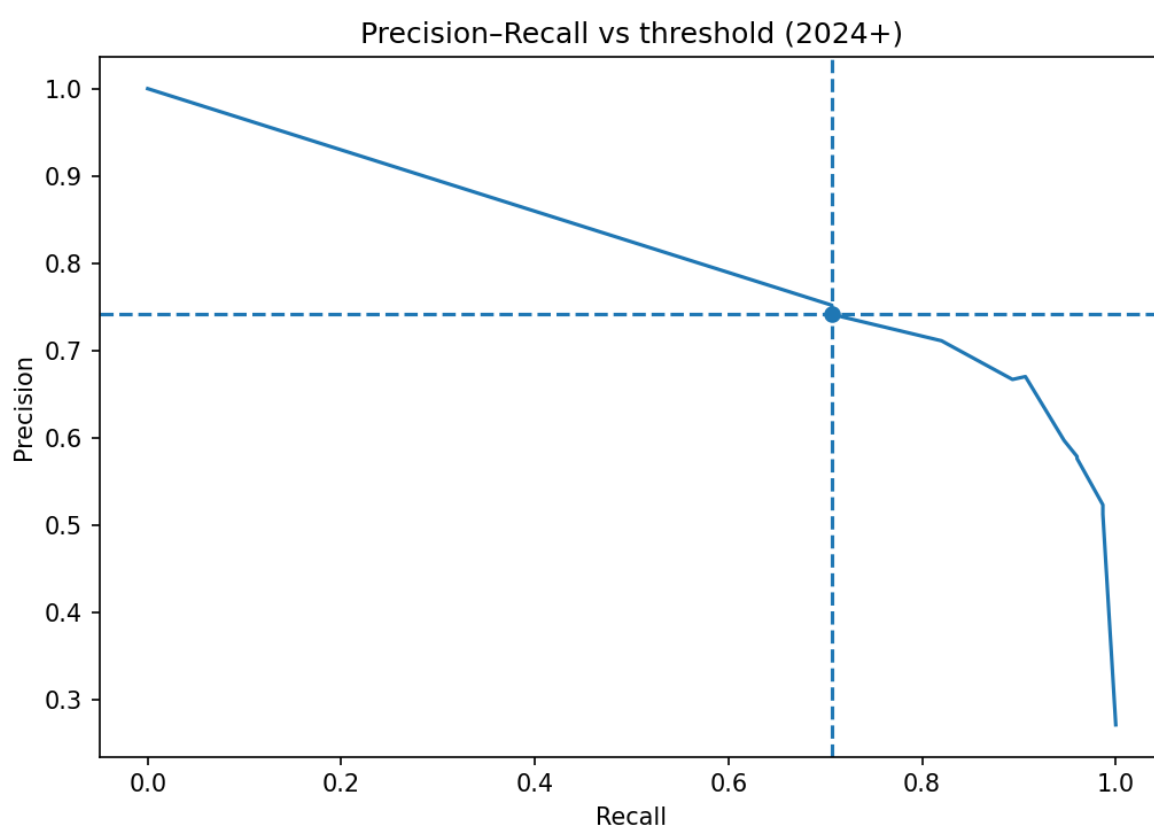


Figure 10: Precision-Recall vs chosen Threshold

Figure 10 makes the policy lever explicit. The chosen operating point ($\tau \approx 0.946$) sits at a sensible trade-off, delivering precision around 0.75 and recall around 0.71 on the 2024+ hold-out. The curve is locally steep near this point, meaning small threshold adjustments (± 0.02 – 0.05) move precision and recall by several percentage points. This supports a policy-led approach: when the cost of missing a warm day is higher (e.g., early summer), the midday upgrade rule effectively lowers the operational

bar for borderline cases; when false alarms are costlier (for example, winter weekends), the confirmation rule tightens decisions without changing the global threshold.

A minority of errors align with data and pipeline issues, such as temporary counter gaps in mobility or re-baselining in Google Trends. These situations can generate both misses and false alarms if features are partially missing and imputed towards neutral. Three factors keep behaviour predictable: include explicit missingness indicators so the model naturally reduces confidence when a feed is absent; fall back to the best validated ablation stage when an input family is unavailable (as established in Section 4.4); and monitor feed freshness and distribution shift so calibration can be refreshed on schedule.

In summary, Table 7 shows that the observed mistakes are understandable and fixable without altering the core model. A single global threshold can be retained for transparency, complemented by two lightweight procedures: a midday refreshes that upgrade shoulder-season ambers only on multi-family confirmation, and an intent-plus-mobility confirmation rule that prevents campaign-only spikes from turning green. Figure 10 confirms that these procedures move the operating point along a well-behaved precision–recall curve rather than redefining the model. Operationally, the guidance is clear: keep the current tau, apply the two procedures, and monitor shoulder-season recall, weekend precision, calibration drift and feed freshness as part of routine governance.

4.7 Limitations

This study has several limitations that should temper interpretation. First, proxy bias is inherent: digital traces (search, mobility, electricity demand) reflect the behaviour of specific user groups and infrastructures, which may under-represent some demographics or neighbourhoods. Second, the target concept can drift. Marketing campaigns, device adoption, tourism patterns, or post-pandemic habits may change how proxies relate to “taps-off,” degrading both discrimination and calibration over time. Third, the pipeline depends on external feeds; outages, delayed publication, or re-baselining (e.g., Google Trends normalisation) can produce brittle periods or require degraded-mode operation. Fourth, the approach is predictive rather than causal. High contributions indicate association, not effect; policy decisions should therefore treat scores as likelihoods and pair them with domain judgment. Fifth, geographic generalisability is limited: the model is trained for Glasgow, and transfer to other cities would require re-engineering features, re-tuning thresholds, and re-calibration against local behaviour and calendars. Finally, while the data are aggregated and contain no personally identifiable information, ethical use still requires clear documentation of sources, retention limits, and compliance with platform terms and institutional policies. Mitigations include scheduled recalibration and quarterly retraining, continuous drift and freshness monitoring, ablation-based fallbacks when a feed is missing, and a conservative operating policy (confirmation rules, optional midday refresh) to reduce costly errors during proxy instability.

4.8 Summary of key takeaways

This project built a simple, low-cost way to spot likely “taps-off” days without using weather data. Instead, it looks at public signals people already create: what they search for online (e.g., sun cream, BBQ), how much people are out and about (city cycling/pedestrian counts), how the country’s electricity use shifts (a clue to the season), and the calendar (weekends and holidays).

The system was trained on 2019-2023 and then tested on 2024 onwards. It performs well: it ranks days in a sensible order from “unlikely” to “very likely,” and the percentages it outputs are realistic enough to support decisions. With a single cut-off, the scores can be turned into green/amber/red bands for straightforward planning.

What matters most is momentum: rising searches and rising movement—especially heading into a weekend—are strong signs a “taps-off” day is coming. Electricity demand and the calendar add steady context.

Across different years and seasons the model stays reliable. When it does slip, it’s usually because (1) spring/autumn days warm up after the morning data are collected, or (2) search interest jumps briefly without people actually going out. Two light fixes handle this: a midday update in shoulder seasons, and a rule that asks for both intent and movement before calling a firm “green.” With simple monitoring and fallbacks if a data feed is down, the approach is practical and ready to use.

5.0 Conclusion

This dissertation set out a simple question: can “taps aff” days in Glasgow be predicted without using any weather data? It approached the problem as a daily yes/no decision and relied only on public, fast signals that reflect how people plan and behave—what they search for (e.g., “bbq”, “suncream”), how much people are out and about (cycling and pedestrian counts), and a broad sense of the season from Great Britain electricity demand—plus basic calendar information. The features focused on recent changes (for example, week-on-week rises) rather than raw levels so the model would react to fresh momentum. Models were tuned on earlier years (2018–2023) and tested strictly on later data (2024+), with one decision cut-off set using only the training years. Probabilities were calibrated on the training period and then held fixed.

The results show that a proxy-only approach can work. The best tree-based model clearly beat fair baselines (majority, calendar-only, and linear models) on ranking measures and produced usable probabilities for day-to-day choices. When converted into a single decision threshold, the model achieved a sensible balance of precision (how often “yes” is right) and recall (how many true “taps aff” days it catches). A step-by-step comparison of signal sources explained where performance comes from: search intent does most of the heavy lifting, electricity demand gives helpful seasonal context, and mobility provides a smaller but useful lift—especially on borderline weekend days in spring and early summer. Year-by-year checks suggested the model’s ranking ability is stable; probability quality moves with how common “taps aff” days are and benefits from routine calibration. Simple charts (ROC/PR curves, reliability plots) and examples made the model’s behaviour transparent.

The study also surfaced where mistakes matter most. Missed days tend to happen in the “shoulder” months when mornings start cool but afternoons turn pleasant and the proxies lag. False alarms often come from short-lived bursts in search interest that are not backed up by mobility. Both issues are fixable with light operational rules: a lunchtime refresh in April–May and September for borderline mornings, and a confirmation rule that asks for both intent and mobility to be rising before a day is called green. These changes improve decisions without adding new model complexity. Clear run-book guidance, simple green/amber/red bands, and periodic monitoring (for score reliability and data freshness) make the system practical to operate.

There are limits. Proxies can be biased or change over time; data feeds can go down; and the model is predictive rather than causal. The work is trained on Glasgow, so moving to other cities would require re-tuning. However, all inputs are aggregated and public, and no personal data are used.

The immediate next step is to harden the service for everyday use. Practically, that means trialling the lunchtime refresh in shoulder months and adopting the “intent + mobility” confirmation rule so campaign-only spikes do not trigger green days. Adding a light events layer (major fixtures, festivals)

and a simple detector for marketing-led search jumps would further reduce weekend false alarms. Probability quality can be strengthened by updating the calibration on a rolling window and, if useful, attaching uncertainty bands to each score. To support earlier staffing and stock decisions, the label and horizon could be varied (for example, predicting “evening taps-off” or looking one or two days ahead). It is also worth testing small modelling tweaks—such as monotonic boosting or shallow stacking—to see if sharper, still-trustworthy probabilities are possible without losing clarity. Finally, to check generalisability, the pipeline should be ported to other UK cities with similar data feeds, while putting in place production-grade monitoring: automatic checks for data freshness and distribution shift, reliability dashboards, and simple fallbacks when a source is down. Together, these steps would reduce the small but costly errors, improve trust in the probabilities, and turn the prototype into a dependable, weather-free planning tool.

6.0 Bibliography

Adam, T. and Goyal, V.K. (2008) 'The Investment Opportunity Set and Its Proxy Variables', *Journal of Financial Research*, 31(1), pp. 41–63. Available at: <https://doi.org/10.1111/j.1475-6803.2008.00231.x>.

Angrist, J.D. and Pischke, J.-S. (2009) *Mostly Harmless Econometrics: An Empiricist's Companion*. Princeton University Press.

Arora, S. and Taylor, J.W. (2013) 'Short-Term Forecasting of Anomalous Load Using Rule-Based Triple Seasonal Methods', *IEEE Transactions on Power Systems*, 28(3), pp. 3235–3242. Available at: <https://doi.org/10.1109/TPWRS.2013.2252929>.

Askitas, N. and Zimmermann, K.F. (2009) 'Google Econometrics and Unemployment Forecasting', *Applied Economics Quarterly*, (2), pp. 107–120. Available at: <https://doi.org/10.3790/aeq.55.2.107>.

Atadeger, A. et al. (2023) 'Deep Learning-Based and Kernel-Based Proxy Models for Nonlinearly Constrained Life-Cycle Production Optimization', in. *SPE Reservoir Characterisation and Simulation Conference and Exhibition*, OnePetro. Available at: <https://doi.org/10.2118/212690-MS>.

Baltagi, B.H. (ed.) (2008) 'Simultaneous Equations Model', in *Econometrics*. Berlin, Heidelberg: Springer, pp. 253–294. Available at: https://doi.org/10.1007/978-3-540-76516-5_11.

Bergmeir, C. and Benítez, J.M. (2012) 'On the use of cross-validation for time series predictor evaluation', *Information Sciences*, 191, pp. 192–213. Available at: <https://doi.org/10.1016/j.ins.2011.12.028>.

Birks, H.H. and Birks, H.J.B. (2006) 'Multi-proxy studies in palaeolimnology', *Vegetation History and Archaeobotany*, 15(4), pp. 235–251. Available at: <https://doi.org/10.1007/s00334-006-0066-6>.

Böcker, L., Dijst, M. and Prillwitz, J. (2013) 'Impact of Everyday Weather on Individual Daily Travel Behaviours in Perspective: A Literature Review', *Transport Reviews*, 33(1), pp. 71–91. Available at: <https://doi.org/10.1080/01441647.2012.747114>.

Box, G.E.P. et al. (2015) *Time Series Analysis: Forecasting and Control*. John Wiley & Sons.

Bradley, A.A., Schwartz, S.S. and Hashino, T. (2008) 'Sampling Uncertainty and Confidence Intervals for the Brier Score and Brier Skill Score'. Available at: <https://doi.org/10.1175/2007WAF2007049.1>.

Breiman, L. (2001) 'Random Forests', *Machine Learning*, 45(1), pp. 5–32. Available at: <https://doi.org/10.1023/A:1010933404324>.

- Bröcker, J. (2009) 'Reliability, sufficiency, and the decomposition of proper scores', *Quarterly Journal of the Royal Meteorological Society*, 135(643), pp. 1512–1519. Available at: <https://doi.org/10.1002/qj.456>.
- Caiado, J. (2010) 'Performance of Combined Double Seasonal Univariate Time Series Models for Forecasting Water Demand', *Journal of Hydrologic Engineering*, 15(3), pp. 215–222. Available at: [https://doi.org/10.1061/\(ASCE\)HE.1943-5584.0000182](https://doi.org/10.1061/(ASCE)HE.1943-5584.0000182).
- Chen, T. and Guestrin, C. (2016) 'XGBoost: A Scalable Tree Boosting System', in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. New York, NY, USA: Association for Computing Machinery (KDD '16), pp. 785–794. Available at: <https://doi.org/10.1145/2939672.2939785>.
- Choi, H. and Varian, H. (2012) 'Predicting the Present with Google Trends', *Economic Record*, 88(s1), pp. 2–9. Available at: <https://doi.org/10.1111/j.1475-4932.2012.00809.x>.
- Coleman, C. *et al.* (2020) 'Selection via Proxy: Efficient Data Selection for Deep Learning'. arXiv. Available at: <https://doi.org/10.48550/arXiv.1906.11829>.
- Davis, J. and Goadrich, M. (2006) 'The relationship between Precision-Recall and ROC curves', in *Proceedings of the 23rd international conference on Machine learning*. New York, NY, USA: Association for Computing Machinery (ICML '06), pp. 233–240. Available at: <https://doi.org/10.1145/1143844.1143874>.
- De Felice, M., Alessandri, A. and Catalano, F. (2015) 'Seasonal climate forecasts for medium-term electricity demand forecasting', *Applied Energy*, 137, pp. 435–444. Available at: <https://doi.org/10.1016/j.apenergy.2014.10.030>.
- Deaner, B. (2023) 'Proxy Controls and Panel Data'. arXiv. Available at: <https://doi.org/10.48550/arXiv.1810.00283>.
- El-Assi, W., Salah Mahmoud, M. and Nurul Habib, K. (2017) 'Effects of built environment and weather on bike sharing demand: a station level analysis of commercial bike sharing in Toronto', *Transportation*, 44(3), pp. 589–613. Available at: <https://doi.org/10.1007/s11116-015-9669-z>.
- Elkan, C. (2001) 'The Foundations of Cost-Sensitive Learning', in *International joint conference on artificial intelligence*, 17(1), pp. 973–978.
- Feng, Y., Zhang, Y. and Wang, Y. (2024) 'Out-of-sample volatility prediction: Rolling window, expanding window, or both?', *Journal of Forecasting*, 43(3), pp. 567–582. Available at: <https://doi.org/10.1002/for.3046>.
- Friedman, J.H. (2001) 'Greedy Function Approximation: A Gradient Boosting Machine', *The Annals of Statistics*, 29(5), pp. 1189–1232.

Gebhart, K. and Noland, R.B. (2014) 'The impact of weather conditions on bikeshare trips in Washington, DC', *Transportation*, 41(6), pp. 1205–1225. Available at: <https://doi.org/10.1007/s11116-014-9540-7>.

Gneiting, T. and Raftery, A.E. (2007) 'Strictly Proper Scoring Rules, Prediction, and Estimation', *Journal of the American Statistical Association*, 102(477), pp. 359–378. Available at: <https://doi.org/10.1198/016214506000001437>.

Greiner, R., Grove, A.J. and Roth, D. (2002) 'Learning cost-sensitive active classifiers☆', *Artificial Intelligence*, 139(2), pp. 137–174. Available at: [https://doi.org/10.1016/S0004-3702\(02\)00209-6](https://doi.org/10.1016/S0004-3702(02)00209-6).

Hasanov, M., Wolter, M. and Glende, E. (2022) 'Time Series Data Splitting for Short-Term Load Forecasting', in *PESS + PELSS 2022; Power and Energy Student Summit. PESS + PELSS 2022; Power and Energy Student Summit*, pp. 1–6. Available at: <https://ieeexplore.ieee.org/abstract/document/10104242> (Accessed: 19 August 2025).

Hong, T. and Fan, S. (2016) 'Probabilistic electric load forecasting: A tutorial review', *International Journal of Forecasting*, 32(3), pp. 914–938. Available at: <https://doi.org/10.1016/j.ijforecast.2015.11.011>.

Hyndman, R.J. and Athanasopoulos, G. (2018) *Forecasting: principles and practice*. OTexts.

Jaber, A. and Csonka, B. (2023) 'Investigating the temporal differences among bike-sharing users through comparative analysis based on count, time series, and data mining models', *Alexandria Engineering Journal*, 77, pp. 1–13. Available at: <https://doi.org/10.1016/j.aej.2023.06.087>.

Kaufman, S. et al. (2012) 'Leakage in data mining: Formulation, detection, and avoidance', *ACM Trans. Knowl. Discov. Data*, 6(4), p. 15:1-15:21. Available at: <https://doi.org/10.1145/2382577.2382579>.

Ke, G. et al. (2017) 'LightGBM: A Highly Efficient Gradient Boosting Decision Tree', in *Advances in Neural Information Processing Systems*. Curran Associates, Inc. Available at: <https://proceedings.neurips.cc/paper/2017/hash/6449f44a102fde848669bdd9eb6b76fa-Abstract.html> (Accessed: 19 August 2025).

Khalid, S., Khalil, T. and Nasreen, S. (2014) 'A survey of feature selection and feature extraction techniques in machine learning', in *2014 Science and Information Conference. 2014 Science and Information Conference*, pp. 372–378. Available at: <https://doi.org/10.1109/SAI.2014.6918213>.

Kloberdanz, E., Kloberdanz, K.G. and Le, W. (2022) 'DeepStability: a study of unstable numerical methods and their solutions in deep learning', in *Proceedings of the 44th International Conference on Software Engineering*. New York, NY, USA: Association for Computing Machinery (ICSE '22), pp. 586–597. Available at: <https://doi.org/10.1145/3510003.3510095>.

Kwon, B.-S., Park, R.-J. and Song, K.-B. (2019) 'Weekly Peak Load Forecasting for 104 Weeks Using Deep Learning Algorithm', in *2019 IEEE PES Asia-Pacific Power and Energy Engineering Conference (APPEEC)*. *2019 IEEE PES Asia-Pacific Power and Energy Engineering Conference (APPEEC)*, pp. 1–4. Available at: <https://doi.org/10.1109/APPEEC45492.2019.8994442>.

Lampos, V. et al. (2021) 'Tracking COVID-19 using online search', *npj Digital Medicine*, 4(1), p. 17. Available at: <https://doi.org/10.1038/s41746-021-00384-w>.

Lazer, D. et al. (2014) 'The Parable of Google Flu: Traps in Big Data Analysis', *Science*, 343(6176), pp. 1203–1205. Available at: <https://doi.org/10.1126/science.1248506>.

Lin, Y.-D. et al. (2022) 'Multi-datasource machine learning in intrusion detection: Packet flows, system logs and host statistics', *Journal of Information Security and Applications*, 68, p. 103248. Available at: <https://doi.org/10.1016/j.jisa.2022.103248>.

Liu, C., Susilo, Y.O. and Karlström, A. (2017) 'Weather variability and travel behaviour – what we know and what we do not know', *Transport Reviews*, 37(6), pp. 715–741. Available at: <https://doi.org/10.1080/01441647.2017.1293188>.

Liu, Y., Wang, Y. and Zhang, J. (2012) 'New Machine Learning Algorithm: Random Forest', in B. Liu, M. Ma, and J. Chang (eds) *Information Computing and Applications*. Berlin, Heidelberg: Springer, pp. 246–252. Available at: https://doi.org/10.1007/978-3-642-34062-8_32.

Louppe, G. et al. (2013) 'Understanding variable importances in forests of randomized trees', in *Advances in Neural Information Processing Systems*. Curran Associates, Inc. Available at: https://proceedings.neurips.cc/paper_files/paper/2013/hash/e3796ae838835da0b6f6ea37bcf8bcb7-Abstract.html (Accessed: 19 August 2025).

Lubotsky, D. and Wittenberg, M. (2006) 'Interpretation of Regressions with Multiple Proxies', *The Review of Economics and Statistics*, 88(3), pp. 549–562. Available at: <https://doi.org/10.1162/rest.88.3.549>.

Lundberg, S.M. and Lee, S.-I. (2017) 'A Unified Approach to Interpreting Model Predictions', in *Advances in Neural Information Processing Systems*. Curran Associates, Inc. Available at: <https://proceedings.neurips.cc/paper/2017/hash/8a20a8621978632d76c43dfd28b67767-Abstract.html> (Accessed: 19 August 2025).

Lunsford, K.G. (2015) 'Identifying Structural VARs with a Proxy Variable and a Test for a Weak Proxy', *Working Paper [Preprint]*, (15–28). Available at: <https://doi.org/10.26509/frbc-wp-201528>.

Majka, M. (2024) 'ARIMAX: Time Series Forecasting with External Variables'.

Meteostat (2025) *Meteostat, The Weather's Record Keeper*. Available at: <https://meteostat.net/en/> (Accessed: 23 August 2025).

Miao, W., Geng, Z. and Tchetgen Tchetgen, E.J. (2018) 'Identifying causal effects with proxy variables of an unmeasured confounder', *Biometrika*, 105(4), pp. 987–993. Available at: <https://doi.org/10.1093/biomet/asy038>.

Miller, K. and Wilson, W.W. (2023) 'Changes in rail rates for U.S. commodity grain shipments over time', *Research in Transportation Economics*, 102, p. 101359. Available at: <https://doi.org/10.1016/j.retrec.2023.101359>.

Miller, S., Curran, K. and Lunney, T. (2021) 'Detection of Anonymising Proxies Using Machine Learning', *International Journal of Digital Crime and Forensics (IJDCF)*, 13(6), pp. 1–17. Available at: <https://doi.org/10.4018/IJDCF.286756>.

Mills, T. (2012) *A Very British Affair: Six Britons and the Development of Time Series Analysis During the 20th Century*. Springer.

Moore, B. and McKee, J.J. (2020) 'The impact of seasonality on multi-scale feature extraction techniques', *International Journal of Digital Earth*, 13(1), pp. 9–21. Available at: <https://doi.org/10.1080/17538947.2018.1556354>.

NESO (2025) *Historic Demand Data | National Energy System Operator*. Available at: <https://www.neso.energy/data-portal/historic-demand-data> (Accessed: 23 August 2025).

Niculescu-Mizil, A. and Caruana, R. (2005) 'Predicting good probabilities with supervised learning', in *Proceedings of the 22nd international conference on Machine learning*. New York, NY, USA: Association for Computing Machinery (ICML '05), pp. 625–632. Available at: <https://doi.org/10.1145/1102351.1102430>.

Nuti, S.V. et al. (2014) 'The Use of Google Trends in Health Care Research: A Systematic Review', *PLOS ONE*, 9(10), p. e109583. Available at: <https://doi.org/10.1371/journal.pone.0109583>.

Ojeda, F.M. et al. (2023) 'Calibrating machine learning approaches for probability estimation: A comprehensive comparison', *Statistics in Medicine*, 42(29), pp. 5451–5478. Available at: <https://doi.org/10.1002/sim.9921>.

Pardo, A., Meneu, V. and Valor, E. (2002) 'Temperature and seasonality influences on Spanish electricity load', *Energy Economics*, 24(1), pp. 55–70. Available at: [https://doi.org/10.1016/S0140-9883\(01\)00082-2](https://doi.org/10.1016/S0140-9883(01)00082-2).

Paul, A. and Schäfer-Neth, C. (2005) 'How to combine sparse proxy data and coupled climate models', *Quaternary Science Reviews*, 24(7), pp. 1095–1107. Available at: <https://doi.org/10.1016/j.quascirev.2004.05.010>.

Platt, J. (1999) 'Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods'.

Prakash, M. and Singaravel, G. (2015) 'An approach for prevention of privacy breach and information leakage in sensitive data mining', *Computers & Electrical Engineering*, 45, pp. 134–140. Available at: <https://doi.org/10.1016/j.compeleceng.2015.01.016>.

Probst, P., Wright, M.N. and Boulesteix, A.-L. (2019) 'Hyperparameters and tuning strategies for random forest', *WIREs Data Mining and Knowledge Discovery*, 9(3), p. e1301. Available at: <https://doi.org/10.1002/widm.1301>.

Pytrends (2025) *pytrends: Pseudo API for Google Trends*. Available at: <https://pypi.org/project/pytrends/>.

Quach, J. and Malekian, R. (2020) 'Exploring the weather impact on bike sharing usage through a clustering analysis'. arXiv. Available at: <https://doi.org/10.48550/arXiv.2008.07249>.

Ralston Fonseca, F. *et al.* (2019) 'Seasonal effects of climate change on intra-day electricity demand patterns', *Climatic Change*, 154(3), pp. 435–451. Available at: <https://doi.org/10.1007/s10584-019-02413-w>.

Saito, T. and Rehmsmeier, M. (2015) 'The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets', *PLOS ONE*, 10(3), p. e0118432. Available at: <https://doi.org/10.1371/journal.pone.0118432>.

Salasakar, S., Prakash, S. and Thakur, G. (2024) 'Recent Trends in Proxy Model Development for Well Placement Optimization Employing Machine Learning Techniques', *Modelling*, 5(4), pp. 1808–1823. Available at: <https://doi.org/10.3390/modelling5040094>.

Schultz, J.A. *et al.* (2015) 'Sensitivity of proxies on non-linear interactions in the climate system', *Scientific Reports*, 5(1), p. 18560. Available at: <https://doi.org/10.1038/srep18560>.

Segal, M.R. (2004) 'Machine Learning Benchmarks and Random Forest Regression'. Available at: <https://escholarship.org/uc/item/35x3v9t4> (Accessed: 19 August 2025).

Selig, J.P., Preacher, K.J. and Little, T.D. (2012) 'Modeling Time-Dependent Association in Longitudinal Data: A Lag as Moderator Approach', *Multivariate Behavioral Research*, 47(5), pp. 697–716. Available at: <https://doi.org/10.1080/00273171.2012.715557>.

Seltzer, R.G.N. (2021) 'The Perilous Use of Proxy Variables', *Evaluation & the Health Professions*, 44(4), pp. 428–435. Available at: <https://doi.org/10.1177/0163278720903358>.

Shwartz-Ziv, R. and Armon, A. (2022) 'Tabular data: Deep learning is not all you need', *Information Fusion*, 81, pp. 84–90. Available at: <https://doi.org/10.1016/j.inffus.2021.11.011>.

Siddiqi, N. (2012) *Credit Risk Scorecards: Developing and Implementing Intelligent Credit Scoring*. John Wiley & Sons.

Sowers, E.A., Ciccantell, P.S. and Smith, D.A. (2017) 'Are Transport and Raw Materials Nodes in Global Commodity Chains Potential Places for Worker/Movement Organization?', *Journal of Labor and Society*, 20(2), pp. 185–205. Available at: <https://doi.org/10.1111/wusa.12288>.

Spencer, J., Bowden, R. and Hadfield, S. (2020) 'Same Features, Different Day: Weakly Supervised Feature Learning for Seasonal Invariance', in. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 6459–6468. Available at: https://openaccess.thecvf.com/content_CVPR_2020/html/Spencer_Same_Features_Different_Day_Weakly_Supervised_Feature_Learning_for_Seasonal_CVPR_2020_paper.html (Accessed: 19 August 2025).

Stehouwer, N. et al. (2025) 'Validity and reliability of Brier scoring for assessment of probabilistic diagnostic reasoning', *Diagnosis*, 12(1), pp. 53–60. Available at: <https://doi.org/10.1515/dx-2023-0109>.

Tashman, L.J. (2000) 'Out-of-sample tests of forecasting accuracy: an analysis and review', *International Journal of Forecasting*, 16(4), pp. 437–450. Available at: [https://doi.org/10.1016/S0169-2070\(00\)00065-0](https://doi.org/10.1016/S0169-2070(00)00065-0).

Taylor, J.W. (2003) 'Short-term electricity demand forecasting using double seasonal exponential smoothing', *Journal of the Operational Research Society*, 54(8), pp. 799–805. Available at: <https://doi.org/10.1057/palgrave.jors.2601589>.

Taylor, J.W. (2010) 'Triple seasonal methods for short-term electricity demand forecasting', *European Journal of Operational Research*, 204(1), pp. 139–152. Available at: <https://doi.org/10.1016/j.ejor.2009.10.003>.

Taylor, S.J. and Letham, B. (2018) 'Forecasting at Scale', *The American Statistician*, 72(1), pp. 37–45. Available at: <https://doi.org/10.1080/00031305.2017.1380080>.

Ting, K.M. et al. (2011) 'Feature-subspace aggregating: ensembles for stable and unstable learners', *Machine Learning*, 82(3), pp. 375–397. Available at: <https://doi.org/10.1007/s10994-010-5224-5>.

uSmart (2025) *uSmart, Cycling Open Data*. Available at: <https://usmart.io/org/cyclingscotland/> (Accessed: 23 August 2025).

Vosen, S. and Schmidt, T. (2011) 'Forecasting private consumption: survey-based indicators vs. Google trends', *Journal of Forecasting*, 30(6), pp. 565–578. Available at: <https://doi.org/10.1002/for.1213>.

Waddell, C. (2019) 'Colin's brain dump', *Taps Aff - How it got to Version 3.0*, March. Available at: <https://colinwaddell.com/articles/taps-aff-v3-0>.

Wadud, Z. (2014) 'Cycling in a changed climate', *Journal of Transport Geography*, 35, pp. 12–20. Available at: <https://doi.org/10.1016/j.jtrangeo.2014.01.001>.

Wang, X. *et al.* (2014) 'Estimating Mixed-Mode Urban Trail Traffic Using Negative Binomial Regression Models', *Journal of Urban Planning and Development*, 140(1), p. 04013006. Available at: [https://doi.org/10.1061/\(ASCE\)UP.1943-5444.0000157](https://doi.org/10.1061/(ASCE)UP.1943-5444.0000157).

Wessel, J. (2022) 'Cycling in the dark – the impact of Standard Time and Daylight Saving Time on bicycle ridership', *PNAS Nexus*, 1(1), p. pgab006. Available at: <https://doi.org/10.1093/pnasnexus/pgab006>.

Wooldridge, J.M. (2010) *Econometric Analysis of Cross Section and Panel Data, second edition*. MIT Press.

Zadrozny, B. and Elkan, C. (2002) 'Transforming classifier scores into accurate multiclass probability estimates', in *Proceedings of the eighth ACM SIGKDD international conference on Knowledge discovery and data mining*. New York, NY, USA: Association for Computing Machinery (KDD '02), pp. 694–699. Available at: <https://doi.org/10.1145/775047.775151>.

7.0 Appendices

Appendix A Data Collection

A1) Google Trends - BBQ

```
# -----  
# Google Trends - BBQ  
# Daily. takes estimated 2-5 minutes depending on data and your device  
# -----  
  
from pytrends.dailydata import get_daily_data  
from pytrends.exceptions import TooManyRequestsError  
from datetime import datetime  
from tqdm.auto import tqdm  
import pandas as pd, time, random  
  
KW          = "bbq"  
GEO         = "GB-SCT"          # Scotland  
START_Y     = 2018  
TODAY       = datetime.today()  
END_Y       = TODAY.year  
  
MAX_RETRY   = 6                  # exponential back-off cap  
BASE_WAIT   = 5                  # seconds  
  
frames = []  
  
for yr in tqdm(range(START_Y, END_Y + 1), desc="Years"):  
    m_start = 1  
    m_end   = 12 if yr < END_Y else TODAY.month  
    for attempt in range(MAX_RETRY):  
        try:
```

```

        chunk = get_daily_data(KW, yr, m_start, yr, m_end, geo=GEO,
wait_time=0)

        if not chunk.empty:

            frames.append(chunk)

            time.sleep(random.uniform(8, 12))    # friendly pause

            break                                # success → next year

    except TooManyRequestsError:

        wait = BASE_WAIT * (2 ** attempt) + random.uniform(0, 3)

        print(f"429 for {yr}, retry in {wait:.1f}s (attempt
{attempt+1})")

        time.sleep(wait)

    else:

        print(f" Skipped {yr} after {MAX_RETRY} retries")

# ——— tidy & save ———

df = (pd.concat(frames)

      .reset_index(names="Date")

      .rename(columns={KW: "bbq_search_interest"})

      .assign(Date=lambda d: pd.to_datetime(d["Date"]))

      .loc[:, ["Date", "bbq_search_interest"]]

      .sort_values("Date")

      .reset_index(drop=True))

# fill any missing calendar days with 0

full = pd.date_range(df["Date"].min(), df["Date"].max(), freq="D")

df   = (df.set_index("Date")

      .reindex(full)

      .fillna(0)

      .rename_axis("Date")

      .reset_index())

```

```
out_path = "google_trends_bbq_daily_glasgow.csv"
df.to_csv(out_path, index=False)

print(f" Saved {len(df):,} rows → {out_path}")
df.head()
```

A2) Google Trends – suncream

```
# -----  
# Google Trends daily data for "suncream" in Scotland  
# -----  
  
from pytrends.dailydata import get_daily_data  
from pytrends.exceptions import TooManyRequestsError    # ← correct import  
from datetime import datetime  
from tqdm.auto import tqdm  
import pandas as pd, time, random  
  
KW    = "suncream"  
GEO   = "GB-SCT"          # Scotland  
START_Y = 2018  
TODAY  = datetime.today()  
END_Y   = TODAY.year  
  
MAX_RETRY = 6  
BASE_WAIT = 5              # seconds between exponential-back-off retries  
  
frames = []  
  
for yr in tqdm(range(START_Y, END_Y + 1), desc="Years"):  
    m_start = 1  
    m_end   = 12 if yr < END_Y else TODAY.month  
    for attempt in range(MAX_RETRY):  
        try:  
            chunk = get_daily_data(KW, yr, m_start, yr, m_end, geo=GEO,  
wait_time=0)  
            if not chunk.empty:  
                frames.append(chunk)  
            # polite pause 8-12 s before next request
```

```

        time.sleep(random.uniform(8, 12))
        break # success → exit retry
loop

except TooManyRequestsError:
    wait = BASE_WAIT * (2 ** attempt) + random.uniform(0, 3)
    print(f"429 for {yr}, retry in {wait:.1f}s (attempt {attempt+1})")
    time.sleep(wait)
else:
    print(f" Skipped {yr} after {MAX_RETRY} retries")

# ————— tidy & save —————
df = (pd.concat(frames)
      .reset_index(names="Date")
      .rename(columns={KW: "suncream_search_interest"})
      .assign(Date=lambda d: pd.to_datetime(d["Date"]))
      .loc[:, ["Date", "suncream_search_interest"]]
      .sort_values("Date")
      .reset_index(drop=True))

# fill calendar gaps with 0
full = pd.date_range(df["Date"].min(), df["Date"].max(), freq="D")
df = (df.set_index("Date")
     .reindex(full)
     .fillna(0)
     .rename_axis("Date")
     .reset_index())

out_path = "google_trends_suncream_daily_glasgow.csv"
df.to_csv(out_path, index=False)
print(f"Saved {len(df):,} rows → {out_path}")

```



```
df.head()
```

A3) NESO – Energy Demand & Feature Engineering

#NESO - Energy Demand

```
import pandas as pd, requests, io, datetime as dt

# Year -> CSV URL map (taken from the portal's "Historic Demand Data"
# page).
YEAR_URLS = {

    2018: "https://api.neso.energy/dataset/8f2fe0af-871c-488d-8bad-
960426f24601/resource/fcb12133-0db0-4f27-a4a5-
1669fd9f6d33/download/demanddata_2018.csv",

    2019: "https://api.neso.energy/dataset/8f2fe0af-871c-488d-8bad-
960426f24601/resource/dd9de980-d724-415a-b344-
d8ae11321432/download/demanddata_2019.csv",

    2020: "https://api.neso.energy/dataset/8f2fe0af-871c-488d-8bad-
960426f24601/resource/33ba6857-2a55-479f-9308-
e5c4c53d4381/download/demanddata_2020.csv",

    2021: "https://api.neso.energy/dataset/8f2fe0af-871c-488d-8bad-
960426f24601/resource/18c69c42-f20d-46f0-84e9-
e279045befc6/download/demanddata_2021.csv",

    2022: "https://api.neso.energy/dataset/8f2fe0af-871c-488d-8bad-
960426f24601/resource/bb44a1b5-75b1-4db2-8491-
257f23385006/download/demanddata_2022.csv",

    2023: "https://api.neso.energy/dataset/8f2fe0af-871c-488d-8bad-
960426f24601/resource/bf5ab335-9b40-4ea4-b93a-
ab4af7bce003/download/demanddata_2023.csv",

    2024: "https://api.neso.energy/dataset/8f2fe0af-871c-488d-8bad-
960426f24601/resource/f6d02c0f-957b-48cb-82ee-
09003f2ba759/download/demanddata_2024.csv",

    2025: "https://api.neso.energy/dataset/8f2fe0af-871c-488d-8bad-
960426f24601/resource/b2bde559-3455-4021-b179-
dfe60c0337b0/download/demanddata_2025.csv",

}

frames = []

for y, url in YEAR_URLS.items():

    r = requests.get(url, timeout=60)
```

```

r.raise_for_status()

df = pd.read_csv(io.BytesIO(r.content))

# SETTLEMENT_DATE, SETTLEMENT_PERIOD

df["datetime_utc"] = pd.to_datetime(df["SETTLEMENT_DATE"]) +
pd.to_timedelta((df["SETTLEMENT_PERIOD"]-1)*30, unit="m")

df = df.set_index("datetime_utc").sort_index()

# demand measure

frames.append(df[["ND", "TSD"]])

hh = pd.concat(frames).sort_index()

# Daily aggregates

daily = pd.DataFrame({
    "gb_daily_avg_nd_mw": hh["ND"].resample("D").mean(),
    "gb_daily_peak_nd_mw": hh["ND"].resample("D").max(),
    "gb_daily_energy_nd_mwh": hh["ND"].resample("D").mean() * 24,    #
    approx
    "gb_daily_avg_tsd_mw": hh["TSD"].resample("D").mean(),
    "gb_daily_peak_tsd_mw": hh["TSD"].resample("D").max(),
    "gb_daily_energy_tsd_mwh": hh["TSD"].resample("D").mean() * 24, #
    approx
}).reset_index().rename(columns={"datetime_utc": "date"})

# Keep 2018-01-01..today

today = dt.date.today()

daily = daily[(daily["date"] >= "2018-01-01") & (daily["date"] <=
pd.Timestamp(today))]

daily.to_csv("gb_demand_daily_from_historic.csv", index=False)

print(daily.shape, daily.head())

```

A4) Mobility data

```
# Glasgow mobility – combine & gap-fill (2018–2025)
# -----
# Output: glasgow_mobility_daily_2018_2025_gapfilled.csv

import pandas as pd
import numpy as np
from pathlib import Path
from typing import List

# 1) Files to combine (edit here if names/paths change)
FILES: List[str] = [
    "2018 cycle.csv",
    "2019 02 Sept- 2020 30 April.csv",
    "2019 Jan-September cycle.csv",
    "2020 01 May - 2020 30 November.csv",
    "2020 01 Dec - 2021 31 May.csv",
    "2021 01 June - 2021 31 Oct.csv",
    "2021 01 Nov - 2022 31 March.csv",
    "2022 01 April - 2022 31 August.csv",
    "2022 01 Sep - 2023 31 Jan.csv",
    "2023 01 Feb - 2023 31 Aug.csv",
    "2023 01 Sep - 2024 31 Jan.csv",
    "2024 01 Feb - 2024 30 June.csv",
    "2024 01 July - 2024 31 Dec.csv",
    "2025 01 Jan - 2025 30 Apr.csv",
    "2025 01 May - 2025 31 July.csv",
]

OUT = Path("glasgow_mobility_daily_2018_2025_gapfilled.csv")
```

```

def load_rows(paths: List[str]) -> pd.DataFrame:
    """Read all rows needed (must have startTime, endTime, class,
    count)."""
    frames = []
    for p in paths:
        df = pd.read_csv(p)
        req = {"startTime", "endTime", "class", "count"}
        if not req.issubset(df.columns):
            # skip unexpected schema
            continue

        # Parse datetimes and derive date at midnight
        df["startTime"] = pd.to_datetime(df["startTime"], errors="coerce")
        df["endTime"] = pd.to_datetime(df["endTime"], errors="coerce")
        df = df.dropna(subset=["startTime", "endTime"]).copy()
        df["date"] = df["startTime"].dt.normalize()

        frames.append(df[["date", "class", "count"]])

    return pd.concat(frames, ignore_index=True) if frames else
pd.DataFrame(columns=["date", "class", "count"])

raw = load_rows(FILEES)

# 2) Aggregate to daily counts (sum across all Glasgow sites)
daily = raw.groupby(["date", "class"], as_index=False)["count"].sum()

# 3) Pivot to wide: cycle_count, pedestrian_count
wide = daily.pivot(index="date", columns="class",
values="count").sort_index()

wide = wide.rename(columns={"cycle": "cycle_count", "pedestrian":
"pedestrian_count"})

for col in ("cycle_count", "pedestrian_count"):
    if col not in wide.columns:

```

```

        wide[col] = np.nan

# 4) Reindex to full daily range, gap-fill missing days
if not wide.empty:
    full_idx = pd.date_range(wide.index.min(), wide.index.max(), freq="D")
    wide = wide.reindex(full_idx)
    wide.index.name = "date"

# Track which rows were missing before gap-filling
gap_mask = wide.isna().any(axis=1) if not wide.empty else pd.Series([],
dtype=bool)

# Interpolate numeric data, then ffill/bfill, then round to ints
for col in ("cycle_count", "pedestrian_count"):
    wide[col] = wide[col].interpolate(method="time",
limit_direction="both").ffill().bfill()
wide["total_count"] = wide[["cycle_count", "pedestrian_count"]].sum(axis=1)

for col in ("cycle_count", "pedestrian_count", "total_count"):
    wide[col] = np rint(wide[col]).astype("Int64")

# 5) Save
out_df = wide.reset_index()

out_df["gapfilled"] = gap_mask.reindex(out_df["date"].values,
fill_value=False).values
out_df.to_csv(OUT, index=False)

print(f"Saved {OUT} with {len(out_df):,} rows from
{out_df['date'].min().date()} to {out_df['date'].max().date()}.")
print(out_df.head())
print(out_df.tail())

```

A5) Meteostat (taps-aff label)

```
from meteostat import Hourly, Point
from datetime import datetime
import pandas as pd, pytz, json, numpy as np

# 1. Hourly weather for Glasgow
gla = Point(55.865, -4.257)
start = datetime(2018, 1, 1)
end = datetime(2025, 7, 7)

wx = Hourly(gla, start, end).fetch()

# 2. Time-zone handling
if wx.index.tz is None:
    wx.index = wx.index.tz_localize('UTC')
wx = wx.tz_convert('Europe/London').copy()
wx.reset_index(inplace=True)

# make sure condition code is in a column called 'code'
if "code" not in wx.columns and "coco" in wx.columns:
    wx = wx.rename(columns={"coco": "code"})

# 3. Load Colin's weather_codes.json
with open("weather_codes.json", "r") as f:
    code_rows = json.load(f)

codes_df = (
    pd.DataFrame([row["fields"] | {"code": row["fields"]["code"]} for row
in code_rows])
    .loc[:, ["code", "terrible", "delta"]]
)
```

```
# 4. Merge metadata in; default to not-terrible / delta 0 if no match
```

```
wx = wx.merge(codes_df, on="code", how="left")
```

```
wx["terrible"].fillna(False, inplace=True)
```

```
wx["delta"].fillna(0, inplace=True)
```

```
# 5. Hour-by-hour taps logic
```

```
wx["hour"] = wx["time"].dt.hour
```

```
wx["is_day"] = wx["hour"].between(8, 19)
```

```
temp_col = "feelslike" if "feelslike" in wx.columns else "temp"
```

```
wx["thr"] = 17 + wx["delta"]
```

```
wx["taps_hour"] = (
```

```
    (~wx["terrible"]) &
```

```
    wx["is_day"] &
```

```
    (wx[temp_col] >= wx["thr"]))
```

```
).fillna(False).astype(int)
```

```
# 6. Collapse to daily label
```

```
daily = (
```

```
    wx.groupby(wx["time"].dt.date)["taps_hour"]
```

```
        .max()
```

```
        .reset_index()
```

```
        .rename(columns={"time": "date",
```

```
                        "taps_hour": "taps_aff"}))
```

```
)
```

```
daily["taps_aff"] = daily["taps_aff"].fillna(0).astype("int8")
```

```
daily.to_csv("glasgow_daily_taps_aff_labels.csv", index=False)
```



```
print("Saved → glasgow_daily_taps_aff_labels.csv")  
daily.head()
```

A6) Incorporating Colin's Rule V3 – [Link](#)

```
from meteostat import Hourly, Point
from datetime import datetime, timedelta
import pandas as pd, pytz, json
from astral import LocationInfo
from astral.sun import sun

# -----
# 1. Fetch hourly Meteostat data (same as before)
# -----

gla    = Point(55.865, -4.257)          # Glasgow centre
start  = datetime(2018, 1, 1)
end    = datetime(2025, 7, 7)

wx = Hourly(gla, start, end).fetch()

# Ensure tz-aware, convert to local UK time
if wx.index.tz is None:
    wx.index = wx.index.tz_localize('UTC')
wx = wx.tz_convert('Europe/London').copy()
wx.reset_index(inplace=True)          # 'time' column is local datetime

# -----
# 2. Build a sunrise/sunset table with Astral
# -----

TZ     = pytz.timezone("Europe/London")
city   = LocationInfo("Glasgow", "UK", TZ.zone, 55.865, -4.257)

sun_rows = []
cur_day  = start.date()
```

```

while cur_day <= end.date():
    s = sun(city.observer, date=cur_day, tzinfo=TZ)
    sun_rows.append(
        {"date": cur_day, "sunrise": s["sunrise"], "sunset": s["sunset"]}
    )
    cur_day += timedelta(days=1)

sun_df = pd.DataFrame(sun_rows)

# -----
# 3. Merge sunrise/sunset onto hourly data
# -----
wx["date"] = wx["time"].dt.date
wx = wx.merge(sun_df, on="date", how="left")

# Flag hours strictly between sunrise & sunset
wx["is_day_ast"] = (
    (wx["time"] >= wx["sunrise"]) & (wx["time"] < wx["sunset"])
)

# -----
# 4. Load Colin's weather_codes.json (if using v2/v3 logic)
#   comment this block out if you're sticking with simple temp ≥17 °C
# -----
with open("weather_codes.json", "r") as f:
    code_rows = json.load(f)

codes_df = pd.DataFrame(
    [r["fields"] | {"code": r["fields"]["code"]} for r in code_rows]
)[["code", "terrible", "delta"]]

```

```

# Meteostat's condition code column is 'coco' → rename to 'code'
if "code" not in wx.columns and "coco" in wx.columns:
    wx = wx.rename(columns={"coco": "code"})

wx = wx.merge(codes_df, on="code", how="left")
wx["terrible"].fillna(False, inplace=True)
wx["delta"].fillna(0, inplace=True)

# -----
# 5. Hour-level taps logic using Astral daylight window
# -----

temp_col = "feelslike" if "feelslike" in wx.columns else "temp"
wx["thr"] = 17 + wx["delta"]

wx["taps_hour"] = (
    (~wx["terrible"])
    & wx["is_day_ast"]
    & (wx[temp_col] >= wx["thr"])
).fillna(False).astype(int)

# -----
# 6. Collapse to daily bit & save
# -----

daily = (
    wx.groupby("date")["taps_hour"]
        .max()
        .reset_index()
        .rename(columns={"taps_hour": "taps_aff"})
)

```

```
daily["taps_aff"] = daily["taps_aff"].fillna(0).astype("int8")
daily.to_csv("glasgow_daily_taps_aff_labels.csv", index=False)

print("Saved → glasgow_daily_taps_aff_labels.csv")
daily.head()
```

Appendix B Feature Engineering

B1) Feature Engineering - BBQ

```
# =====  
# Feature-engineering for "bbq" Google Trends  
# =====  
  
import pandas as pd  
import numpy as np  
import holidays  
  
RAW_PATH = "google_trends_bbq_daily_glasgow.csv"          # input file  
OUT_PATH = "google_trends_bbq_daily_glasgow_engineered.csv"  
  
# -----  
# 1. Load & make sure index is a DatetimeIndex  
# -----  
df = pd.read_csv(RAW_PATH, parse_dates=["Date"])  
df.columns = ["date", "bbq"]          # standardise names  
df = df.sort_values("date").set_index("date")  
df.index = pd.to_datetime(df.index)   # force DateTimeIndex  
  
# -----  
# 2. Lag & momentum features  
# -----  
LAGS = [1, 7, 14, 30]  
for L in LAGS:  
    df[f"lag_{L}"] = df["bbq"].shift(L)  
    df[f"diff_{L}"] = df["bbq"] - df[f"lag_{L}"]  
    df[f"pct_{L}"] = np.where(  
        df[f"lag_{L}"] == 0,          # avoid 0 ÷ 0 NaNs
```

```

    0,
    (df["bbq"] / df[f"lag_{L}"] - 1) * 100
)

# -----
# 3. Rolling-window statistics
# -----
ROLLS = [7, 30, 90]
for W in ROLLS:
    df[f"roll{W}_mean"] = df["bbq"].rolling(W, min_periods=1).mean()
    df[f"roll{W}_std"]  = df["bbq"].rolling(W, min_periods=1).std()
    df[f"roll{W}_max"]  = df["bbq"].rolling(W, min_periods=1).max()
    df[f"ema{W}"]       = df["bbq"].ewm(span=W, adjust=False).mean()

# -----
# 4. Calendar & cyclic features
# -----
cal = df.index.to_series()
df["day_of_week"] = cal.dt.dayofweek          # 0 = Mon
df["weekend_flag"] = (df["day_of_week"] >= 5).astype(int)
df["month_num"]    = cal.dt.month
df["week_of_year"] = cal.dt.isocalendar().week.astype(int)
df["day_of_year"]  = cal.dt.dayofyear

# Cyclical encodings
df["annual_sin"]   = np.sin(2 * np.pi * df["day_of_year"] / 365)
df["annual_cos"]   = np.cos(2 * np.pi * df["day_of_year"] / 365)
df["weekly_sin"]   = np.sin(2 * np.pi * df["day_of_week"] / 7)
df["weekly_cos"]   = np.cos(2 * np.pi * df["day_of_week"] / 7)

```

```

# Season & holidays
df["summer_flag"] = df["month_num"].isin([6, 7, 8]).astype(int)
uk_holidays = holidays.UnitedKingdom()
df["uk_holiday_flag"] = df.index.map(lambda d: 1 if d in uk_holidays else 0)

# -----

# 5. Final clean-up
# • Replace any ±inf from divide-by-zero with 0
# • Drop first 30 rows (lag_30 undefined)
# -----

df.replace([np.inf, -np.inf], 0, inplace=True)
df_clean = df.iloc[30:].reset_index() # keep 'date' as a column

# -----

# 6. Save & quick peek
# -----

df_clean.to_csv(OUT_PATH, index=False)
print(f"Saved engineered features → {OUT_PATH} ({len(df_clean):,} rows)")
df_clean.head(20)

```


B2) Feature Engineering - suncream

```
# =====  
# Feature-engineering for "suncream" Google Trends  
# =====  
  
import pandas as pd  
import numpy as np  
import holidays  
  
RAW_PATH = "google_trends_suncream_daily_glasgow.csv"      # adjust if  
needed  
OUT_PATH = "google_trends_suncream_daily_glasgow_engineered.csv"  
  
# -----  
# 1. Load & make sure index is a DatetimeIndex  
# -----  
df = pd.read_csv(RAW_PATH, parse_dates=["Date"])  
df.columns = ["date", "suncream"]  
df = df.sort_values("date").set_index("date")  
  
# force DatetimeIndex (prevents .dayofweek attribute error)  
df.index = pd.to_datetime(df.index)  
  
# -----  
# 2. Lag & momentum features  
# -----  
LAGS = [1, 7, 14, 30]  
for L in LAGS:  
    df[f"lag_{L}"] = df["suncream"].shift(L)  
    df[f"diff_{L}"] = df["suncream"] - df[f"lag_{L}"]  
    df[f"pct_{L}"] = (df["suncream"] / df[f"lag_{L}"] - 1) * 100
```

```

# -----
# 3. Rolling-window statistics
# -----

ROLLS = [7, 30, 90]
for W in ROLLS:
    df[f"roll{W}_mean"] = df["suncream"].rolling(W, min_periods=1).mean()
    df[f"roll{W}_std"]  = df["suncream"].rolling(W, min_periods=1).std()
    df[f"roll{W}_max"]  = df["suncream"].rolling(W, min_periods=1).max()
    df[f"ema{W}"]       = df["suncream"].ewm(span=W, adjust=False).mean()

# -----
# 4. Calendar & cyclic features
# -----

cal_series = df.index.to_series()
df["dayofweek"] = cal_series.dt.dayofweek          # 0 = Monday
df["is_weekend"] = (df["dayofweek"] >= 5).astype(int)
df["month"]      = cal_series.dt.month
df["weekofyear"] = cal_series.dt.isocalendar().week.astype(int)
df["dayofyear"]  = cal_series.dt.dayofyear

# Cyclical encodings
df["annual_sin"] = np.sin(2 * np.pi * df["dayofyear"] / 365)
df["annual_cos"] = np.cos(2 * np.pi * df["dayofyear"] / 365)
df["weekly_sin"] = np.sin(2 * np.pi * df["dayofweek"] / 7)
df["weekly_cos"] = np.cos(2 * np.pi * df["dayofweek"] / 7)

# Summer flag
df["is_summer"] = df["month"].isin([6, 7, 8]).astype(int)

```

```

# UK public-holiday flag
uk_holidays = holidays.UnitedKingdom()
df["is_holiday"] = df.index.map(lambda d: 1 if d in uk_holidays else 0)

# 4b. Sanitise infinities & NaNs -----
df.replace([np.inf, -np.inf], np.nan, inplace=True)
df.fillna(0, inplace=True)          # treat undefined % moves as 0

# 5. Drop only the first 30 warm-up rows (lag_30 not defined)
df_clean = df.iloc[30:].reset_index()

df_clean.to_csv(OUT_PATH, index=False)
print(f"Saved engineered features → {OUT_PATH} ({len(df_clean):,} rows)")
display(df_clean.head(20))

```

B3) Feature Engineering - Mobility

```
# =====  
# Feature-engineering for "Mobility" Cycle and Pedestrian  
# =====  
  
import pandas as pd  
import numpy as np  
from pathlib import Path  
  
# -----  
# paths  
# -----  
IN_PATH = Path("glasgow_mobility_daily_2018_2025.csv")  
OUT_PATH = Path("glasgow_mobility_daily_2018_2025_engineered.csv")  
  
# -----  
# parameters  
# -----  
LAGS = [1, 7, 14, 30]  
ROLL_WINDOWS = [7, 30, 90]  
ROLL_MIN_PERIODS = 1      # rolling(..., min_periods=1)  
ROLL_STD_DDof = 0         # std(ddof=0) population std  
EWM_SPANS = [7, 30, 90]  
EWM_ADJUST = False        # ewm(adjust=False)  
EWM_MIN_PERIODS = 0        # ewm(..., min_periods=0)  
  
# -----  
# If statements  
# -----  
  
def _detect_date_col(df: pd.DataFrame) -> str:
```

```

for c in df.columns:
    if c.lower() in {"date", "ds", "day", "timestamp"}:
        return c
return df.columns[0]

def _pct_change_safe(cur: pd.Series, prev: pd.Series) -> pd.Series:
    return (cur - prev) / prev.replace(0, np.nan)

def _rolling(s: pd.Series, w: int):
    return s.rolling(window=w, min_periods=ROLL_MIN_PERIODS)

def _ewm(s: pd.Series, span: int):
    return s.ewm(span=span, adjust=EWM_ADJUST,
min_periods=EWM_MIN_PERIODS)

def _add_family(df: pd.DataFrame, value_col: str, stem: str) ->
pd.DataFrame:
    """Create lag/diff/pct + rolling mean/std/max + EMA features for one
series."""
    out = df[["date", value_col]].copy()

    # Lags / diffs / pct changes
    for k in LAGS:
        out[f"{stem}_lag_{k}"] = out[value_col].shift(k)
        out[f"{stem}_diff_{k}"] = out[value_col] - out[f"{stem}_lag_{k}"]
        out[f"{stem}_pct_{k}"] = _pct_change_safe(out[value_col],
out[f"{stem}_lag_{k}"])

    # Rolling stats and EMAs
    for w in ROLL_WINDOWS:
        r = _rolling(out[value_col], w)
        out[f"{stem}_roll_{w}_mean"] = r.mean()

```

```

        out[f"{stem}_roll{w}_std"] = r.std(ddof=ROLL_STD_DDof)
        out[f"{stem}_roll{w}_max"] = r.max()
    for span in EWM_SPANS:
        out[f"{stem}_ema{span}"] = _ewm(out[value_col], span).mean()

    return out.drop(columns=[value_col])

# -----
# load & validate
# -----

mob = pd.read_csv(IN_PATH)
date_col = _detect_date_col(mob)
mob["date"] = pd.to_datetime(mob[date_col],
errors="coerce").dt.normalize()

required = {"glasgow_cycle_total", "glasgow_ped_total"}
missing = required - set(mob.columns)

if missing:
    raise ValueError(f"Input is missing required columns:
{sorted(missing)}")

mob = mob[["date", "glasgow_cycle_total", "glasgow_ped_total"]].copy()
mob["glasgow_cycle_total"] = pd.to_numeric(mob["glasgow_cycle_total"],
errors="coerce")
mob["glasgow_ped_total"] = pd.to_numeric(mob["glasgow_ped_total"],
errors="coerce")
mob = mob.sort_values("date").reset_index(drop=True)

# -----
# engineer features
# -----

```

```

cycle_feats = _add_family(mob.rename(columns={"glasgow_cycle_total":
"val"}), "val", "cycle")

ped_feats   = _add_family(mob.rename(columns={"glasgow_ped_total":
"val"}),  "val", "ped")

engineered = mob.merge(cycle_feats, on="date",
how="left").merge(ped_feats, on="date", how="left")

# -----
#output
# -----

engineered.to_csv(OUT_PATH, index=False)

print("Saved:", OUT_PATH)
print("Rows x Cols:", engineered.shape)
print("Feature columns added:", len([c for c in engineered.columns if
c.startswith(('cycle_', 'ped_'))]))

```

Appendix C – Combining Files (Master File)

```
import pandas as pd

# def formula

def read_with_date(path: str) -> pd.DataFrame:
    """Read CSV, normalise a date-like column to 'date' as
    datetime.date."""
    df = pd.read_csv(path)

    candidates = [c for c in df.columns if c.lower() in
["date", "ds", "day", "dt", "timestamp"] or "date" in c.lower()]

    if not candidates:
        candidates = [df.columns[0]]

    date_col = candidates[0]

    df = df.rename(columns={date_col: "date"}).copy()

    df["date"] = pd.to_datetime(df["date"]).dt.date

    return df


def unique_prefixed(df: pd.DataFrame, prefix: str, known_prefixes=None) ->
pd.DataFrame:
    """
    Add `prefix` to non-date columns that don't already start with a known
    prefix.

    Prevents name clashes like 'lag_7' across sources.
    """
    if known_prefixes is None:
        known_prefixes = [prefix]

    out = df.copy()

    renames = {}

    for c in out.columns:
        if c == "date":
            continue
```



```

        if not any(c.lower().startswith(kp.lower()) for kp in
known_prefixes):

            renames[c] = f"{prefix}{c}"

    if renames:

        out = out.rename(columns=renames)

    return out

def uniq(seq):
    seen=set(); out=[]
    for x in seq:
        if x not in seen:
            out.append(x); seen.add(x)
    return out

# -----
# Load files
# -----

taps_df = read_with_date("glasgow_daily_taps_aff_labels.csv")
bbq_df  = read_with_date("google_trends_bbq_daily_glasgow_engineered.csv")
sun_df  =
read_with_date("google_trends_suncream_daily_glasgow_engineered.csv")
mob_df  =
read_with_date("glasgow_mobility_daily_2018_2025_engineered.csv")
eng_df  = read_with_date("gb_demand_daily_from_historic.csv")

# Keep taps label(s) as-is; prefix others only where needed to avoid
clashes

taps_df = taps_df[["date"]] + [c for c in taps_df.columns if c !=
"date"]].copy()

bbq_df  = unique_prefixed(bbq_df,  prefix="bbq_",
known_prefixes=["bbq","bbq_"])

sun_df  = unique_prefixed(sun_df,  prefix="suncream_",
known_prefixes=["suncream","suncream_","sun_"])

```

```

mob_df = unique_prefixed(mob_df, prefix="mob_",
known_prefixes=["mob","mob_","mobility"])

eng_df = unique_prefixed(eng_df, prefix="energy_",
known_prefixes=["energy","energy_","gb_daily"])

# -----
# Merge on date, inner = keep dates
# -----

combined = (
    taps_df
        .merge(bbq_df, on="date", how="inner")
        .merge(sun_df, on="date", how="inner")
        .merge(mob_df, on="date", how="inner")
        .merge(eng_df, on="date", how="inner")
)

# -----
# Order columns: date; taps aff; BBQ; suncream; mobility; energy
# -----

date_col = ["date"]

taps_cols = [c for c in combined.columns if c != "date" and
c.lower().startswith("taps")]

bbq_cols = [c for c in combined.columns if c.lower().startswith("bbq")]

sun_cols = [c for c in combined.columns if
c.lower().startswith(("suncream","sun_"))]

mob_cols = [c for c in combined.columns if
c.lower().startswith(("mob","glasgow_","ped_","cycle_"))]

eng_cols = [c for c in combined.columns if
c.lower().startswith(("energy_","gb_daily"))]

ordered_cols = date_col + uniq(taps_cols) + uniq(bbq_cols) +
uniq(sun_cols) + uniq(mob_cols) + uniq(eng_cols)

```

```
combined =  
combined[ordered_cols].sort_values("date").reset_index(drop=True)  
  
# -----  
# saving to csv  
# -----  
  
combined.to_csv("  
master_daily_tapsaff_bbq_suncream_mobility_demand_engineered.csv",  
index=False)  
  
print("Saved:", "  
master_daily_tapsaff_bbq_suncream_mobility_demand_engineered.csv",  
"shape=", combined.shape)
```

Appendix D Baseline Model

```
# =====  
# Baseline models for taps-aff prediction  
# =====  
  
import pandas as pd  
import numpy as np  
from sklearn.pipeline import make_pipeline  
from sklearn.preprocessing import StandardScaler  
from sklearn.linear_model import LogisticRegression  
from sklearn.tree import DecisionTreeClassifier  
from sklearn.metrics import (  
    accuracy_score, precision_score, recall_score, f1_score, roc_auc_score  
)  
  
# 1 — Load the master feature matrix  
MASTER_PATH =  
r"master_daily_tapsaff_bbq_suncream_mobility_demand_engineered.csv"  
data = pd.read_csv(MASTER_PATH, parse_dates=["date"])  
  
# 1b — Keep only rows that have a label  
data = data.dropna(subset=["taps_aff"]).copy()  
  
# 2 — Train-test split by date  
train_mask = data["date"] < "2024-01-01"  
  
X_train = data.loc[train_mask].drop(columns=["date", "taps_aff"])  
y_train = data.loc[train_mask, "taps_aff"]  
  
X_test = data.loc[~train_mask].drop(columns=["date", "taps_aff"])  
y_test = data.loc[~train_mask, "taps_aff"]
```

```

# Replace any inf/-inf with 0 and fill remaining NaNs in features
for X in [X_train, X_test]:
    X.replace([np.inf, -np.inf], 0, inplace=True)
    X.fillna(0, inplace=True)

# Helper to collect metrics
def eval_model(name, y_true, y_pred, y_prob=None):
    return {
        "Model": name,
        "Accuracy": accuracy_score(y_true, y_pred),
        "Precision": precision_score(y_true, y_pred, zero_division=0),
        "Recall": recall_score(y_true, y_pred, zero_division=0),
        "F1": f1_score(y_true, y_pred, zero_division=0),
        "ROC_AUC": roc_auc_score(y_true, y_prob) if y_prob is not None
    else np.nan
    }

results = []

# 3 — Majority baseline
majority_label = y_train.mode()[0]
y_pred_majority = np.full_like(y_test, majority_label)
results.append(eval_model("Majority", y_test, y_pred_majority))

# 4 — Logistic Regression baseline
log_clf = make_pipeline(
    StandardScaler(),
    LogisticRegression(max_iter=1000, random_state=42)
)
log_clf.fit(X_train, y_train)

```

```

y_pred_log = log_clf.predict(X_test)
y_prob_log = log_clf.predict_proba(X_test)[:, 1]
results.append(eval_model("LogisticRegression", y_test, y_pred_log,
y_prob_log))

# 5 — Decision Tree baseline (depth = 4)
tree_clf = DecisionTreeClassifier(max_depth=4, random_state=42)
tree_clf.fit(X_train, y_train)
y_pred_tree = tree_clf.predict(X_test)
y_prob_tree = tree_clf.predict_proba(X_test)[:, 1]
results.append(eval_model("DecisionTree(depth=4)", y_test, y_pred_tree,
y_prob_tree))

# 6 — Display tidy table
results_df = pd.DataFrame(results)
print(results_df.to_string(index=False, float_format="%.3f"))

```

Appendix E Advanced Models

```
# =====  
# Advanced tree-based models for taps-aff prediction  
# =====  
  
import pandas as pd, numpy as np, warnings, importlib  
  
from sklearn.metrics import roc_auc_score, accuracy_score,  
precision_score, recall_score, f1_score  
  
from sklearn.ensemble import RandomForestClassifier,  
GradientBoostingClassifier, HistGradientBoostingClassifier  
  
warnings.filterwarnings("ignore")  
  
# --- Load & prep ---  
  
MASTER_PATH = r"/Users/andyho/Documents/Dissertation/master  
file/master_daily_tapsaff_bbq_suncream_mobility_demand_engineered.csv"  
  
df = pd.read_csv(MASTER_PATH,  
parse_dates=['date']).dropna(subset=['taps_aff']).sort_values('date')  
  
feature_cols = df.columns.difference(['date', 'taps_aff'])  
  
df[feature_cols] = df[feature_cols].replace([np.inf, -np.inf], 0).fillna(0)  
  
X = df[feature_cols].to_numpy()  
y = df['taps_aff'].to_numpy()  
dates = df['date']  
  
# --- Split once: 2018–2023 for CV/tuning; 2024+ = hold-out ---  
  
train_mask = dates < "2024-01-01"  
  
X_tr, y_tr, d_tr = X[train_mask], y[train_mask], dates[train_mask]  
X_ho, y_ho      = X[~train_mask], y[~train_mask]  
  
# --- Year-aligned expanding folds (2019..2023) ---  
  
def year_folds(dates, y):  
    yrs = pd.to_datetime(dates).dt.year.to_numpy()
```

```

    folds = []
    for Y in range(2019, 2024):    # validation years 2019..2023
        tr = np.where(yrs < Y)[0]
        va = np.where(yrs == Y)[0]
        if va.size and np.unique(y[va]).size > 1:    # skip single-class
folds
            folds.append((tr, va))
    return folds

cv = year_folds(d_tr, y_tr)
print("Folds (validate years):", [int(pd.unique(d_tr.iloc[v].dt.year)[0])
for _, v in cv])

# --- Models with sensible defaults (no tuning here) ---
models = {
    "RF": RandomForestClassifier(n_estimators=400, max_depth=None,
                                min_samples_leaf=1,
                                class_weight='balanced', n_jobs=-1,
                                random_state=42),
    "GB": GradientBoostingClassifier(n_estimators=400,
                                     learning_rate=0.05, max_depth=3, random_state=42),
    "HGB": HistGradientBoostingClassifier(max_depth=3, learning_rate=0.05,
                                           max_iter=400,
                                           l2_regularization=0.0,
                                           random_state=42),
}
if importlib.util.find_spec("xgboost") is not None:
    from xgboost import XGBClassifier
    models["XGB"] = XGBClassifier(
        n_estimators=600, learning_rate=0.05, max_depth=3,
        subsample=0.8, colsample_bytree=0.8, objective='binary:logistic',
        eval_metric='auc', n_jobs=-1, random_state=42
    )

```



```

def cv_auc(model, X, y, cv):
    aucs = []
    for tr_idx, va_idx in cv:
        model.fit(X[tr_idx], y[tr_idx])
        p = model.predict_proba(X[va_idx])[:, 1]
        aucs.append(roc_auc_score(y[va_idx], p))
    return float(np.mean(aucs)), float(np.std(aucs))

# --- Compare by CV AUC (2019..2023), then score hold-out (2024+) with a
# temp 0.5 cut-off ---
rows = []
for name, mdl in models.items():
    mean_auc, std_auc = cv_auc(mdl, X_tr, y_tr, cv)
    mdl.fit(X_tr, y_tr)
    prob = mdl.predict_proba(X_ho)[:, 1]
    pred = (prob >= 0.5).astype(int) # placeholder; replace later with
    your tuned threshold
    rows.append({
        "Model": name,
        "CV AUC (2019-2023)": f"{mean_auc:.3f} ± {std_auc:.3f}",
        "Hold-out AUC (2024+)": f"{roc_auc_score(y_ho, prob):.3f}",
        "Accuracy": f"{accuracy_score(y_ho, pred):.3f}",
        "Precision": f"{precision_score(y_ho, pred,
zero_division=0):.3f}",
        "Recall": f"{recall_score(y_ho, pred, zero_division=0):.3f}",
        "F1": f"{f1_score(y_ho, pred, zero_division=0):.3f}",
    })

import pandas as pd
print(pd.DataFrame(rows).to_string(index=False))

```

Appendix F Hyperparameter tuning & hold-out scoring

F1) Hyper-parameter tuning

```
# =====  
# Hyper-parameter tuning  
# =====  
  
import pandas as pd, numpy as np, warnings  
  
from sklearn.model_selection import GridSearchCV  
  
from sklearn.ensemble import RandomForestClassifier,  
HistGradientBoostingClassifier  
  
  
warnings.filterwarnings("ignore")  
  
  
# --- Load & prep ---  
  
MASTER_PATH = r"/Users/andyho/Documents/Dissertation/master  
file/master_daily_tapsaff_bbq_suncream_mobility_demand_engineered.csv"  
  
df = pd.read_csv(MASTER_PATH,  
parse_dates=["date"]).dropna(subset=["taps_aff"]).sort_values("date")  
  
feature_cols = df.columns.difference(["date", "taps_aff"])  
  
df[feature_cols] = df[feature_cols].replace([np.inf, -np.inf], 0).fillna(0)  
  
  
X = df[feature_cols].to_numpy()  
y = df["taps_aff"].to_numpy()  
dates = df["date"]  
  
  
# --- Restrict tuning strictly to 2018–2023; 2024+ is untouched hold-out -  
--  
  
train_mask = dates < "2024-01-01"  
  
X_tr, y_tr, d_tr = X[train_mask], y[train_mask], dates[train_mask]  
  
  
def year_folds(dates, y):  
    years = pd.to_datetime(dates).dt.year.to_numpy()
```

```

cv=[]
for Y in range(2019, 2024): # validation years 2019..2023
    tr = np.where(years < Y)[0]
    va = np.where(years == Y)[0]
    if va.size and np.unique(y[va]).size > 1: # skip single-class
        folds
            cv.append((tr, va))
    return cv

cv = year_folds(d_tr, y_tr)

# --- Models & compact grids ---
rf = RandomForestClassifier(random_state=42, n_jobs=-1)
hgb = HistGradientBoostingClassifier(random_state=42)

rf_grid = {"n_estimators":[200,400,600], "max_depth":[None,6,10],
"min_samples_leaf":[1,3,5], "class_weight":["balanced"]}
hgb_grid = {"learning_rate":[0.03,0.05,0.1], "max_depth":[3,5,None],
"max_iter":[300,500,700], "l2_regularization":[0.0,0.5]}

def tune(model, grid, X, y, cv, label):
    gs = GridSearchCV(model, grid, scoring="roc_auc", cv=cv, n_jobs=-1,
refit=True, verbose=1)
    gs.fit(X, y)
    print(f"Best {label}:", gs.best_params_, "| mean CV AUC =",
round(gs.best_score_, 3))
    return gs

rf_cv = tune(rf, rf_grid, X_tr, y_tr, cv, "RF")
hgb_cv = tune(hgb, hgb_grid, X_tr, y_tr, cv, "HGB")

best = hgb_cv if hgb_cv.best_score_ >= rf_cv.best_score_ else rf_cv

```

```
best_model = best.best_estimator_  
print("Selected winner:", best_model.__class__.__name__)  
# NOTE: Do calibration & threshold selection on TRAIN ONLY (OOF), then  
score 2024+ in a separate cell.
```

F2) Hold out scoring

```
# =====  
  
# Hold-out (2024+) scoring – run only AFTER calibration & threshold  
selection  
  
# =====  
  
from sklearn.metrics import roc_auc_score, accuracy_score,  
precision_score, recall_score, f1_score  
  
hold_mask = df["date"] >= "2024-01-01"  
X_ho = df.loc[hold_mask, feature_cols].to_numpy()  
y_ho = df.loc[hold_mask, "taps_aff"].to_numpy()  
  
# Example uses a placeholder 0.5 threshold – replace with your tuned  
threshold  
  
ho_prob = best_model.predict_proba(X_ho)[:, 1]  
tau = 0.5 # replace with your train-derived threshold  
ho_pred = (ho_prob >= tau).astype(int)  
  
print("\n Hold-out (2024+)")  
print("AUC      :", round(roc_auc_score(y_ho, ho_prob), 3))  
print("Accuracy :", round(accuracy_score(y_ho, ho_pred), 3))  
print("Precision:", round(precision_score(y_ho, ho_pred, zero_division=0),  
3))  
print("Recall   :", round(recall_score(y_ho, ho_pred, zero_division=0),  
3))  
print("F1       :", round(f1_score(y_ho, ho_pred, zero_division=0), 3))
```

Risk Check for 2024+

Ensure no 2024+ touched the tuning/threshold phases

```
assert df.loc[df["date"] < "2024-01-01", "date"].max() <
pd.Timestamp("2024-01-01")
```

If you used explicit folds:

```
for tr_idx, va_idx in cv:
```

```
    assert df.loc[df["date"] < "2024-01-01"].iloc[va_idx]["date"].max() <=
pd.Timestamp("2023-12-31")
```

F3) HGB Post Calibration

```
import warnings, numpy as np, pandas as pd

from pathlib import Path

from sklearn.base import clone

from sklearn.metrics import accuracy_score, precision_score, recall_score,
f1_score, confusion_matrix

from sklearn.metrics import roc_auc_score, precision_recall_curve

from sklearn.ensemble import RandomForestClassifier,
HistGradientBoostingClassifier

from sklearn.calibration import CalibratedClassifierCV

warnings.filterwarnings("ignore")

MASTER_CSV =
"master_daily_tapsaff_bbq_suncream_mobility_demand_engineered.csv"

OUT = Path("outputs"); OUT.mkdir(parents=True, exist_ok=True)

BETA = 0.5 # choose F_beta to match costs

# --- Load & split

df = pd.read_csv(MASTER_CSV,
parse_dates=["date"]).dropna(subset=["taps_aff"]).sort_values("date")

feature_cols = [c for c in df.columns if c not in ("date", "taps_aff")]

X = df[feature_cols].replace([np.inf, -np.inf], 0).fillna(0).to_numpy()
y = df["taps_aff"].astype(int).to_numpy()
dates = df["date"]

train_mask = dates < pd.Timestamp("2024-01-01")
X_tr, y_tr, d_tr = X[train_mask], y[train_mask], dates[train_mask]
X_ho, y_ho      = X[~train_mask], y[~train_mask]

# --- Expanding, year-aligned folds (validate on 2019..2023)
def year_folds(dates, y):
```

```

years = pd.to_datetime(dates).dt.year.to_numpy()
folds = []
for Y in range(2019, 2024):
    tr_idx = np.where(years < Y)[0]
    va_idx = np.where(years == Y)[0]
    if va_idx.size and np.unique(y[va_idx]).size > 1:
        folds.append((tr_idx, va_idx))
return folds

cv = year_folds(d_tr, y_tr)

# --- Candidates with sensible defaults
rf = RandomForestClassifier(n_estimators=400, max_depth=None,
min_samples_leaf=1,
                           class_weight="balanced", n_jobs=1,
random_state=42)
hgb = HistGradientBoostingClassifier(learning_rate=0.05, max_depth=3,
max_iter=400,
                                   l2_regularization=0.0,
random_state=42)

def mean_cv_auc(model, X, y, folds):
    aucs = []
    for tr_idx, va_idx in folds:
        m = clone(model).fit(X[tr_idx], y[tr_idx])
        p = m.predict_proba(X[va_idx])[:,1]
        aucs.append(roc_auc_score(y[va_idx], p))
    return float(np.mean(aucs))

best_model = hgb if mean_cv_auc(hgb, X_tr, y_tr, cv) >= mean_cv_auc(rf,
X_tr, y_tr, cv) else rf

```



```

# --- Choose threshold  $\tau$  on TRAIN via OOF  $F_{\beta}$ 
oof = np.full_like(y_tr, np.nan, dtype=float)
for tr_idx, va_idx in cv:
    m = clone(best_model).fit(X_tr[tr_idx], y_tr[tr_idx])
    oof[va_idx] = m.predict_proba(X_tr[va_idx])[:,1]

prec, rec, thr = precision_recall_curve(y_tr[np.isfinite(oof)],
oof[np.isfinite(oof)])

fb = (1 + BETA**2) * (prec[:-1] * rec[:-1]) / (BETA**2 * prec[:-1] +
rec[:-1] + 1e-12)

tau = float(thr[np.nanargmax(fb)])

# --- Fit on train-core and calibrate on train-tail (no peeking)
cal_mask = (d_tr >= (d_tr.max() - pd.Timedelta(days=180)))
if np.unique(y_tr[cal_mask]).size < 2:
    cal_mask = (d_tr >= (d_tr.max() - pd.Timedelta(days=365)))
if np.unique(y_tr[cal_mask]).size < 2:
    cal_mask = (d_tr.dt.year == d_tr.max().year)

base = clone(best_model).fit(X_tr[~cal_mask], y_tr[~cal_mask])
y_prob_pre = base.predict_proba(X_ho)[:,1]

y_prob_post = None
if np.unique(y_tr[cal_mask]).size == 2:
    calib = CalibratedClassifierCV(base, method="isotonic",
cv="prefit").fit(X_tr[cal_mask], y_tr[cal_mask])
    y_prob_post = calib.predict_proba(X_ho)[:,1]

# --- Build Table 4.4 (one tidy table)
def table_44(y_true, y_prob, tau, name):
    pred = (y_prob >= tau).astype(int)

```

```

tn, fp, fn, tp = confusion_matrix(y_true, pred).ravel()
row = {
    "Model": name, "Threshold  $\tau$ ": round(tau,3),
    "Accuracy": round(accuracy_score(y_true, pred), 3),
    "Precision": round(precision_score(y_true, pred, zero_division=0),
3),
    "Recall": round(recall_score(y_true, pred, zero_division=0), 3),
    "F1": round(f1_score(y_true, pred, zero_division=0), 3),
    "TP": int(tp), "FP": int(fp), "TN": int(tn), "FN": int(fn)
}
return row

name = f"{'HGB' if best_model is hgb else 'RF'} (post-cal)" if y_prob_post
is not None else f"{'HGB' if best_model is hgb else 'RF'} (pre-cal)"
final_prob = y_prob_post if y_prob_post is not None else y_prob_pre
t44 = pd.DataFrame([table_44(y_ho, final_prob, tau, name)])

# Save & preview
path = OUT / "Table_4_4_Confusion_and_Metrics.csv"
t44.to_csv(path, index=False)
display(t44)
print("Saved:", path.resolve())

```

Appendix G Leakage safe & threshold-tuned version

```
import pandas as pd, numpy as np, warnings

from sklearn.model_selection import TimeSeriesSplit, GridSearchCV

from sklearn.metrics import roc_auc_score, precision_recall_curve,
f1_score, accuracy_score, precision_score, recall_score

from sklearn.ensemble import RandomForestClassifier,
HistGradientBoostingClassifier


warnings.filterwarnings("ignore")


MASTER_PATH = r"/Users/andyho/Documents/Dissertation/master
file/master_daily_tapsaff_bbq_suncream_mobility_demand_engineered.csv"

df = pd.read_csv(MASTER_PATH,
parse_dates=["date"]).dropna(subset=["taps_aff"]).sort_values("date")


# Features

feature_cols = df.columns.difference(["date", "taps_aff"])

df[feature_cols] = df[feature_cols].replace([np.inf, -np.inf],
0).fillna(0)


# Train / Hold-out split

train_mask = df["date"] < "2024-01-01"

X_tr, y_tr = df.loc[train_mask, feature_cols].values, df.loc[train_mask,
"taps_aff"].values

X_ho, y_ho = df.loc[~train_mask, feature_cols].values, df.loc[~train_mask,
"taps_aff"].values


tscv = TimeSeriesSplit(n_splits=5)


# Grids

rf_grid = {"n_estimators": [200, 400, 600], "max_depth": [None, 6, 10],
"min_samples_leaf": [1, 3, 5], "class_weight": ["balanced"]}
```

```

hgb_grid = {"learning_rate":[0.03,0.05,0.1], "max_depth":[3,5,None],
"max_iter":[300,500,700], "l2_regularization":[0.0,0.5]}

rf = RandomForestClassifier(random_state=42, n_jobs=-1)
hgb = HistGradientBoostingClassifier(random_state=42)

rf_cv = GridSearchCV(rf, rf_grid, scoring="roc_auc", cv=tscv, n_jobs=-1, verbose=0).fit(X_tr, y_tr)
hgb_cv = GridSearchCV(hgb, hgb_grid, scoring="roc_auc", cv=tscv, n_jobs=-1, verbose=0).fit(X_tr, y_tr)

best = hgb_cv if hgb_cv.best_score_ >= rf_cv.best_score_ else rf_cv
best_model = best.best_estimator_
print(f"Best model: {best_model.__class__.__name__} | CV AUC (train) = {best.best_score_:.3f}")
best_model.fit(X_tr, y_tr)

# --- Threshold tuning on train via CV predictions ---
# Get out-of-fold probabilities from TimeSeriesSplit for the chosen model
oof_prob = np.zeros_like(y_tr, dtype=float)
for tr_idx, val_idx in tscv.split(X_tr):
    m = best_model.__class__(**best_model.get_params())
    m.fit(X_tr[tr_idx], y_tr[tr_idx])
    oof_prob[val_idx] = m.predict_proba(X_tr[val_idx])[:,1]

prec, rec, thresh = precision_recall_curve(y_tr, oof_prob)
beta = 0.5 # favour precision if FN cheaper set >1 to favour recall
fb = (1+beta**2) * (prec*rec) / (beta**2 * prec + rec + 1e-12)
thr_star = thresh[np.nanargmax(fb)]
print(f"Chosen threshold (F{beta} on train CV): {thr_star:.3f}")

# --- Hold-out evaluation ---

```

```

ho_prob = best_model.predict_proba(X_ho)[: ,1]
ho_pred = (ho_prob >= thr_star).astype(int)

print("\nHold-out (2024+)")
print("AUC      :", round(roc_auc_score(y_ho, ho_prob), 3))
print("Accuracy :", round(accuracy_score(y_ho, ho_pred), 3))
print("Precision :", round(precision_score(y_ho, ho_pred,
zero_division=0), 3))
print("Recall    :", round(recall_score(y_ho, ho_pred, zero_division=0),
3))
print("F1        :", round(f1_score(y_ho, ho_pred, zero_division=0), 3))

```

Appendix H Figures

H1) Figure 2 - ROC Curve & Precision-Recall

```
# =====  
# ROC & PR  
# =====  
  
import warnings, numpy as np, pandas as pd  
from pathlib import Path  
import matplotlib.pyplot as plt  
from sklearn.base import clone  
  
from sklearn.metrics import roc_curve, auc, precision_recall_curve,  
average_precision_score  
  
from sklearn.metrics import roc_auc_score  
  
from sklearn.ensemble import RandomForestClassifier,  
HistGradientBoostingClassifier  
  
warnings.filterwarnings("ignore")  
  
MASTER_CSV =  
"master_daily_tapsaff_bbq_suncream_mobility_demand_engineered.csv" #  
adjust if needed  
  
OUT = Path("outputs"); OUT.mkdir(parents=True, exist_ok=True)  
  
# ---- Load & split  
  
df = pd.read_csv(MASTER_CSV,  
parse_dates=["date"]).dropna(subset=["taps_aff"]).sort_values("date")  
feature_cols = [c for c in df.columns if c not in ("date", "taps_aff")]  
X_all = df[feature_cols].replace([np.inf, -np.inf],  
0).fillna(0).to_numpy()  
y_all = df["taps_aff"].astype(int).to_numpy()  
dates = df["date"]  
  
train_mask = dates < pd.Timestamp("2024-01-01")
```

```

hold_mask = ~train_mask
X_tr, y_tr, d_tr = X_all[train_mask], y_all[train_mask], dates[train_mask]
X_ho, y_ho, d_ho = X_all[hold_mask], y_all[hold_mask], dates[hold_mask]

# ---- Year-aligned expanding folds (validate on 2019..2023)
def year_folds(dates, y):
    years = pd.to_datetime(dates).dt.year.to_numpy()
    folds = []
    for Y in range(2019, 2024): # 2019..2023
        tr_idx = np.where(years < Y)[0]
        va_idx = np.where(years == Y)[0]
        if va_idx.size and np.unique(y[va_idx]).size > 1:
            folds.append((tr_idx, va_idx))
    return folds

cv = year_folds(d_tr, y_tr)

# ---- Candidate models (sensible defaults, fast)
rf = RandomForestClassifier(n_estimators=400, max_depth=None,
min_samples_leaf=1,
                                class_weight="balanced", n_jobs=1,
random_state=42)

hgb = HistGradientBoostingClassifier(learning_rate=0.05, max_depth=3,
max_iter=400,
                                l2_regularization=0.0,
random_state=42)

def mean_cv_auc(model, X, y, cv):
    aucs = []
    for tr_idx, va_idx in cv:
        m = clone(model).fit(X[tr_idx], y[tr_idx])
        p = m.predict_proba(X[va_idx])[:, 1]

```

```

        aucs.append(roc_auc_score(y[va_idx], p))
    return float(np.mean(aucs))

rf_cv, hgb_cv = mean_cv_auc(rf, X_tr, y_tr, cv), mean_cv_auc(hgb, X_tr,
y_tr, cv)
best, best_name = (hgb, "HGB") if hgb_cv >= rf_cv else (rf, "RF")

# ---- Refit best on all TRAIN and predict 2024+
best.fit(X_tr, y_tr)
y_test = y_ho
y_prob_sel = best.predict_proba(X_ho)[:, 1]

# ROC
from sklearn.metrics import roc_curve, auc
fpr, tpr, _ = roc_curve(y_test, y_prob_sel)
roc_auc = auc(fpr, tpr)
plt.figure()
plt.plot(fpr, tpr)
plt.plot([0,1],[0,1], linestyle="--")
plt.xlabel("False positive rate"); plt.ylabel("True positive rate")
plt.title("ROC (2024+ hold-out)")
plt.tight_layout()
plt.savefig(OUT / "Figure_4_2_ROC.png"); plt.close()

# Precision-Recall
from sklearn.metrics import precision_recall_curve,
average_precision_score
prec, rec, _ = precision_recall_curve(y_test, y_prob_sel)
pr_auc = average_precision_score(y_test, y_prob_sel)
plt.figure()
plt.plot(rec, prec)

```



```
plt.xlabel("Recall"); plt.ylabel("Precision")
plt.title("Precision-Recall (2024+ hold-out)")
plt.tight_layout()
plt.savefig(OUT / "Figure_4_2_PR.png"); plt.close()

print(f"Selected: {best_name} | Train CV AUC:",
      round(max(rf_cv, hgb_cv), 3))
print("ROC-AUC (2024+):", round(roc_auc, 3),
      "| PR-AUC (2024+):", round(pr_auc, 3))
print("Saved:",
      OUT / "ROC.png",
      OUT / "PR.png")
```

H2) Figure 3 - Reliability

Reliability

```
import warnings, numpy as np, pandas as pd, matplotlib.pyplot as plt

from pathlib import Path

from sklearn.base import clone

from sklearn.ensemble import RandomForestClassifier,
HistGradientBoostingClassifier

from sklearn.metrics import brier_score_loss

from sklearn.calibration import CalibratedClassifierCV, calibration_curve

warnings.filterwarnings("ignore")

MASTER_CSV =
"master_daily_tapsaff_bbq_suncream_mobility_demand_engineered.csv" #
adjust if needed

OUT = Path("outputs"); OUT.mkdir(parents=True, exist_ok=True)

# --- Load & split

df = pd.read_csv(MASTER_CSV,
parse_dates=["date"]).dropna(subset=["taps_aff"]).sort_values("date")

feature_cols = [c for c in df.columns if c not in ("date", "taps_aff")]

X_all = df[feature_cols].replace([np.inf, -np.inf],
0).fillna(0).to_numpy()

y_all = df["taps_aff"].astype(int).to_numpy()

dates = df["date"]

train_mask = dates < pd.Timestamp("2024-01-01")

hold_mask = ~train_mask

X_tr, y_tr, d_tr = X_all[train_mask], y_all[train_mask], dates[train_mask]
X_ho, y_ho = X_all[hold_mask], y_all[hold_mask]

best_model = HistGradientBoostingClassifier(
    learning_rate=0.05, max_depth=3, max_iter=400, random_state=42
```

```

)

# -Fit base on TRAIN (excluding a tail slice reserved for calibration)
cal_mask = (d_tr >= (d_tr.max() - pd.Timedelta(days=180)))
if np.unique(y_tr[cal_mask]).size < 2:
    cal_mask = (d_tr >= (d_tr.max() - pd.Timedelta(days=365)))
if np.unique(y_tr[cal_mask]).size < 2:
    cal_mask = (d_tr.dt.year == d_tr.max().year)

X_core, y_core = X_tr[~cal_mask], y_tr[~cal_mask] # fit area
X_cal, y_cal = X_tr[cal_mask], y_tr[cal_mask] # calibration area

base = clone(best_model).fit(X_core, y_core)
y_prob_pre = base.predict_proba(X_ho)[: , 1]

# --- Isotonic calibration on TRAIN only (if both classes present)
y_prob_post = None
if np.unique(y_cal).size == 2:
    calib = CalibratedClassifierCV(base, method="isotonic", cv="prefit")
    calib.fit(X_cal, y_cal)
    y_prob_post = calib.predict_proba(X_ho)[: , 1]

# --- Plot reliability curves on 2024+ hold-out
plt.figure()
plt.plot([0,1], [0,1], linestyle="--") # perfect calibration

pt_pre, pp_pre = calibration_curve(y_ho, y_prob_pre, n_bins=10,
strategy="uniform")
plt.plot(pp_pre, pt_pre, label="Pre-calibration")

```

```

title = f"Reliability (2024+ hold-out) | Brier pre =
{brier_score_loss(y_ho, np.clip(y_prob_pre,1e-6,1-1e-6)):.2f}"

if y_prob_post is not None:
    pt_post, pp_post = calibration_curve(y_ho, y_prob_post, n_bins=10,
strategy="uniform")

    plt.plot(pp_post, pt_post, label="Post-calibration")

    title += f"    · post = {brier_score_loss(y_ho, np.clip(y_prob_post,1e-
6,1-1e-6)):.2f}"

plt.xlabel("Predicted probability")
plt.ylabel("Observed frequency")
plt.title(title)
plt.legend()
plt.tight_layout()
plt.savefig(OUT / "Reliability.png"); plt.close()

print("Saved", OUT / "Reliability.png")

```

H3) Figure 4 - Metrics Deltas by stage, Table 5 - Contributions of trends, demands & mobility

```
# =====

# Bar chart of metric deltas by stage (AUC/Brier)

# S1: Trends + calendar
# S2: S1 + Demand
# S3: S2 + Mobility
# =====

import numpy as np, pandas as pd, matplotlib.pyplot as plt
from pathlib import Path

from sklearn.metrics import roc_auc_score, brier_score_loss,
average_precision_score

from sklearn.ensemble import HistGradientBoostingClassifier

# ----- Paths -----
MASTER_CSV =
"master_daily_tapsaff_bbq_suncream_mobility_demand_engineered.csv"
OUTDIR = Path("outputs"); OUTDIR.mkdir(exist_ok=True, parents=True)

# ----- Load data -----
df = pd.read_csv(MASTER_CSV,
parse_dates=["date"]).dropna(subset=["taps_aff"]).sort_values("date")
y = df["taps_aff"].astype(int).to_numpy()
dates = df["date"]

# Clean features
Xfull = df.drop(columns=["date", "taps_aff"]).replace([np.inf, -np.inf],
0).fillna(0)

# Trends (intent)
TRENDS_PATTERNS = ["bbq", "suncream", "trend"]
```

```

# Demand (seasonal context)
DEMAND_PATTERNS = ["demand"]

# Mobility (revealed behaviour)
MOBILITY_PATTERNS = ["mobility", "cycle", "cycl", "pedestr", "footfall",
"bike"]

# Calendar / seasonality encodings
CAL_PATTERNS = [
    "annual_", "weekly_", "sin", "cos",
    "weekend_flag", "uk_holiday", "holiday",
    "month_num", "week_of_year", "day_of_year",
    "summer_flag"
]

def cols_matching(patterns, columns):
    pats = [p.lower() for p in patterns]
    keep = []
    for c in columns:
        lc = c.lower()
        if any(p in lc for p in pats):
            keep.append(c)
    return keep

all_cols = list(Xfull.columns)
tr_cols = cols_matching(TRENDS_PATTERNS, all_cols)
dm_cols = cols_matching(DEMAND_PATTERNS, all_cols)
mb_cols = cols_matching(MOBILITY_PATTERNS, all_cols)
cal_cols = cols_matching(CAL_PATTERNS, all_cols)

# de-duplicate while preserving order
def uniq(seq): return list(dict.fromkeys(seq))

```

```

S1_cols = uniq(cal_cols + tr_cols)          # Trends + calendar
S2_cols = uniq(S1_cols + dm_cols)           # + Demand
S3_cols = uniq(S2_cols + mb_cols)           # + Mobility

for name, cols in [("S1 (Trends+Cal)", S1_cols), ("S2 (+Demand)",
S2_cols), ("S3 (+Mobility)", S3_cols)]:
    if len(cols) == 0:
        raise ValueError(f"No columns selected for {name}. Check your
*_PATTERNS lists.")
    print(f"{name}: {len(cols)} features")

# ----- Train / hold-out split -----
train_mask = dates < pd.Timestamp("2024-01-01")

def split(cols):
    X = Xfull[cols].to_numpy()
    return X[train_mask], y[train_mask], X[~train_mask], y[~train_mask]

X1_tr, y_tr, X1_ho, y_ho = split(S1_cols)
X2_tr, _,   X2_ho, _   = split(S2_cols)
X3_tr, _,   X3_ho, _   = split(S3_cols)

# ----- Model (kept constant across stages; pre-calibration) -----
--
SEED = 42
def fit_and_score(Xtr, ytr, Xho, yho):
    m = HistGradientBoostingClassifier(
        learning_rate=0.05, max_depth=3, max_iter=400,
        l2_regularization=0.0, random_state=SEED
    )
    m.fit(Xtr, ytr)

```

```

prob = m.predict_proba(Xho)[: , 1]
auc   = roc_auc_score(yho, prob)
brier = brier_score_loss(yho, prob)
prauc = average_precision_score(yho, prob)
return auc, brier, prauc

auc1, bri1, pr1 = fit_and_score(X1_tr, y_tr, X1_ho, y_ho) # S1
auc2, bri2, pr2 = fit_and_score(X2_tr, y_tr, X2_ho, y_ho) # S2
auc3, bri3, pr3 = fit_and_score(X3_tr, y_tr, X3_ho, y_ho) # S3

# ----- Deltas -----
# Positive AUC = improvement; Negative Brier = improvement.
d_auc_s2_s1 = auc2 - auc1
d_auc_s3_s2 = auc3 - auc2
d_bri_s2_s1 = bri2 - bri1
d_bri_s3_s2 = bri3 - bri2

# Save a small table
metrics = pd.DataFrame({
    "Stage": ["S1 (Trends)", "S2 (+Demand)", "S3 (+Mobility)"],
    "ROC_AUC_2024+": [auc1, auc2, auc3],
    "PR_AUC_2024+": [pr1, pr2, pr3],
    "Brier_2024+": [bri1, bri2, bri3],
    "n_features": [len(S1_cols), len(S2_cols), len(S3_cols)]
}).round({"ROC_AUC_2024+":3, "PR_AUC_2024+":3, "Brier_2024+":2})
metrics_path = OUTDIR / "Ablation_Stages.csv"
metrics.to_csv(metrics_path, index=False)
print("Saved metrics table →", metrics_path.resolve())

deltas = pd.DataFrame({

```



```

        "Delta": ["S2 - S1", "S3 - S2"],
        "ΔROC_AUC": [d_auc_s2_s1, d_auc_s3_s2],
        "ΔBrier": [d_bri_s2_s1, d_bri_s3_s2]
    }).round(3)
print("\nMetric deltas (2024+):")
print(deltas.to_string(index=False))

fig, ax = plt.subplots(figsize=(7.2, 4.6))
x = np.arange(len(deltas))
width = 0.36

b1 = ax.bar(x - width/2, deltas["ΔROC_AUC"], width, label="ΔAUC")
b2 = ax.bar(x + width/2, deltas["ΔBrier"], width, label="ΔBrier")

ax.axhline(0, linewidth=1, color="black")
ax.set_xticks(x, deltas["Delta"])
ax.set_ylabel("Delta on 2024+ (AUC ↑ better, Brier ↓ better)")
ax.set_title("Metric deltas by stage")
ax.legend(loc="best")

# annotate values
for bars in (b1, b2):
    for bar in bars:
        h = bar.get_height()
        ax.annotate(f"{h:.3f}", xy=(bar.get_x()+bar.get_width()/2, h),
                    xytext=(0, 3), textcoords="offset points",
                    ha="center", va="bottom", fontsize=8)

plt.tight_layout()
fig_path = OUTDIR / "metric_deltas.png"

```

```
plt.savefig(fig_path, dpi=200)  
print("Saved figure →", fig_path.resolve())
```

H4) Figure 5 - ROC-AUC by year, Figure 6 - Brier by year, Figure 7 - Reliability by year & Table 6 - slicing metrics

```
# =====

# ROCE-AUC by year (figure 5), Brier by year (figure 6), Reliability by
# year (figure 7) & slicing metrics table (table 6)

# =====

import numpy as np, pandas as pd, matplotlib.pyplot as plt
from pathlib import Path
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.calibration import CalibratedClassifierCV
from sklearn.metrics import roc_auc_score, average_precision_score,
brier_score_loss

# ----- Paths -----
MASTER_CSV =
"master_daily_tapsaff_bbq_suncream_mobility_demand_engineered.csv" # <-
change if needed
OUT = Path("outputs"); OUT.mkdir(parents=True, exist_ok=True)

# ----- Load & prep -----
df = (pd.read_csv(MASTER_CSV, parse_dates=["date"]))
    .dropna(subset=["taps_aff"])
    .sort_values("date")
feature_cols = [c for c in df.columns if c not in ("date", "taps_aff")]
X = (df[feature_cols]
    .replace([np.inf, -np.inf], 0)
    .fillna(0)
    .to_numpy())
y = df["taps_aff"].astype(int).to_numpy()
dates = df["date"]
```

```

# Base model (kept constant)
BASE = HistGradientBoostingClassifier(
    learning_rate=0.05, max_depth=3, max_iter=400,
    l2_regularization=0.0, random_state=42
)

def fit_with_tail_calibration(X_tr, y_tr, d_tr, base_model,
    tail_days=180):
    """Fit on train-core, calibrate on train-tail (isotonic) if both
    classes present."""
    tail_start = pd.Series(d_tr).max() - pd.Timedelta(days=tail_days)
    tail_mask = (d_tr >= tail_start)
    # widen tail if needed
    if len(np.unique(y_tr[tail_mask])) < 2:
        tail_start = pd.Series(d_tr).max() - pd.Timedelta(days=365)
        tail_mask = (d_tr >= tail_start)
    # fit on core
    core_mask = ~tail_mask
    model = HistGradientBoostingClassifier(**base_model.get_params())
    model.fit(X_tr[core_mask], y_tr[core_mask])
    # calibrate if possible
    if len(np.unique(y_tr[tail_mask])) == 2 and tail_mask.sum() >= 30:
        cal = CalibratedClassifierCV(model, method="isotonic",
cv="prefit")
        cal.fit(X_tr[tail_mask], y_tr[tail_mask])
        return cal, True
    return model, False

def reliability_points(y_true, y_prob, n_bins=8):
    """Bin by predicted prob and compute mean predicted vs observed
    frequency."""
    bins = np.linspace(0, 1, n_bins+1)

```

```

idx = np.digitize(y_prob, bins) - 1
xs, ys = [], []
for b in range(n_bins):
    m = (idx == b)
    if m.sum() == 0:
        continue
    xs.append(float(y_prob[m].mean()))
    ys.append(float(y_true[m].mean()))
return np.array(xs), np.array(ys)

# =====
# Year-by-year backtests (AUC & Brier lines)
# =====
years = sorted(df["date"].dt.year.unique())
auc_by_year, brier_by_year, prauc_by_year, posrate_by_year = {}, {}, {}, {}

for Y in years[1:]: # need at least one prior year
    tr_mask = (df["date"].dt.year < Y)
    te_mask = (df["date"].dt.year == Y)
    if te_mask.sum() < 15:
        continue
    X_tr, y_tr, d_tr = X[tr_mask], y[tr_mask], dates[tr_mask]
    X_te, y_te = X[te_mask], y[te_mask]
    posrate_by_year[Y] = float(np.mean(y_te))
    # If test is single-class, only Brier and pos rate are meaningful
    if len(np.unique(y_tr)) < 2 or len(np.unique(y_te)) < 2:
        proba_const = np.full_like(y_te, fill_value=np.mean(y_tr),
dtype=float)
        auc_by_year[Y] = np.nan
        prauc_by_year[Y] = np.nan

```

```

        brier_by_year[Y] = float(brier_score_loss(y_te, proba_const))
        continue

    model, used_cal = fit_with_tail_calibration(X_tr, y_tr, d_tr, BASE,
tail_days=180)

    prob = model.predict_proba(X_te)[: ,1]
    auc_by_year[Y] = float(roc_auc_score(y_te, prob))
    prauc_by_year[Y] = float(average_precision_score(y_te, prob))
    brier_by_year[Y] = float(brier_score_loss(y_te, prob))

# Plots
ys = sorted(auc_by_year.keys())

plt.figure(figsize=(7.2,4.4))
plt.plot(ys, [auc_by_year[y] for y in ys], marker="o")
plt.xlabel("Year"); plt.ylabel("ROC-AUC"); plt.ylim(0,1);
plt.grid(alpha=0.3)
plt.title("Year-by-year ROC-AUC")
plt.tight_layout()
plt.savefig(OUT/"AUC_by_year.png", dpi=200)
plt.close()

plt.figure(figsize=(7.2,4.4))
plt.plot(ys, [brier_by_year[y] for y in ys], marker="o")
plt.xlabel("Year"); plt.ylabel("Brier (lower is better)");
plt.grid(alpha=0.3)
plt.title("Year-by-year Brier")
plt.tight_layout()
plt.savefig(OUT/"Brier_by_year.png", dpi=200)
plt.close()

# =====

```

```

# Calibration curves - Reliability by year

# (a) by season on 2024+ (b) by year (each Y tested with model
# trained  $\leq Y-1$ )

# =====

# (a) Season reliability on 2024+
train_mask = (df["date"] < "2024-01-01")
hold_mask = (df["date"] >= "2024-01-01")
X_tr, y_tr, d_tr = X[train_mask], y[train_mask], dates[train_mask]
X_ho, y_ho, d_ho = X[hold_mask], y[hold_mask], dates[hold_mask]

model_2024, _ = fit_with_tail_calibration(X_tr, y_tr, d_tr, BASE,
tail_days=180)
prob_ho = model_2024.predict_proba(X_ho)[: ,1]

def season_of_month(m):
    if m in (12,1,2): return "DJF"
    if m in (3,4,5): return "MAM"
    if m in (6,7,8): return "JJA"
    return "SON"
seasons = pd.Series(d_ho.dt.month).map(season_of_month).values

fig, axes = plt.subplots(2,2, figsize=(8,6), sharex=True, sharey=True)
for ax, s in zip(axes.ravel(), ["DJF", "MAM", "JJA", "SON"]):
    m = (seasons == s)
    ax.plot([0,1],[0,1], linestyle="--")
    if m.sum():
        xp, yp = reliability_points(y_ho[m], prob_ho[m], n_bins=8)
        if len(xp):
            ax.plot(xp, yp, marker="o", label=f"{s} (n={int(m.sum())})")
            ax.legend(loc="lower right", fontsize=8)
    ax.set_title(s)

```

```

for ax in axes[-1]:
    ax.set_xlabel("Predicted probability")
for ax in axes[:,0]:
    ax.set_ylabel("Observed frequency")
fig.suptitle("Reliability by season", y=0.98)
fig.tight_layout()
fig.savefig(OUT/"Reliability_by_season.png", dpi=200)
plt.close(fig)

# (b) Per-year reliability small multiples
years_eval = [y for y in years[1:] if (df["date"].dt.year==y).sum() >= 15]
n = len(years_eval)
rows = int(np.ceil(n/3)); cols = 3
fig, axes = plt.subplots(rows, cols, figsize=(3.2*cols, 2.8*rows),
sharex=True, sharey=True)
axes = np.array(axes).reshape(rows, cols)

for i, Y in enumerate(years_eval):
    ax = axes.ravel()[i]
    tr_mask = (df["date"].dt.year < Y)
    te_mask = (df["date"].dt.year == Y)
    X_tr, y_tr, d_tr = X[tr_mask], y[tr_mask], dates[tr_mask]
    X_te, y_te, d_te = X[te_mask], y[te_mask], dates[te_mask]
    ax.plot([0,1],[0,1], linestyle="--")
    if len(np.unique(y_tr))==2 and len(np.unique(y_te))==2:
        modelY, _ = fit_with_tail_calibration(X_tr, y_tr, d_tr, BASE,
tail_days=180)
        p = modelY.predict_proba(X_te)[: ,1]
        xp, yp = reliability_points(y_te, p, n_bins=8)
        if len(xp):
            ax.plot(xp, yp, marker="o")

```



```

        ax.set_title(str(Y))
# tidy axes
for j in range(rows):
    axes[j,0].set_ylabel("Observed")
for j in range(cols):
    axes[-1,j].set_xlabel("Predicted")
fig.suptitle("Reliability by year", y=0.98)
fig.tight_layout()
fig.savefig(OUT/"Reliability_by_year.png", dpi=200)
plt.close(fig)

# =====
# Slice-wise metrics (year / season / weekday-weekend)
# =====

rows_tbl = []

# (i) Year rows from backtests
for Y in ys:
    rows_tbl.append({
        "Slice type": "Year",
        "Slice": str(Y),
        "Positive rate": round(posrate_by_year.get(Y, np.nan), 3),
        "ROC-AUC": round(auc_by_year.get(Y, np.nan), 3) if not
np.isnan(auc_by_year.get(Y, np.nan)) else "",
        "PR-AUC": round(prauc_by_year.get(Y, np.nan), 3) if not
np.isnan(prauc_by_year.get(Y, np.nan)) else "",
        "Brier": round(brier_by_year.get(Y, np.nan), 3)
    })

# (ii) Seasons on 2024+
for s in ["DJF","MAM","JJA","SON"]:

```

```

m = (seasons == s)
if m.sum() == 0:
    continue
prob_s = prob_ho[m]
brier_s = brier_score_loss(y_ho[m], prob_s)
auc_s = roc_auc_score(y_ho[m], prob_s) if len(np.unique(y_ho[m]))==2
else np.nan
pr_s = average_precision_score(y_ho[m], prob_s) if
len(np.unique(y_ho[m]))==2 else np.nan
rows_tbl.append({
    "Slice type": "Season (2024+)",
    "Slice": s,
    "Positive rate": round(float(np.mean(y_ho[m])), 3),
    "ROC-AUC": round(auc_s, 3) if not np.isnan(auc_s) else "",
    "PR-AUC": round(pr_s, 3) if not np.isnan(pr_s) else "",
    "Brier": round(float(brier_s), 3)
})

```

(iii) Weekday vs Weekend on 2024+

```

is_weekend = d_ho.dt.dayofweek >= 5
for name, mask in [("Weekday (2024+)", ~is_weekend), ("Weekend (2024+)",
is_weekend)]:
    prob_s = prob_ho[mask]
    brier_s = brier_score_loss(y_ho[mask], prob_s)
    auc_s = roc_auc_score(y_ho[mask], prob_s) if
len(np.unique(y_ho[mask]))==2 else np.nan
    pr_s = average_precision_score(y_ho[mask], prob_s) if
len(np.unique(y_ho[mask]))==2 else np.nan
    rows_tbl.append({
        "Slice type": "Calendar",
        "Slice": name,
        "Positive rate": round(float(np.mean(y_ho[mask])), 3),

```

```

        "ROC-AUC": round(auc_s, 3) if not np.isnan(auc_s) else "",
        "PR-AUC": round(pr_s, 3) if not np.isnan(pr_s) else "",
        "Brier": round(float(brier_s), 3)
    })

table_46 = pd.DataFrame(rows_tbl)
csv_path = OUT/"Table_4_6_Slice_wise_metrics.csv"
table_46.to_csv(csv_path, index=False)
print("Saved:", csv_path.resolve())
table_46

```

H5) Figure 8 - Permutation importance, Figure 9 - SHAP & Table 7 – Case Day

```
# =====

# permutation importance (figure 8), SHAP (figure 9), case-day table
# (table 7)

# =====

import numpy as np, pandas as pd, warnings, matplotlib.pyplot as plt
from pathlib import Path

from sklearn.ensemble import HistGradientBoostingClassifier,
RandomForestClassifier

from sklearn.inspection import permutation_importance

from sklearn.metrics import roc_auc_score

import shap

warnings.filterwarnings("ignore")

plt.rcParams.update({"figure.dpi": 150})

# ----- paths -----

CSV_PATH =
"master_daily_tapsaff_bbq_suncream_mobility_demand_engineered.csv" # <--
your master CSV

CHOSEN_MODEL = "HGB" # "HGB" or "RF"

HGB_PARAMS = dict(learning_rate=0.05, max_depth=3, max_iter=400,
l2_regularization=0.0, random_state=42)

RF_PARAMS = dict(n_estimators=400, max_depth=None, min_samples_leaf=1,
class_weight="balanced",

                    n_jobs=-1, random_state=42)

TAU = 0.946 # <-- your operating threshold (fixed from training)

OUTDIR = Path("outputs_figs_tables"); OUTDIR.mkdir(exist_ok=True,
parents=True)

# -----
```

```

# load & prep

df = pd.read_csv(CSV_PATH,
parse_dates=["date"]).dropna(subset=["taps_aff"]).sort_values("date")

feature_cols = [c for c in df.columns if c not in ("date","taps_aff")]

X_all = df[feature_cols].replace([np.inf,-np.inf],0).fillna(0).to_numpy()
y_all = df["taps_aff"].astype(int).to_numpy()

dates = df["date"]

# Train / hold-out split (no future)

train_mask = dates < pd.Timestamp("2024-01-01")
X_tr, y_tr = X_all[train_mask], y_all[train_mask]
X_ho, y_ho = X_all[~train_mask], y_all[~train_mask]
dates_ho = dates[~train_mask]

X_tr_df, X_ho_df = pd.DataFrame(X_tr, columns=feature_cols),
pd.DataFrame(X_ho, columns=feature_cols)

# Fit the production model on 2018-2023 (pre-calibration model for
interpretability)

if CHOSEN_MODEL.upper() == "RF":
    model = RandomForestClassifier(**RF_PARAMS).fit(X_tr, y_tr)
else:
    model = HistGradientBoostingClassifier(**HGB_PARAMS).fit(X_tr, y_tr)

# Probabilities on 2024+ (for ranking and case selection)

y_prob_ho = (model.predict_proba(X_ho)[: ,1]
              if hasattr(model, "predict_proba") else model.predict(X_ho))
# HGB has predict_proba

# =====

# Top-20 permutation importances (AUC drop on hold-out)

# =====

perm = permutation_importance(

```

```

        model, X_ho, y_ho, scoring="roc_auc", n_repeats=40, random_state=42,
n_jobs=-1
    )
imp_df = (pd.DataFrame({"feature": feature_cols,
                        "mean_auc_drop": perm.importances_mean,
                        "std": perm.importances_std})
          .sort_values("mean_auc_drop", ascending=False)
          .head(20))

plt.figure(figsize=(8,6))
plt.barh(imp_df["feature"][::-1], imp_df["mean_auc_drop"][::-1],
        xerr=imp_df["std"][::-1], alpha=0.9)
plt.xlabel("AUC decrease when permuted (higher = more important)")
plt.title("Top-20 permutation importances")
plt.tight_layout()
fig_47 = OUTDIR / "Permutation_Importances.png"
plt.savefig(fig_47); plt.close()
print("Saved:", fig_47.resolve())

# =====
# Figure 4.8 – SHAP summary (beeswarm) and waterfalls for exemplar days
# Note: explains the *model*; calibration (isotonic) is not explained
# here.
# =====
# Use a small sample of hold-out rows for SHAP speed
np.random.seed(42)
samp_idx = np.random.choice(np.arange(X_ho.shape[0]), size=min(300,
X_ho.shape[0]), replace=False)
X_shap = X_ho_df.iloc[samp_idx]

# Build a robust SHAP explainer

```

```

try:
    if CHOSEN_MODEL.upper() == "RF":
        explainer = shap.TreeExplainer(model)
        shap_vals = explainer.shap_values(X_shap)[1] # class 1
contributions

    else:
        # Generic explainer: uses model.predict_proba; may be slower but
robust

        explainer = shap.Explainer(model.predict_proba, X_tr_df,
algorithm="auto")
        sv = explainer(X_shap)
        shap_vals = sv.values[:, :, 1] # contributions to class 1
except Exception:
    # Fallback to KernelExplainer (slower); use tiny background/sample
    bg = shap.kmeans(X_tr_df, 50)
    explainer = shap.KernelExplainer(model.predict_proba, bg)
    sv = explainer.shap_values(X_shap, nsamples=100) # returns list
[class0, class1]
    shap_vals = sv[1]

# Beeswarm summary
plt.figure()
shap.summary_plot(shap_vals, X_shap, max_display=20, show=False)
plt.title("SHAP summary (2024+)")
fig_48_sum = OUTDIR / "SHAP_summary.png"
plt.tight_layout(); plt.savefig(fig_48_sum, bbox_inches="tight");
plt.close()
print("Saved:", fig_48_sum.resolve())

# Waterfalls for exemplar TP/FP/TN/FN
pred_ho = (y_prob_ho >= TAU).astype(int)
cats = pd.Series(np.where((pred_ho==1)&(y_ho==1), "TP",

```

```

np.where((pred_ho==1)&(y_ho==0), "FP",
np.where((pred_ho==0)&(y_ho==0), "TN","FN"))))

# Choose one confident example of each (if present)
def pick_index(mask):
    idx = np.where(mask)[0]
    if idx.size == 0: return None
    # choose the most confident (furthest from 0.5 prob)
    return idx[np.argmax(np.abs(y_prob_ho[idx]-0.5))]

idx_tp = pick_index(cats.values=="TP")
idx_fp = pick_index(cats.values=="FP")
idx_tn = pick_index(cats.values=="TN")
idx_fn = pick_index(cats.values=="FN")
chosen = [("TP", idx_tp), ("FP", idx_fp), ("TN", idx_tn), ("FN", idx_fn)]

# SHAP values must align to the SHAP sample we computed
# Map chosen hold-out indices to nearest sampled row for plotting
def nearest_in_sample(i):
    if i is None: return None
    # crude nearest by absolute prob difference (faster than distance)
    j = np.argmin(np.abs(y_prob_ho[samp_idx] - y_prob_ho[i]))
    return j

for label, idx in chosen:
    j = nearest_in_sample(idx)
    if j is None:
        print(f"No {label} example available.")
        continue
    # Build Explanation object if we used the generic explainer

```



```

try:
    exp = shap.Explanation(
        values=shap_vals[j],
        base_values=(explainer.expected_value[1]
                     if isinstance(explainer.expected_value, (list,
np.ndarray)) else explainer.expected_value),
        data=X_shap.iloc[j].values,
        feature_names=list(X_shap.columns)
    )
    shap.plots.waterfall(exp, max_display=12, show=False)
except Exception:
    # Fallback classic waterfall
    shap.waterfall_plot(shap.Explanation(values=shap_vals[j],

base_values=np.mean(y_prob_ho[samp_idx]),

                                data=X_shap.iloc[j].values,

feature_names=list(X_shap.columns)),

                        max_display=12, show=False)
    plt.title(f"Figure 4.8 – Waterfall ({label})")
    path = OUTDIR / f"Figure_4_8_Waterfall_{label}.png"
    plt.tight_layout(); plt.savefig(path, bbox_inches="tight");
plt.close()
    print("Saved:", path.resolve())

# =====
# Case-day narratives (date, pred, outcome, top contributors)
# =====
def top_contributors(shap_row, cols, k=5):
    order = np.argsort(np.abs(shap_row))[:, -1][:k]
    return "; ".join([f"{cols[i]} ({shap_row[i]:+.3f})" for i in order])

```

```

rows = []
for label, idx in chosen:
    if idx is None:
        continue

    # Map to sampled row for SHAP contributions
    j = nearest_in_sample(idx)
    top5 = top_contributors(shap_vals[j], list(X_shap.columns), k=5)
    rows.append({
        "Date": str(dates_ho.iloc[idx].date()),
        "Pred prob": round(float(y_prob_ho[idx]), 3),
        "Pred label@τ": int(pred_ho[idx]),
        "True label": int(y_ho[idx]),
        "Case": label,
        "Top contributing features (SHAP)": top5
    })

table_47 = pd.DataFrame(rows)
path_47 = OUTDIR / "Case_Day_Narratives.csv"
table_47.to_csv(path_47, index=False)
print("Saved:", path_47.resolve())

display(imp_df)
display(table_47)
print(f"AUC on 2024+ for sanity: {roc_auc_score(y_ho, y_prob_ho):.3f}")

```

H6) Figure 10 - Precision–Recall vs threshold (chosen point), Table 8 - Error taxonomy

```
# =====  
# (figure 10) Precision–Recall vs threshold (chosen point)  
# Table 8) – Error taxonomy with counts/examples  
# =====  
  
import warnings, numpy as np, pandas as pd  
from pathlib import Path  
from sklearn.base import clone  
from sklearn.metrics import (  
    roc_auc_score, precision_recall_curve,  
    accuracy_score, precision_score, recall_score, f1_score,  
    confusion_matrix  
)  
from sklearn.ensemble import RandomForestClassifier,  
HistGradientBoostingClassifier  
from sklearn.calibration import CalibratedClassifierCV  
import matplotlib.pyplot as plt  
  
warnings.filterwarnings("ignore")  
  
# -----  
# Paths & parameters  
# -----  
  
MASTER_CSV =  
"master_daily_tapsaff_bbq_suncream_mobility_demand_engineered.csv" # <-  
adjust if needed  
  
OUT = Path("outputs"); OUT.mkdir(parents=True, exist_ok=True)  
  
BETA = 0.5 # F_beta used to choose  $\tau$  on TRAIN ( $\beta < 1$  favors precision;  $> 1$   
favors recall)  
  
HOLDOUT_START = pd.Timestamp("2024-01-01")
```

```

# -----
# Load
# -----

df = (pd.read_csv(MASTER_CSV, parse_dates=["date"])
      .dropna(subset=["taps_aff"])
      .sort_values("date"))

feature_cols = [c for c in df.columns if c not in ("date", "taps_aff")]
X_full = df[feature_cols].replace([np.inf, -np.inf],
0).fillna(0).to_numpy()
y_full = df["taps_aff"].astype(int).to_numpy()
dates = df["date"]

train_mask = dates < HOLDOUT_START
X_tr, y_tr, d_tr = X_full[train_mask], y_full[train_mask],
dates[train_mask]
X_ho, y_ho, d_ho = X_full[~train_mask], y_full[~train_mask],
dates[~train_mask]

# -----
# Expanding, year-aligned folds (2019..2023)
# -----

def year_folds(dates, y):
    years = pd.to_datetime(dates).dt.year.to_numpy()
    folds = []
    for Y in range(2019, 2024):
        tr_idx = np.where(years < Y)[0]
        va_idx = np.where(years == Y)[0]
        if va_idx.size and np.unique(y[va_idx]).size > 1:
            folds.append((tr_idx, va_idx))

```

```

        return folds

cv = year_folds(d_tr, y_tr)

# -----
# Pick a primary model by mean CV ROC-AUC
# -----
rf = RandomForestClassifier(
    n_estimators=400, max_depth=None, min_samples_leaf=1,
    class_weight="balanced", n_jobs=1, random_state=42
)

hgb = HistGradientBoostingClassifier(
    learning_rate=0.05, max_depth=3, max_iter=400,
    l2_regularization=0.0, random_state=42
)

def mean_cv_auc(model, X, y, folds):
    aucs = []
    for tr_idx, va_idx in folds:
        m = clone(model).fit(X[tr_idx], y[tr_idx])
        p = m.predict_proba(X[va_idx])[:,1]
        aucs.append(roc_auc_score(y[va_idx], p))
    return float(np.mean(aucs))

auc_rf = mean_cv_auc(rf, X_tr, y_tr, cv)
auc_hgb = mean_cv_auc(hgb, X_tr, y_tr, cv)
best_model = hgb if auc_hgb >= auc_rf else rf
best_name = "HGB" if best_model is hgb else "RF"
print(f"Selected model: {best_name} | mean CV AUC (2019..23) =
{max(auc_rf, auc_hgb):.3f}")

```

```

# -----
# Choose a TRAIN threshold  $\tau$  via out-of-fold  $F_{\text{beta}}$ 
# -----
oof = np.full_like(y_tr, np.nan, dtype=float)
for tr_idx, va_idx in cv:
    m = clone(best_model).fit(X_tr[tr_idx], y_tr[tr_idx])
    oof[va_idx] = m.predict_proba(X_tr[va_idx])[:,1]

prec, rec, thr = precision_recall_curve(y_tr[np.isfinite(oof)],
oof[np.isfinite(oof)])

fb = (1 + BETA**2) * (prec[:-1] * rec[:-1]) / (BETA**2 * prec[:-1] +
rec[:-1] + 1e-12)

tau = float(thr[np.nanargmax(fb)])
print(f"Chosen threshold  $\tau$  ( $F_{\text{BETA}}$  on train): {tau:.3f}")

# -----
# Fit on TRAIN and (optionally) calibrate within TRAIN
# -----
# Split a small calibration tail from training (last ~180 days; relax if
class is single)
cal_mask = (d_tr >= (d_tr.max() - pd.Timedelta(days=180)))
if np.unique(y_tr[cal_mask]).size < 2:
    cal_mask = (d_tr >= (d_tr.max() - pd.Timedelta(days=365)))
if np.unique(y_tr[cal_mask]).size < 2:
    cal_mask = (d_tr.dt.year == d_tr.max().year)

base = clone(best_model).fit(X_tr[~cal_mask], y_tr[~cal_mask])
y_prob_pre = base.predict_proba(X_ho)[:,1]

y_prob_post = None
if np.unique(y_tr[cal_mask]).size == 2:

```

```

    calib = CalibratedClassifierCV(base, method="isotonic",
cv="prefit").fit(X_tr[cal_mask], y_tr[cal_mask])

    y_prob_post = calib.predict_proba(X_ho)[:,-1]

final_prob = y_prob_post if y_prob_post is not None else y_prob_pre
postfix    = "(post-cal)" if y_prob_post is not None else "(pre-cal)"
print(f"Scoring hold-out with {best_name} {postfix}")

# -----
# Precision-Recall vs threshold (hold-out)
# -----

prec_ho, rec_ho, thr_ho = precision_recall_curve(y_ho, final_prob)
# Find the PR point nearest to  $\tau$ 
idx_tau = np.abs(thr_ho - tau).argmin() if len(thr_ho) else None
pr_at_tau = (rec_ho[idx_tau], prec_ho[idx_tau]) if idx_tau is not None
else (np.nan, np.nan)

plt.figure(figsize=(7,5))
plt.plot(rec_ho, prec_ho)
if idx_tau is not None:
    plt.scatter([pr_at_tau[0]], [pr_at_tau[1]])
    plt.axvline(pr_at_tau[0], linestyle="--")
    plt.axhline(pr_at_tau[1], linestyle="--")
plt.xlabel("Recall")
plt.ylabel("Precision")
plt.title("Precision-Recall vs threshold (2024+)")
fig14_path = OUT / "PR_vs_threshold.png"
plt.tight_layout(); plt.savefig(fig14_path, dpi=150); plt.close()
print("Saved:", fig14_path.resolve())

# -----

```

```

# FAMILY SCORES (robust momentum/context) for error taxonomy
# -----
# Build a DataFrame of hold-out features aligned to dates
X_ho_df = (df.loc[~train_mask, feature_cols]
            .replace([np.inf, -np.inf], 0)
            .fillna(0))

# Define column families
intent_cols = [c for c in feature_cols
               if any(k in c.lower() for k in
["suncream", "bbq", "search", "trend", "beer_garden", "tsd_"])]
mobility_cols = [c for c in feature_cols if c.lower().startswith(("ped_",
"cycle_", "bike_"))]
demand_cols    = [c for c in feature_cols if c.lower().startswith(("gb_",
"nd_"))]

def family_score(df_slice, cols):
    """
    Per-row score for a family: median robust z across that family's
    columns.

    z = (x - median) / MAD (column-wise). Uses small epsilon if MAD==0.
    """
    if not cols:
        return pd.Series(0.0, index=df_slice.index)
    Xfam = df_slice[cols].apply(pd.to_numeric,
errors="coerce").fillna(0.0)
    med = Xfam.median()
    mad = (Xfam.sub(med).abs().median()).replace(0, 1e-9)
    Z = Xfam.sub(med).div(mad)
    return Z.median(axis=1)

intent_mom    = family_score(X_ho_df, intent_cols).rename("intent_mom")

```



```

mobility_mom = family_score(X_ho_df, mobility_cols).rename("mobility_mom")

demand_ctx    = family_score(X_ho_df, demand_cols).rename("demand_ctx")  #
higher ~ more winter-like load

# -----
# Error taxonomy
# -----

y_pred = (final_prob >= tau).astype(int)

tax_df = pd.DataFrame({
    "date": d_ho.values,
    "y_true": y_ho,
    "y_prob": final_prob,
    "y_pred": y_pred,
    "intent_mom": intent_mom.values,
    "mobility_mom": mobility_mom.values,
    "demand_ctx": demand_ctx.values
})

tax_df["weekend"] = tax_df["date"].dt.dayofweek >= 5
tax_df["month"]   = tax_df["date"].dt.month
shoulder_months = {4,5,9}  # Apr, May, Sep
tax_df["shoulder"] = tax_df["month"].isin(shoulder_months)

# Define high/low cut points from hold-out distribution
q_int_hi = tax_df["intent_mom"].quantile(0.75)
q_int_lo = tax_df["intent_mom"].quantile(0.25)
q_mob_hi = tax_df["mobility_mom"].quantile(0.75)
q_dem_hi = tax_df["demand_ctx"].quantile(0.75)  # "winter-ish" when high

intent_hi = tax_df["intent_mom"] >= q_int_hi
intent_lo = tax_df["intent_mom"] <= q_int_lo

```

```

mob_hi    = tax_df["mobility_mom"] >= q_mob_hi
dem_hi    = tax_df["demand_ctx"] >= q_dem_hi

FN = (tax_df["y_true"]==1) & (tax_df["y_pred"]==0)
FP = (tax_df["y_true"]==0) & (tax_df["y_pred"]==1)

# Simple, interpretable buckets
cond_FN_shoulder = FN & tax_df["shoulder"] & (~intent_hi) & (~mob_hi)
cond_FN_other    = FN & (~cond_FN_shoulder)

cond_FP_intent_only = FP & intent_hi & (~mob_hi)
cond_FP_mob_event   = FP & mob_hi & (~intent_hi) & tax_df["weekend"]
cond_FP_other       = FP & (~cond_FP_intent_only) & (~cond_FP_mob_event)

def ex_dates(mask, k=3):
    return ", ".join(tax_df.loc[mask, "date"].dt.strftime("%Y-%m-%d").head(k).tolist())

rows = [
    {"Type": "FN", "Rule": "Shoulder late-warming (Apr/May/Sep), low intent & low mobility",
     "Count": int(cond_FN_shoulder.sum()),
     "Example dates": ex_dates(cond_FN_shoulder),
     "Suggested mitigation": "12:00 refresh + two-family confirmation"},
    {"Type": "FN", "Rule": "Other missed positives",
     "Count": int(cond_FN_other.sum()),
     "Example dates": ex_dates(cond_FN_other),
     "Suggested mitigation": "Check feeds; consider  $\Delta 1/\Delta 3$  features"},
    {"Type": "FP", "Rule": "Intent-only spike (high intent, low mobility)",
     "Count": int(cond_FP_intent_only.sum()),
     "Example dates": ex_dates(cond_FP_intent_only),

```

```

        "Suggested mitigation": "Require intent+mobility confirmation"},
        {"Type": "FP", "Rule": "Event-like mobility on weekend (high mobility,
low intent)",
        "Count": int(cond_FP_mob_event.sum()),
        "Example dates": ex_dates(cond_FP_mob_event),
        "Suggested mitigation": "Light events calendar / hold at amber"},
        {"Type": "FP", "Rule": "Other false alarms",
        "Count": int(cond_FP_other.sum()),
        "Example dates": ex_dates(cond_FP_other),
        "Suggested mitigation": "Monitor; inspect feature drift"}
    ]
table7 = pd.DataFrame(rows)

# Save CSV
t7_path = OUT / "Error_Taxonomy.csv"
table7.to_csv(t7_path, index=False)
display(table7)
print("Saved:", t7_path.resolve())

# also print the top-line hold-out metrics at  $\tau$ 
acc = accuracy_score(y_ho, y_pred)
prec_at_tau = precision_score(y_ho, y_pred, zero_division=0)
rec_at_tau = recall_score(y_ho, y_pred, zero_division=0)
f1_at_tau = f1_score(y_ho, y_pred, zero_division=0)
tn, fp, fn, tp = confusion_matrix(y_ho, y_pred).ravel()
print(f"\nHold-out @  $\tau$ ={tau:.3f} | Acc={acc:.3f} Prec={prec_at_tau:.3f}
Rec={rec_at_tau:.3f} F1={f1_at_tau:.3f}")
print(f"TP={tp} FP={fp} TN={tn} FN={fn}")

```